

# *Udemy Course on Amazon Bedrock & Generative AI Beginner to Advanced*



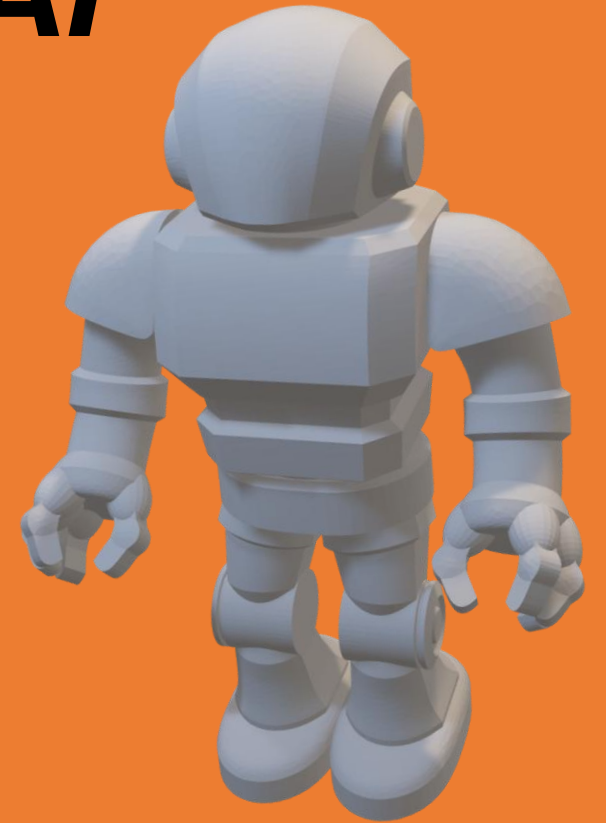
Image Generated using the use case built in this course

# Course Outline

---

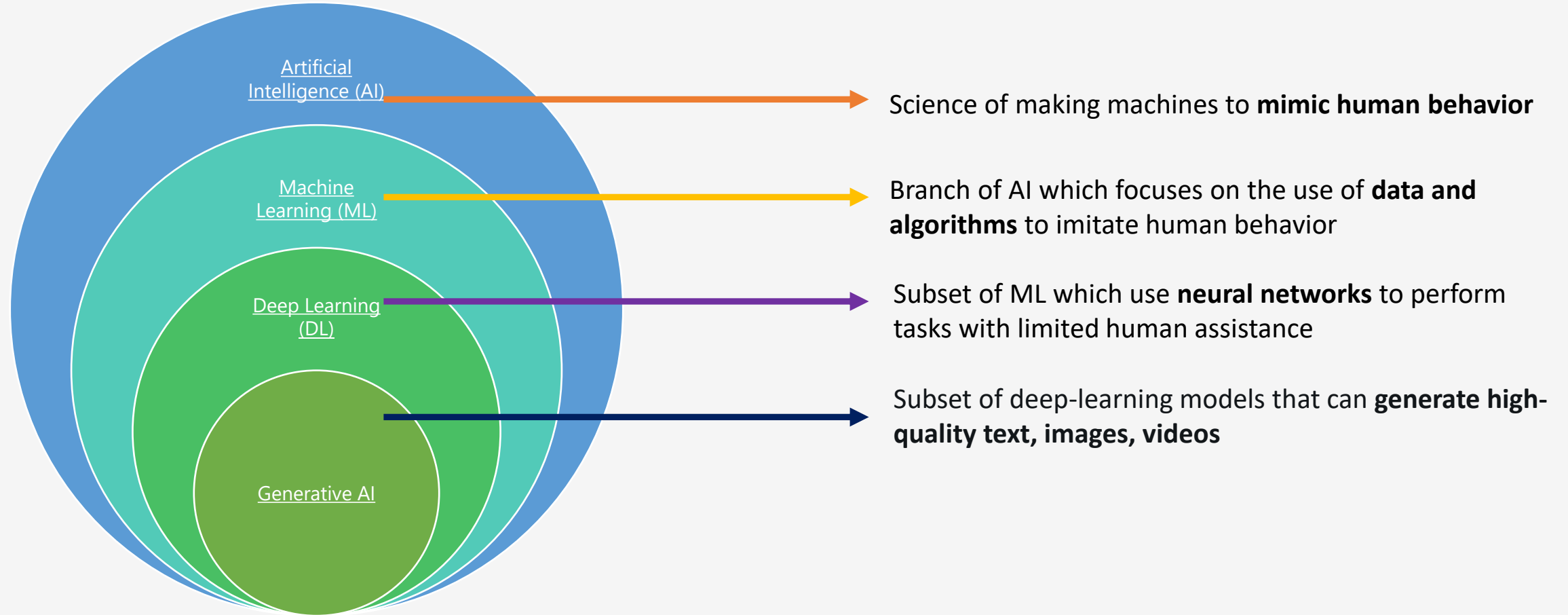
1. *Evolution of Generative AI*
2. *Generative AI Fundamentals*
3. *Overview of Amazon Bedrock*
4. *Use Case - GenAI-Powered Equipment SME Assistant*
5. *Use Case - Building a Chatbot*
6. *Use Case - Employee HR Q & A Application with RAG*
7. *Use Case - Serverless e-Learning App using Knowledge Base for Bedrock*
8. *Use Case - Build Retail Banking Agent*
9. *Refresher – Python, AWS Lambda and API Gateway*

# ***Evolution of Generative AI***



# What is Generative Artificial Intelligence – Context ?

---



# Artificial Intelligence - Overview

---



Artificial Intelligence is the science and engineering of **making** intelligent machines, especially **intelligent computer programs** that **simulate human intelligence and decision making.**

# Artificial Intelligence - Examples

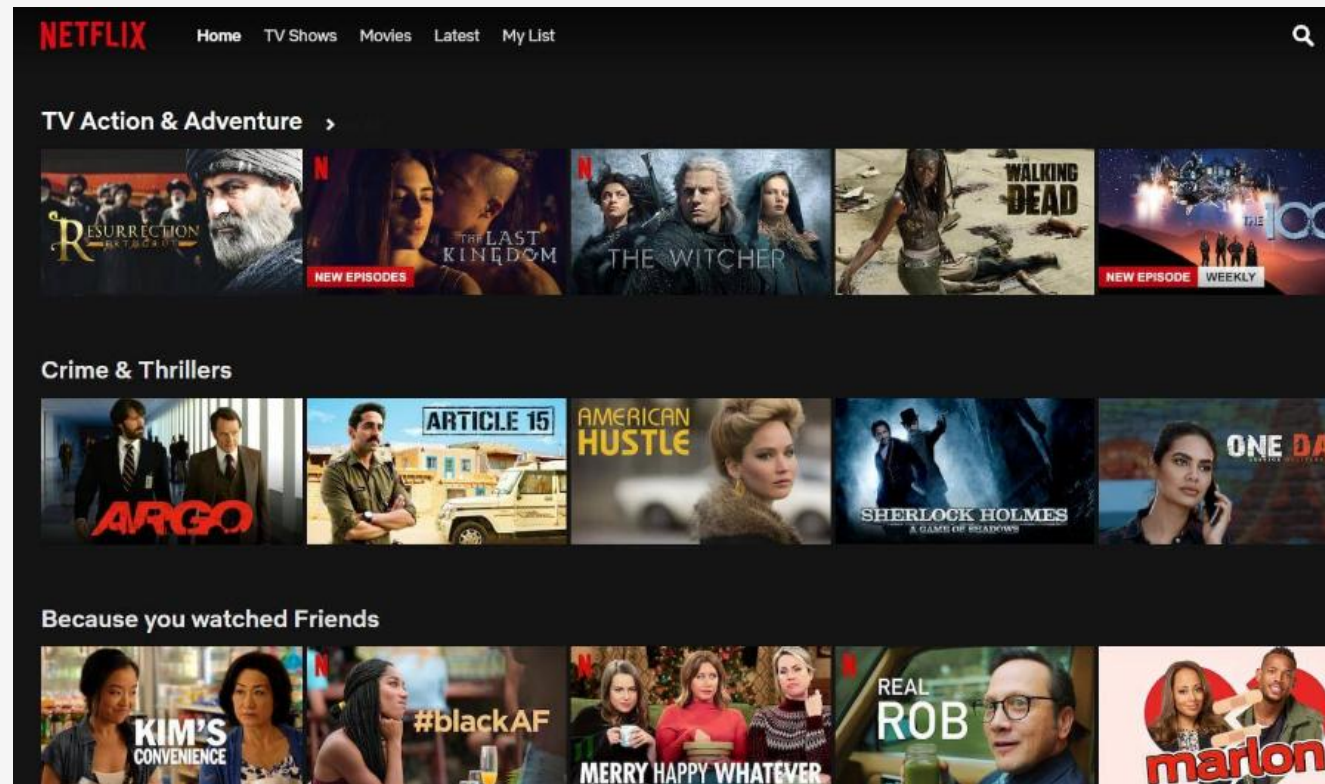
---



**Self-driving cars** use artificial intelligence to make real-time decisions based on sensor data.



# Artificial Intelligence - Examples



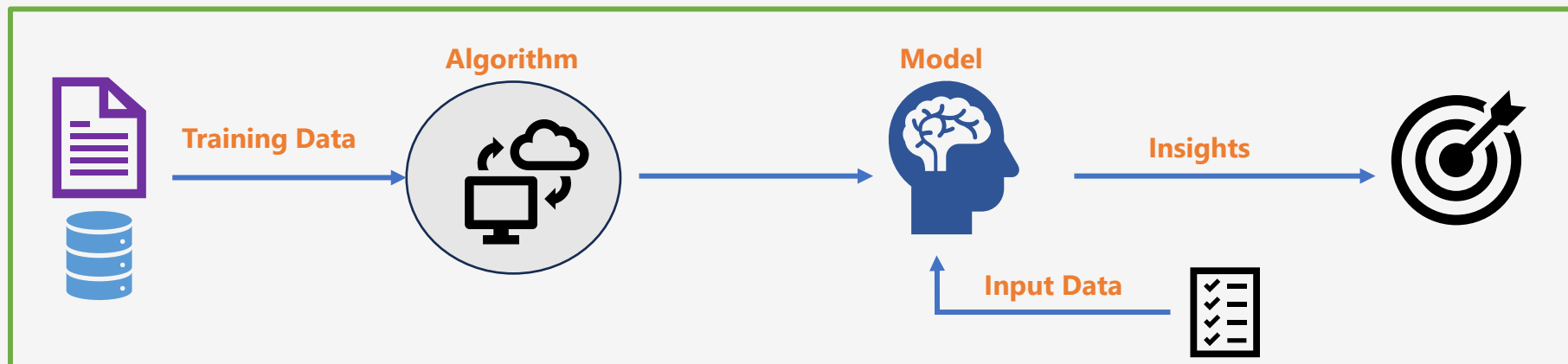
**Recommendation engines** such as ones from **Netflix and Amazon** use AI to provide custom insights

# Machine Learning - Overview

---



- Machine Learning is a **subset of AI**
- In ML the **algorithm** is trained using the **historical data** to make **predictions on new data**.





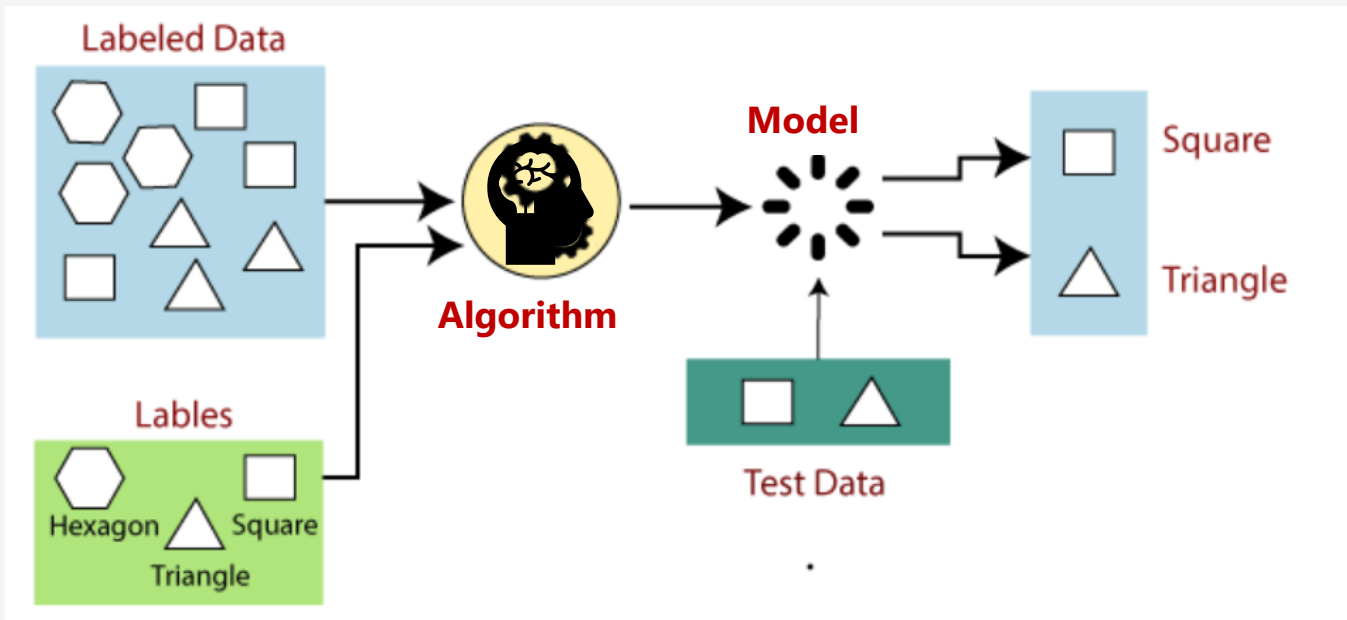
# Machine Learning – Broad Categorization

---



# Machine Learning – Supervised Learning

- In supervised learning, **algorithms are trained using labelled dataset**.
- Once the **training is completed**, the **model is tested using test data** and then it predicts the output.



## Use Case (Classification) – Gmail Spam Filter



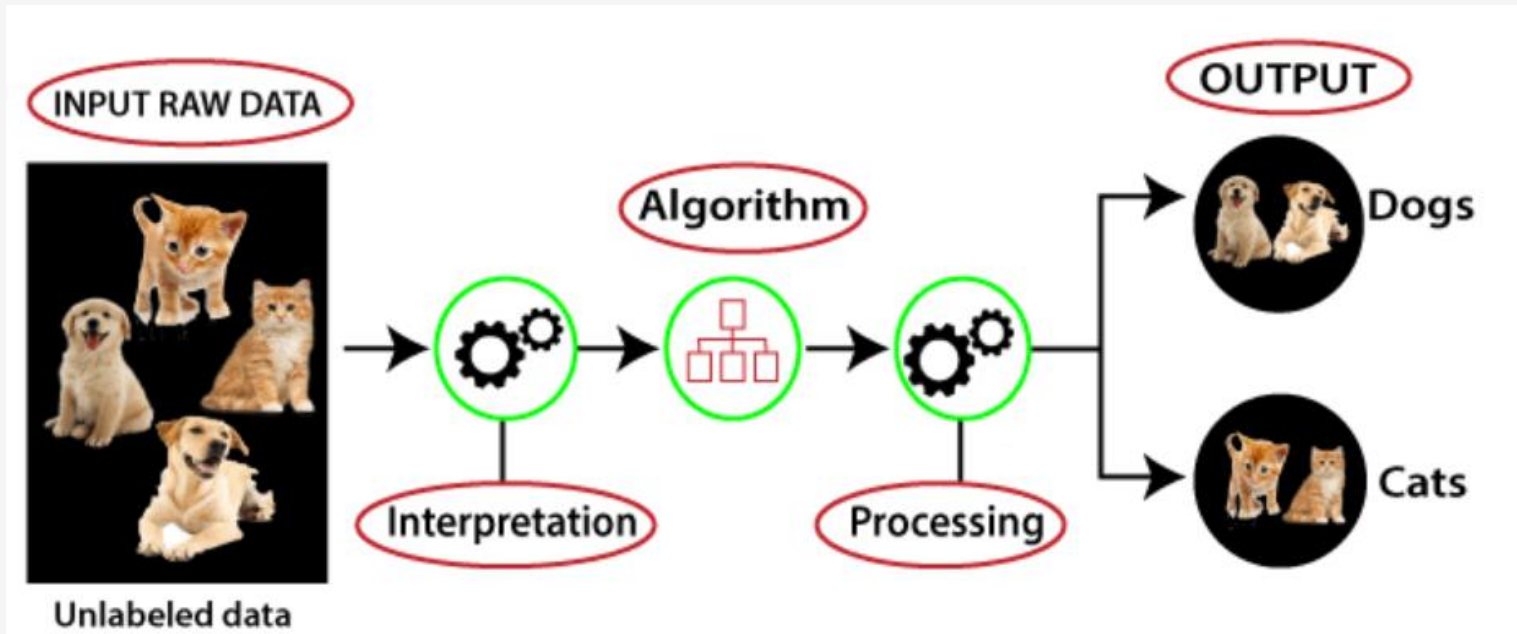
| Advertisement | Sales  |
|---------------|--------|
| \$90          | \$1000 |
| \$120         | \$1300 |
| \$150         | \$1800 |
| \$100         | \$1200 |
| \$130         | \$1380 |
| \$200         | ??     |

## Use Case (Regression) – Forecasting

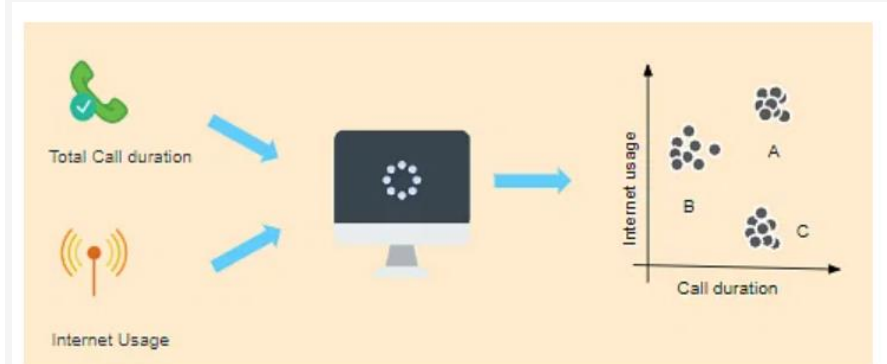
## Common Algorithms - Classification and Regression

# Machine Learning – Unsupervised Learning

- Unsupervised learning is a type of machine learning in which **algorithm are trained using unlabeled dataset**
- Based on the dataset, model finds **hidden patterns and insights** based on the **similarities of the data**



## Use Case (Clustering) – Reduce Churn Rate



## Use Case (Association) – Market Basket Analysis

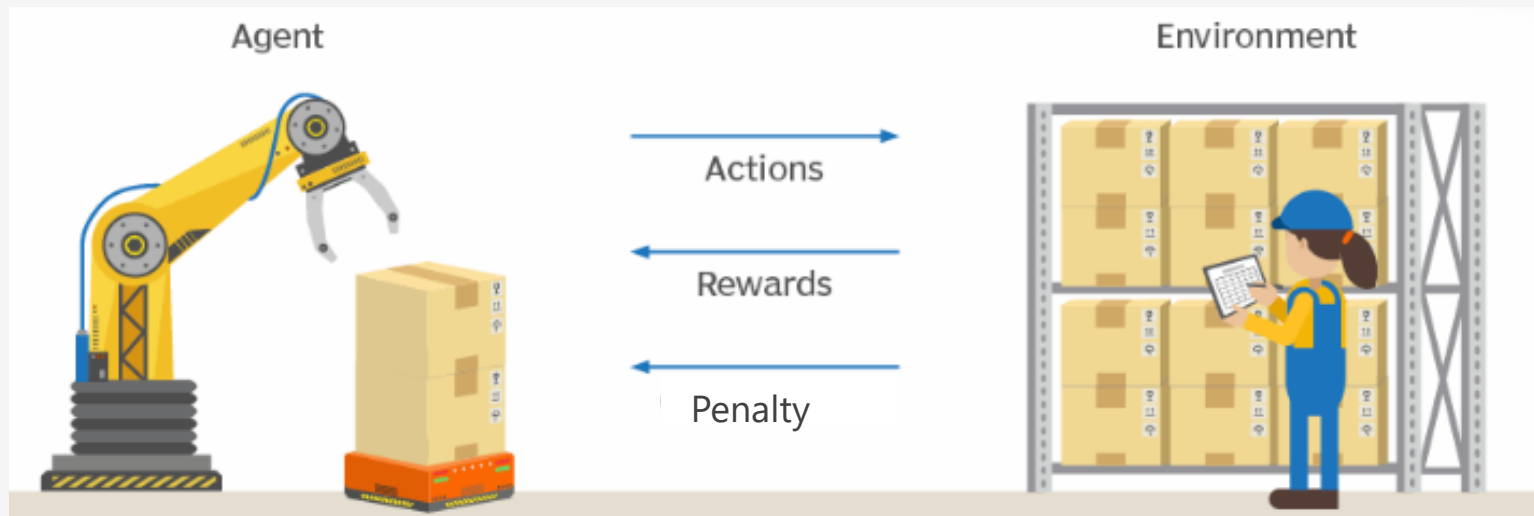


Common Algorithms - **Clustering** and **Association**

# Machine Learning – Reinforced Learning



- Reinforcement learning (RL) is a ML technique that focuses on **training an algorithm following the trial and error approach**.
- No pre-defined dataset
- The **algorithm (agent) evaluates a current situation** (state), **takes an action**, and **receives feedback** (reward and feedback) from the environment after each act.



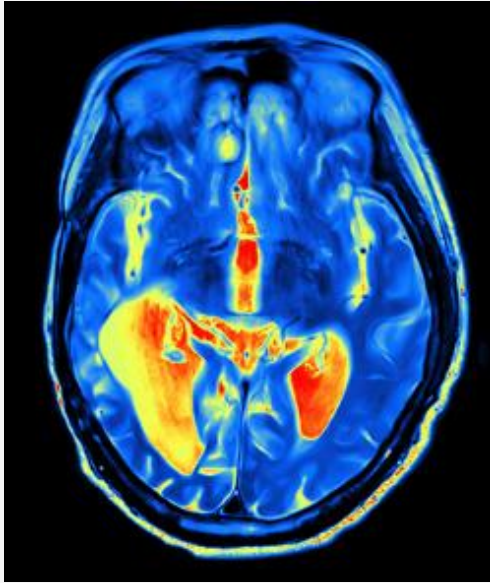
## Real World Use Case

- Training Industrial Robots
- Training Self Driving Cars

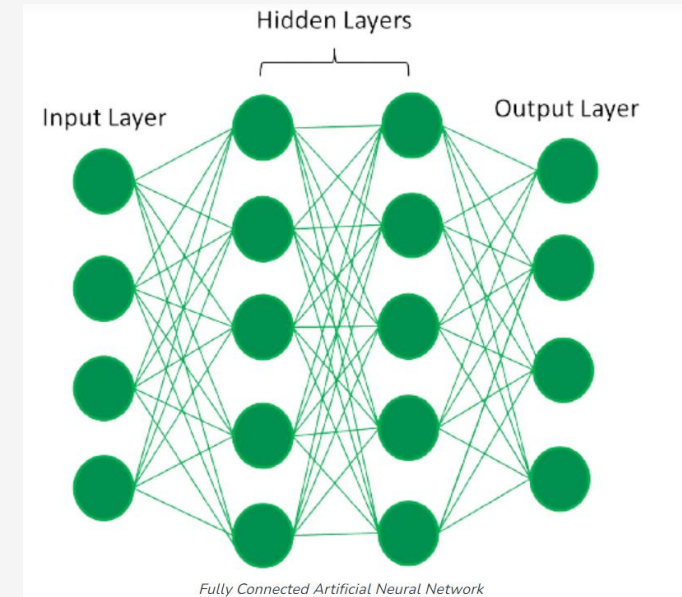
# Machine Learning – Deep Learning and Artificial Neural Networks

---

## Neural Pathways in Human Brain

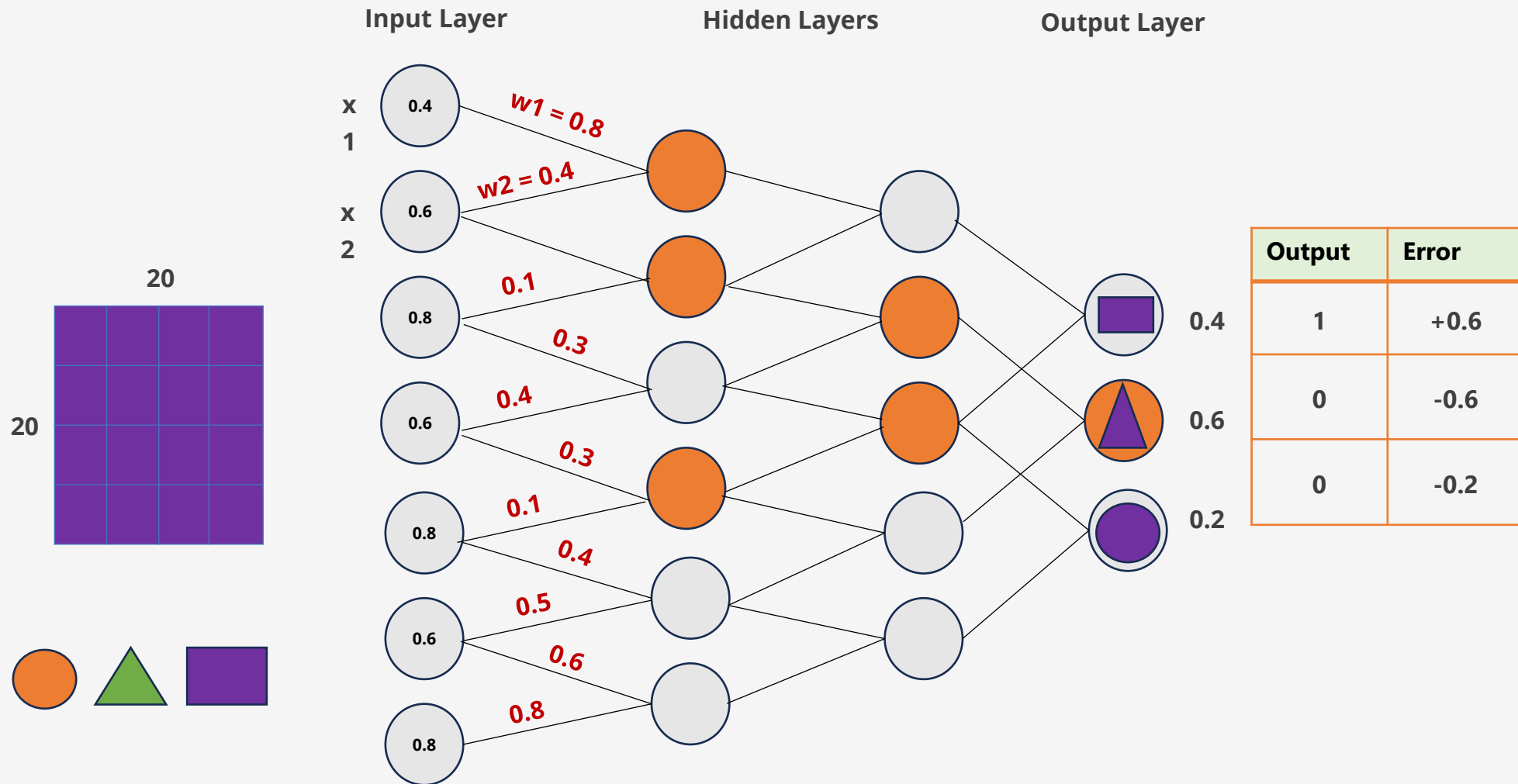


## Artificial Neural Network

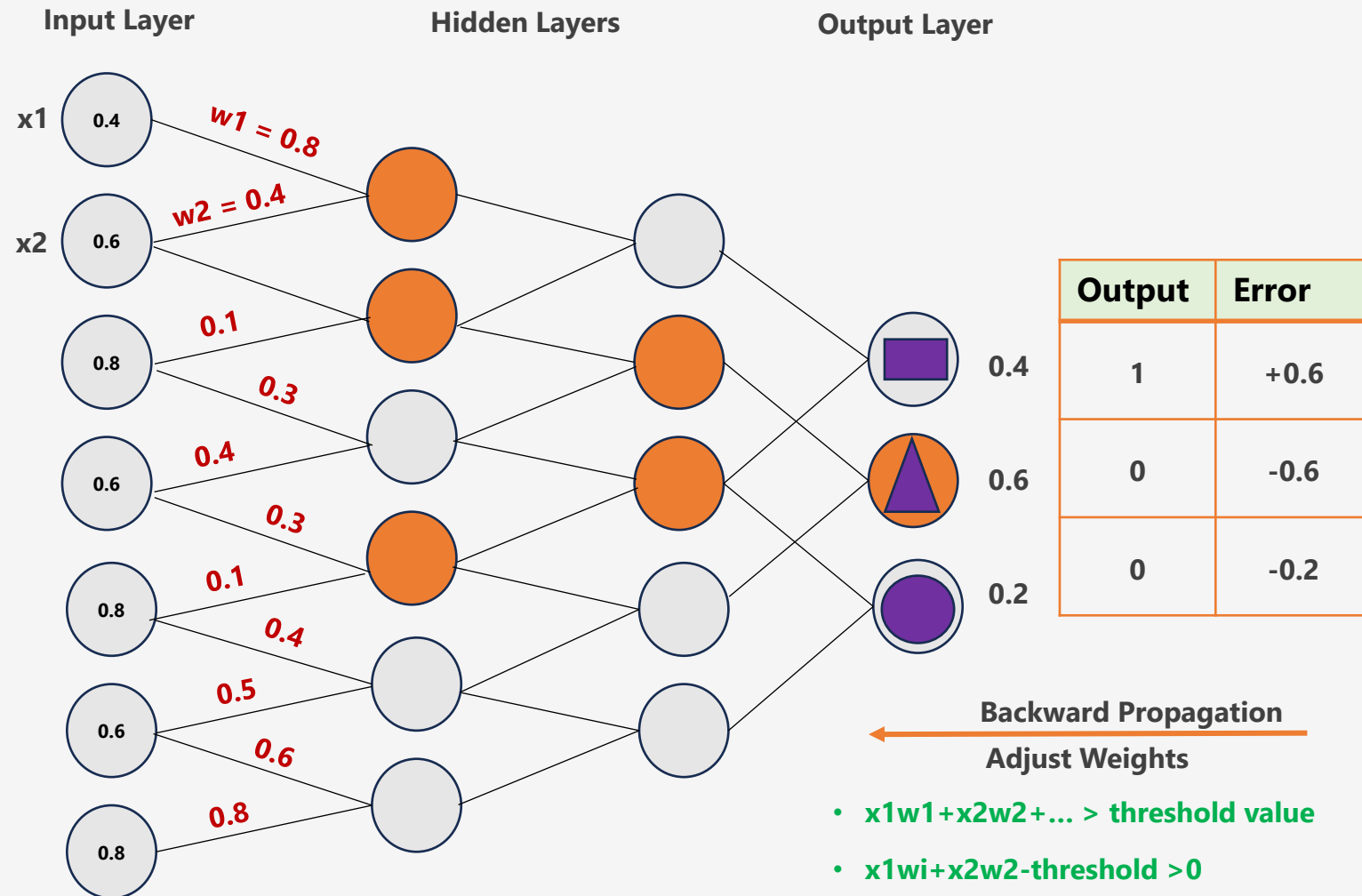
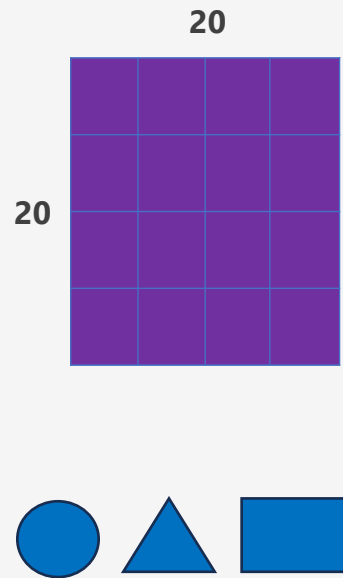


- **Artificial Neural networks** are designed to **mimic the structure of the human brain**
- Deep Learning is a subset of Machine Learning that **focuses on building artificial neural networks that can learn from data.**
- Neural network with **multiple hidden layers and multiple nodes** in each hidden layer is known as a **deep learning (>3)**

# Machine Learning – Deep Learning with Artificial Neural Networks



# Machine Learning – Deep Learning with Artificial Neural Networks



## Key terms

- Neurons (Binary Value)
- Weights
- Bias/Threshold
- Activation Function
- Feedforward network
- Actual Output
- Error
- Forward Propagation
- Backward Propagation

Backward Propagation  
Adjust Weights

- $x_1w_1 + x_2w_2 + \dots > \text{threshold value}$
- $x_1w_i + x_2w_2 - \text{threshold} > 0$
- $x_1w_1 + x_2w_2 + \text{Bias} >>> \text{Activation Function}$



# Generative Artificial Intelligence(AI)

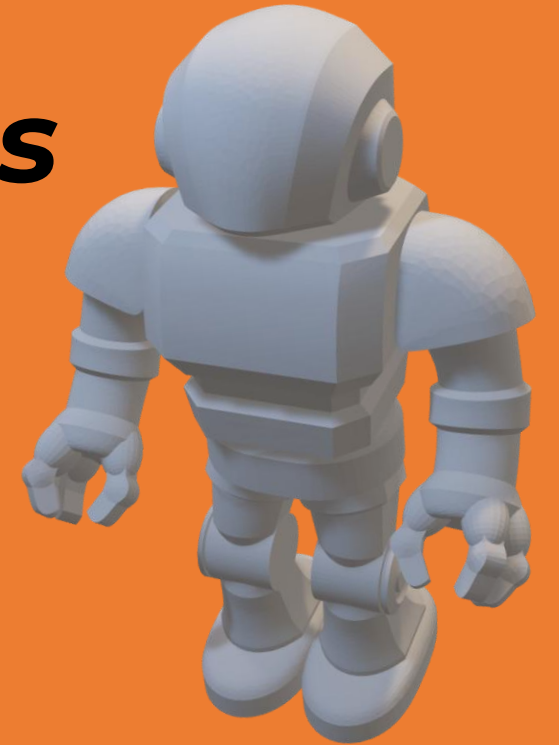
---



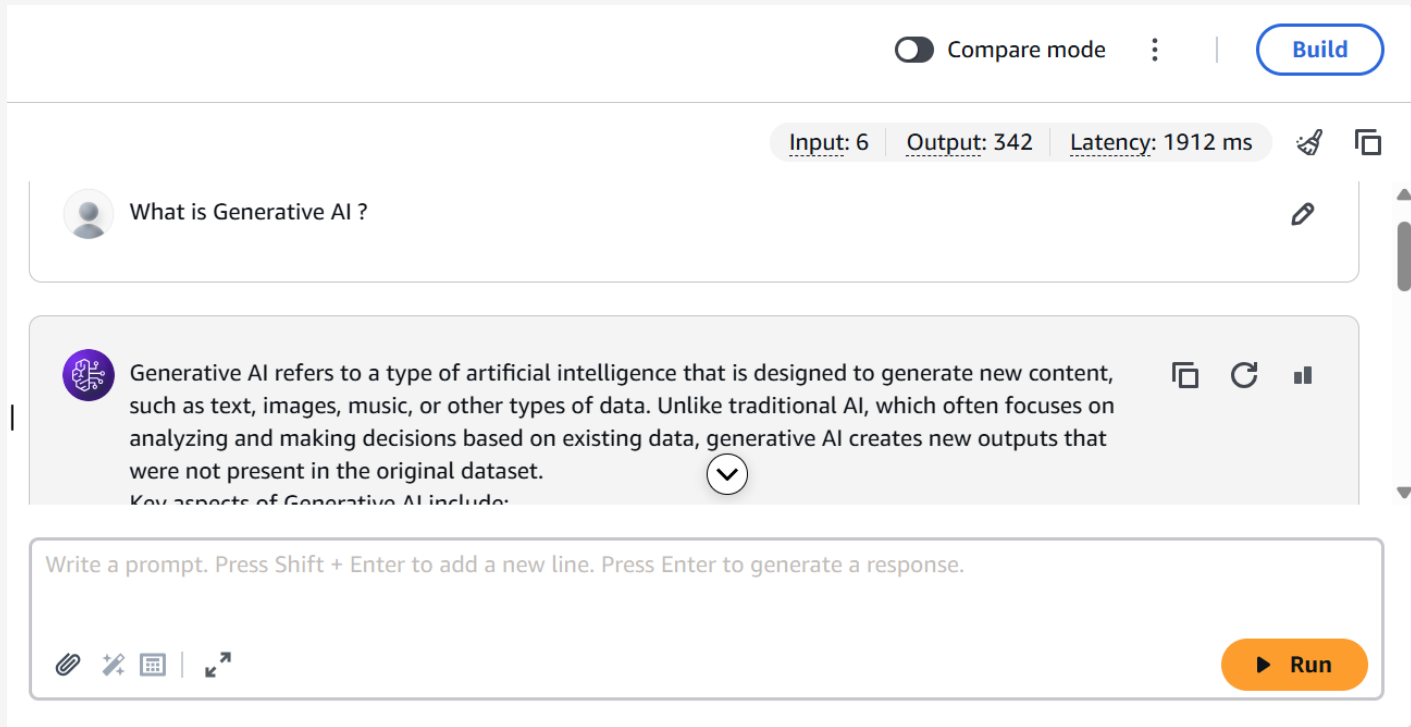
## Generative AI

- Generative AI is a **subset of Deep Learning** which focuses on **creating models** capable of **generating new content** :
  - Chat and Text
  - Images and Videos
  - Code Generation

# ***Generative AI Fundamentals***



# What is Generative AI ?



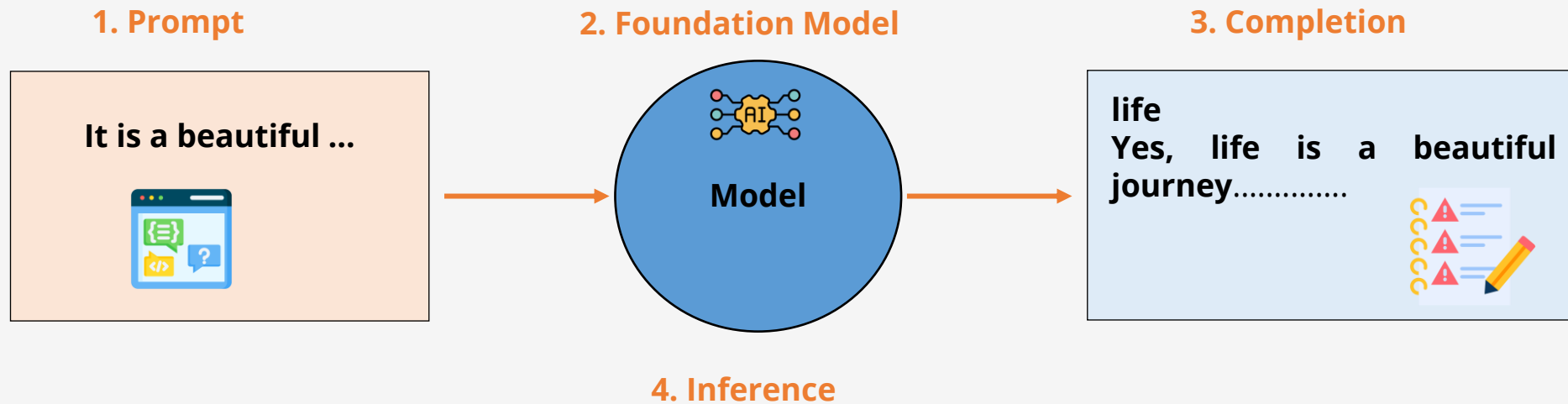
Answer generated **using Amazon Nova Model** from **Amazon Bedrock Service**

## Generative AI refers to :

- Branch of AI that use **deep-learning models**
- **Generate high-quality text, images, code** based on the data they were trained on
- Popular examples of generative AI include
  - GPT from OpenAI (ChatGPT app)
  - Claude from Anthropic
  - Nova Models from AWS

# How Generative AI works - Basic Concepts and Terminology - 1

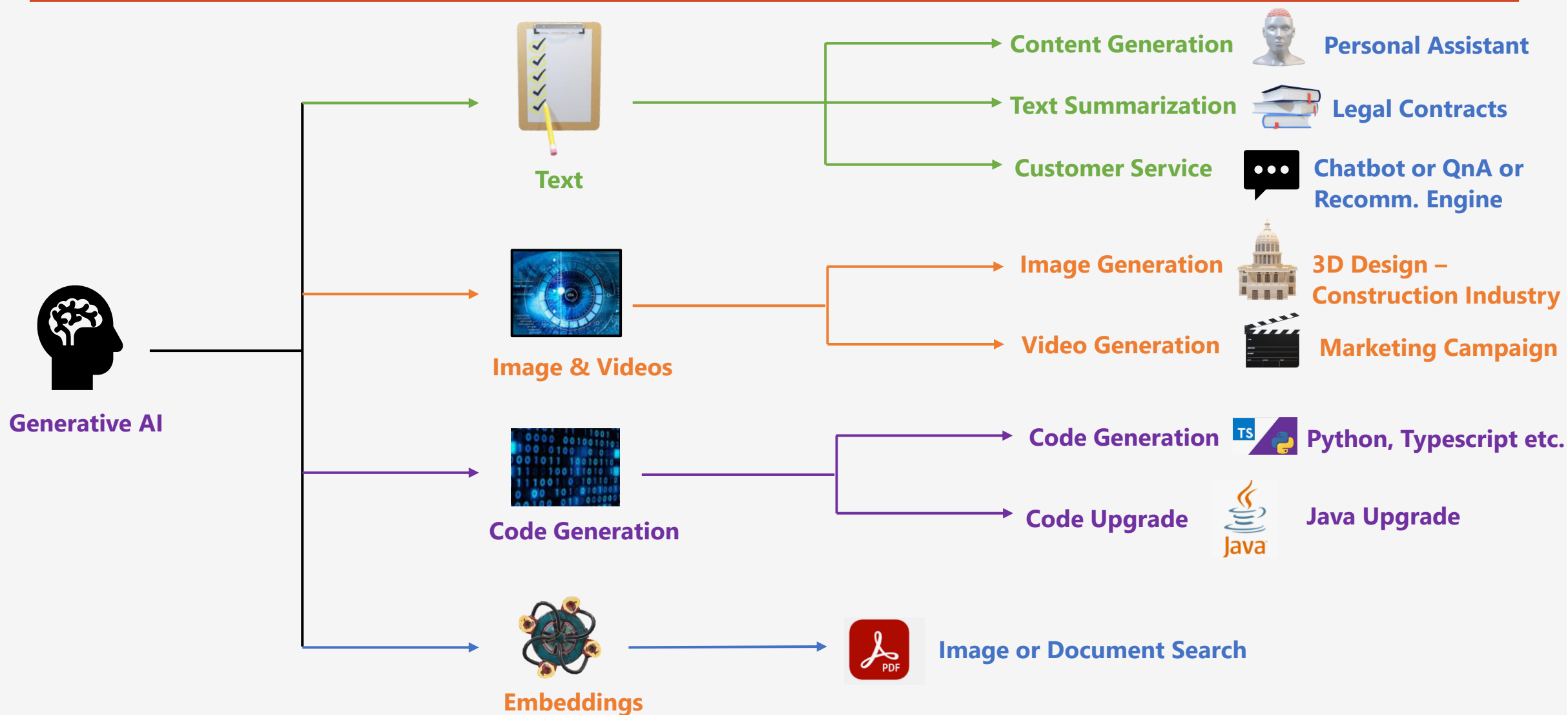
---



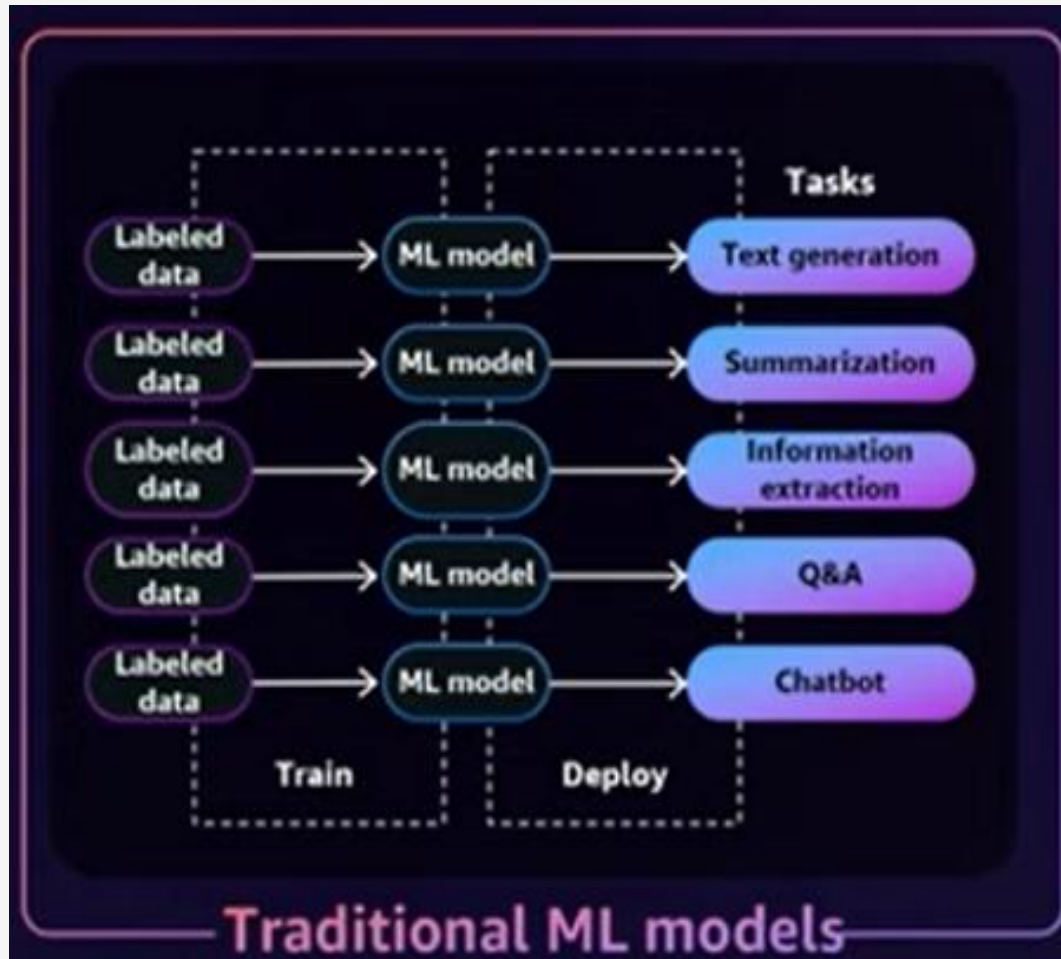
## GenAI Terminology

1. Prompt – Input provided to Model
2. Model/ Foundation Model – GPT, Claude
3. Completion – Output of Model
4. Inference - Act of using model to generate text is called Inference

# Generative AI – Use Cases



# Challenges in Traditional Machine Learning Models



- Each Model focused on **specific task** such as



Text Generation



Summarization

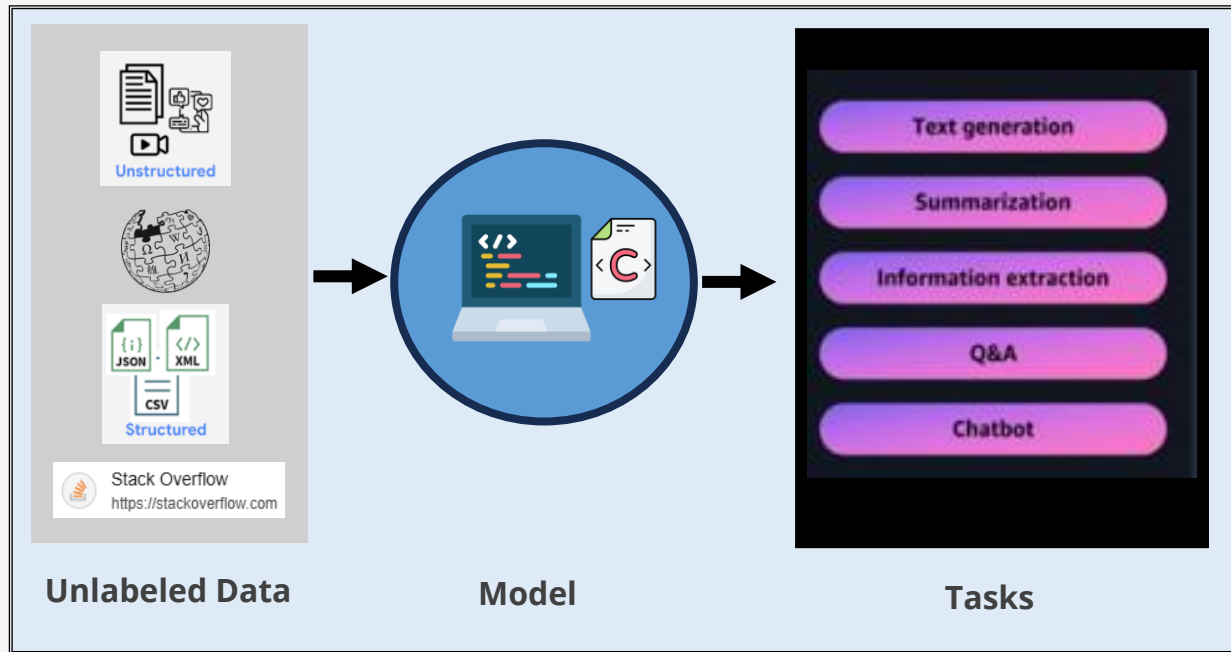


Q&A

- Needs labeled data which requires human intervention and can be cost prohibitive



# Key Characteristics of Foundational Models



Foundation Model

Organization's can utilize **existing FMs as the base for building** task-specific models and **Adapt** them

## Key Characteristics

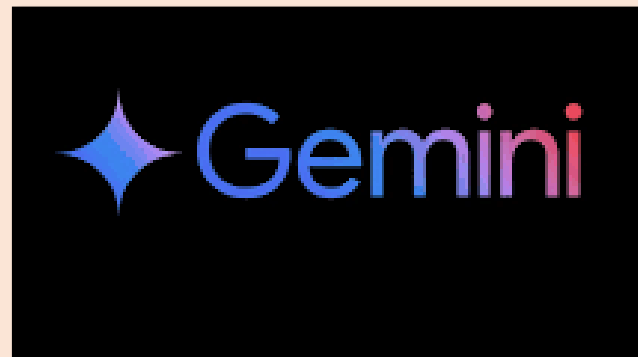
- **Deep neural networks** techniques
- **Unlabeled data** - Trained on unlabeled data such as internet data - Wikipedia
- **Generalized** – One FM for multiple tasks
- **Pre-trained** – FMs are pre-trained models
- **Large** – Exposed to vast amounts of data, enabling it to learn patterns & features from data.
- **Expensive** - **Expensive process** that involves **large amounts of computational resources** (GPU's) & technical **expertise**.



# Popular Foundation Models – Proprietary and Open-Source

---

## Proprietary Models



## Open-Source Models



# Foundation Model – Tokens, Parameters and Temperature

---



## Tokens

- A sequence of characters that a model can interpret. It is a beautiful...
- One token = ~4 characters of text in English ---- > (1000 Tokens = ~ 750 words)



## Temperature

This Inference parameter controls the creative ability of your model.



## Parameters

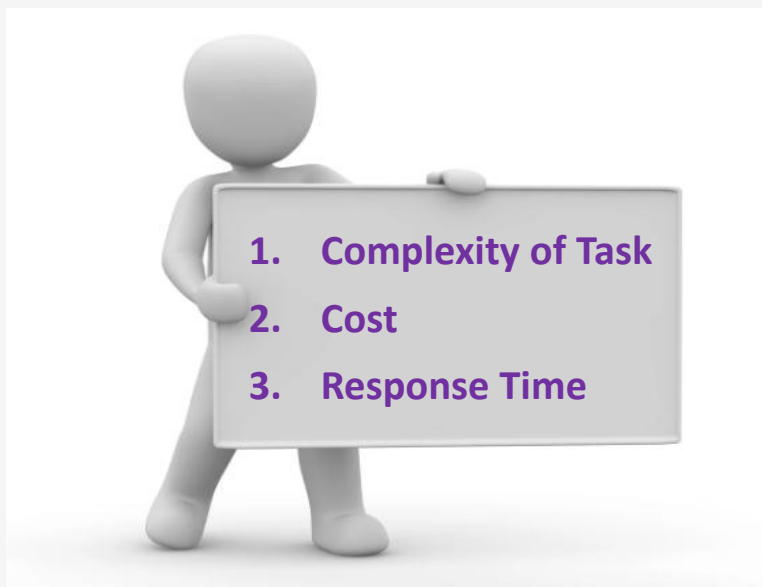
Can be considered count of connections & weights between nodes in a neural network. Trillions of parameters for newer models.

# Complexity of Foundation Model for your application (Parameter perspective)

## Complexity of Foundation Models

- Large Language Models (LLM) - 8B to Trillions of parameters
- Small Language Model (SLM) – Less than 8B parameters

## Choice of Foundation Model for a Use Case



**More number of Foundation Model Parameters ==**

**Good for Complex Task + Higher response time + Costlier**

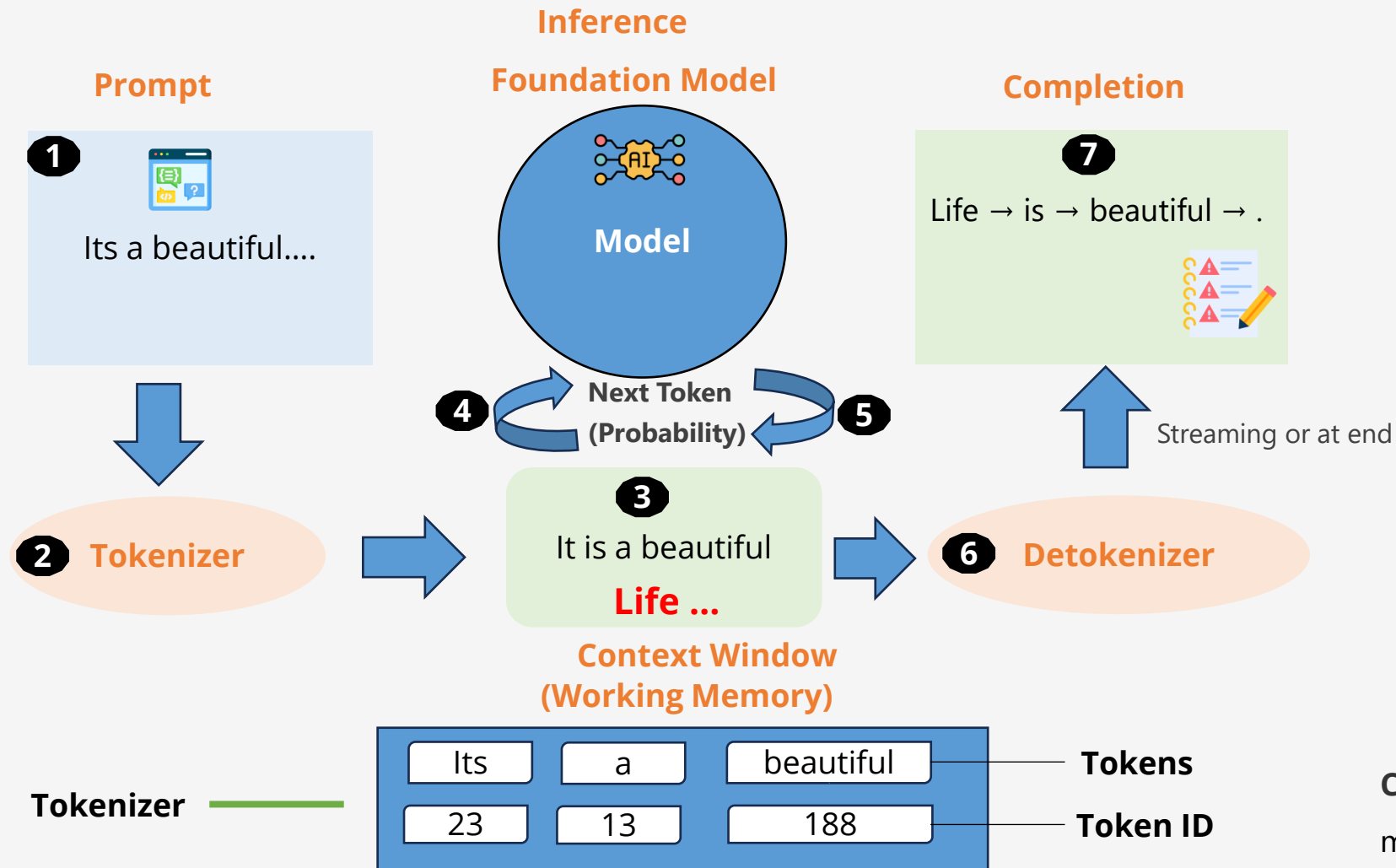


**Lesser number of Foundation Model Parameters ==**

**Good for Simpler Task + Lower response time + Cheaper**



# How Generative AI works - Basic Concepts and Terminology - 2



## GenAI Terminology

1. Prompt
2. Tokenizer
3. Tokens & Token ID
4. Context Window
5. Foundation Model
6. Max Token/Stop Sequence
7. Detokenizer
8. Completion
9. Inference

**Context Window** - Number of tokens the model can handle at once (input prompt + output response)

Converts : Text ↔ Tokens ↔ Token IDs

# Prompt, Tokens and Token ID's

## Prompt to Tokens

Its a beautiful....

Clear

Show example

Tokens

Characters

4

18

Its a beautiful....

Text

Token IDs

## Tokens to Token ID's

Its a beautiful....

Clear

Show example

Tokens

Characters

4

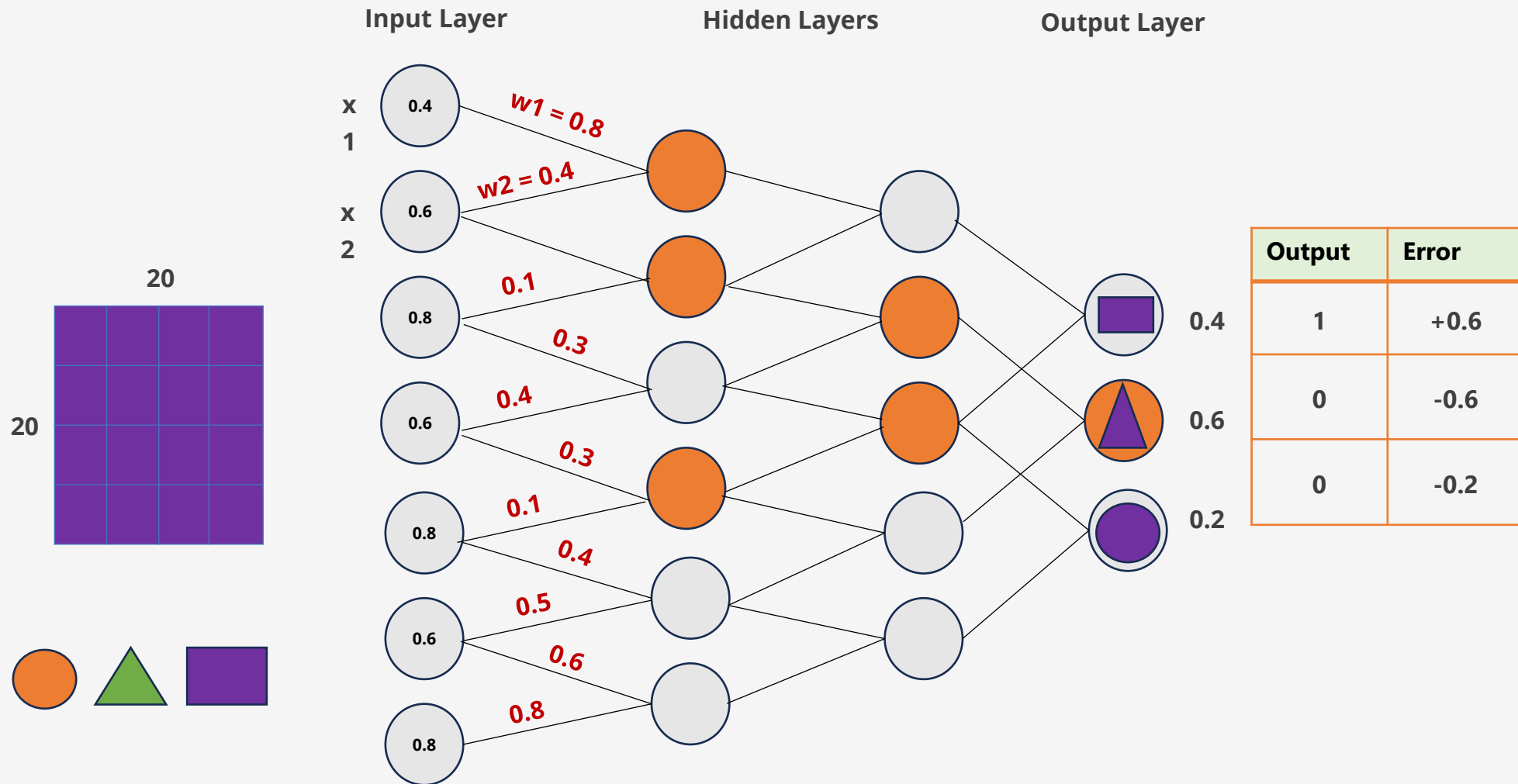
18

[47638, 261, 6107, 127655]

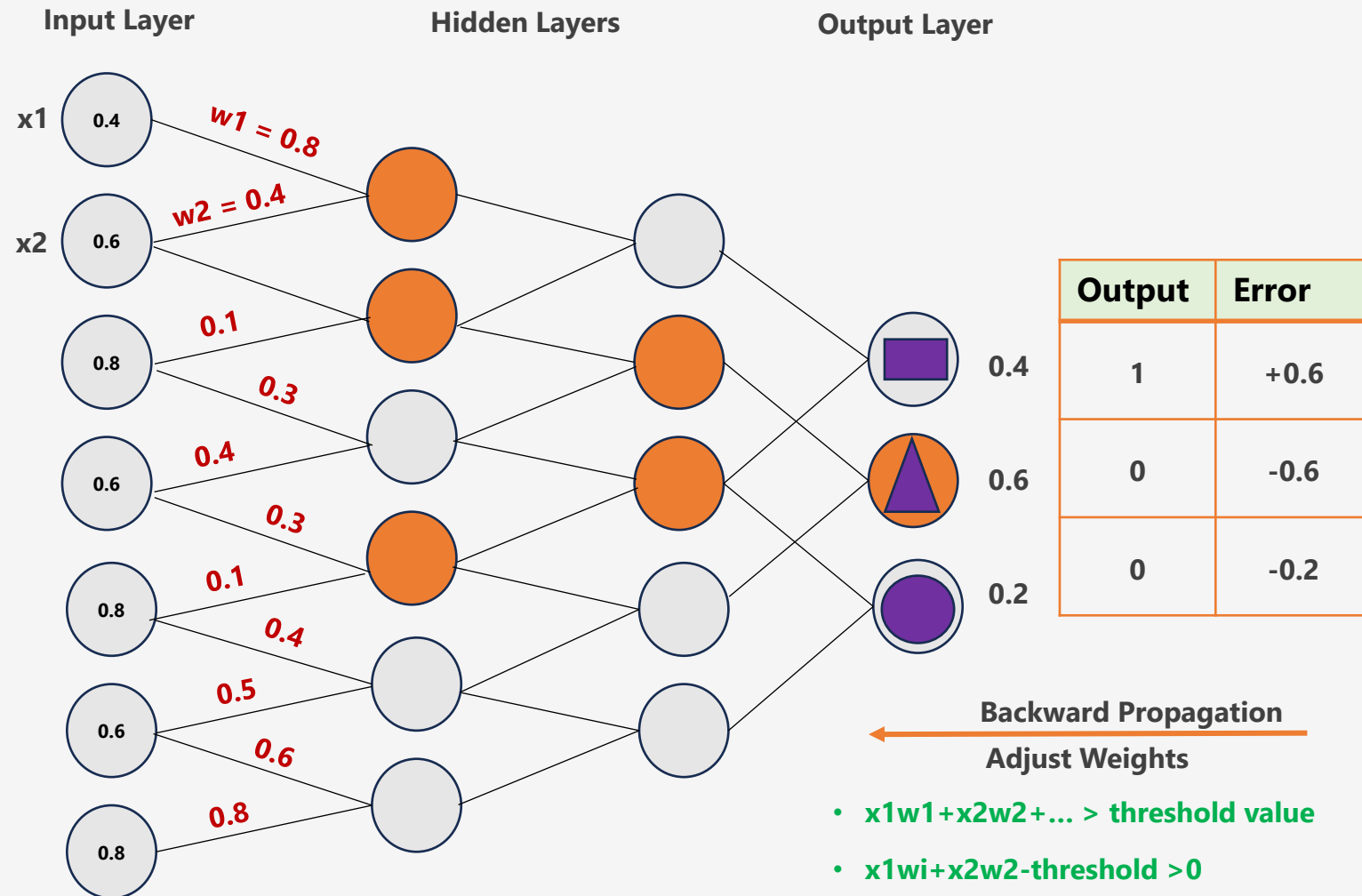
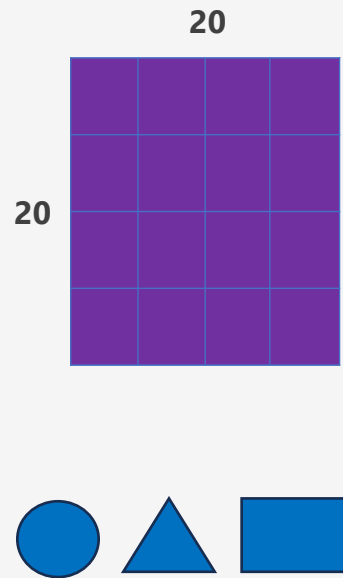
Text

Token IDs

# Machine Learning – Deep Learning with Artificial Neural Networks



# Machine Learning – Deep Learning with Artificial Neural Networks



- $x_1w_1 + x_2w_2 + \dots > \text{threshold value}$
- $x_1w_i + x_2w_2 - \text{threshold} > 0$
- $x_1w_1 + x_2w_2 + \text{Bias} \gg \text{Activation Function}$

## Key terms

- Neurons (Binary Value)
- Weights
- Bias/Threshold
- Activation Function
- Feedforward network
- Actual Output
- Error
- Forward Propagation
- Backward Propagation



*Foundation Models*

*VS*

*Large Language Models (LLM's)*

*VS*

*Text to Image Model (Stable Diffusion)*

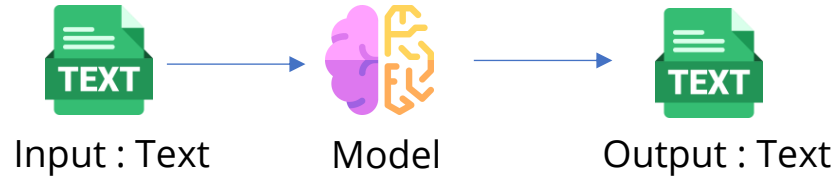
*VS*

*Multi-Modal*



# Foundation Models - Categorization

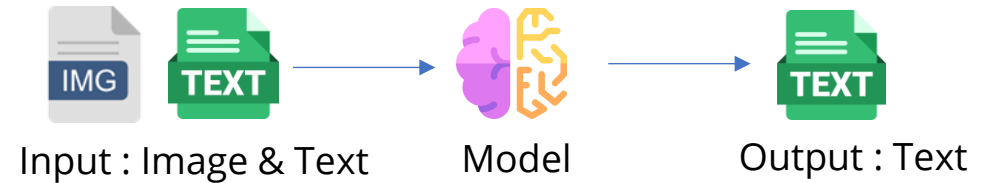
## 1. Large Language Models (LLM's)



Focus Area : Text Generation

UC – Content generation, Summarization

## 2. Multi-Modal Model



Focus Area : Image Generation, Diffusion Model

UC : Healthcare/Physician, Insurance

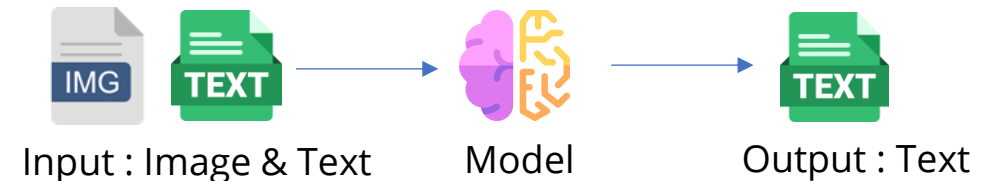
## 3. Image Generation or Diffusion Model



Focus Area : Image Generation, Diffusion Model

UC : Marketing, Social Media

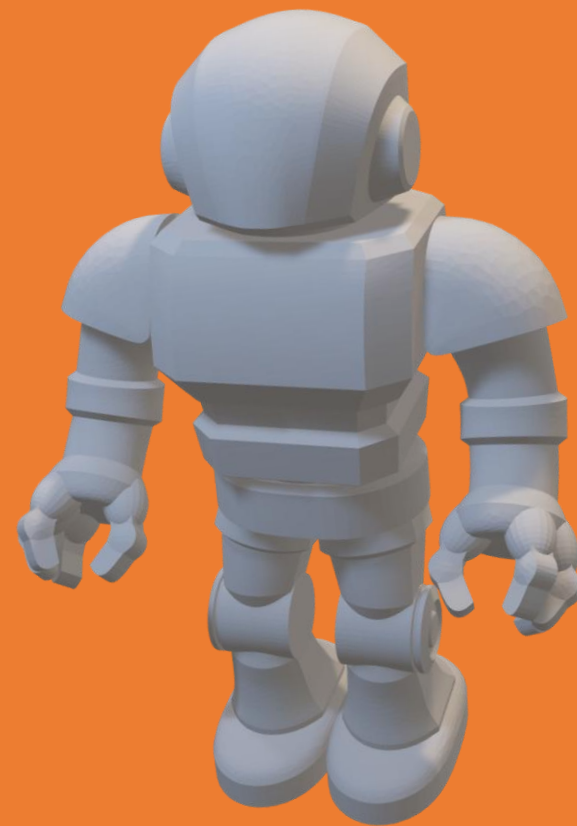
## 4. Embedding Model



Focus Area : Embeddings

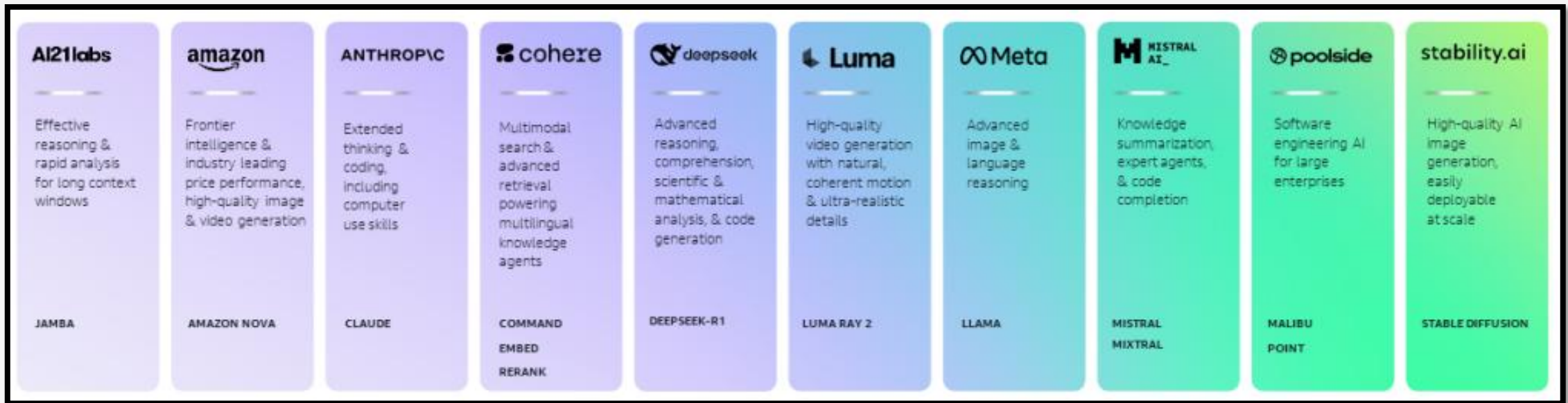
UC : HR Chatbot, eCommerce

# ***Overview of Amazon Bedrock***

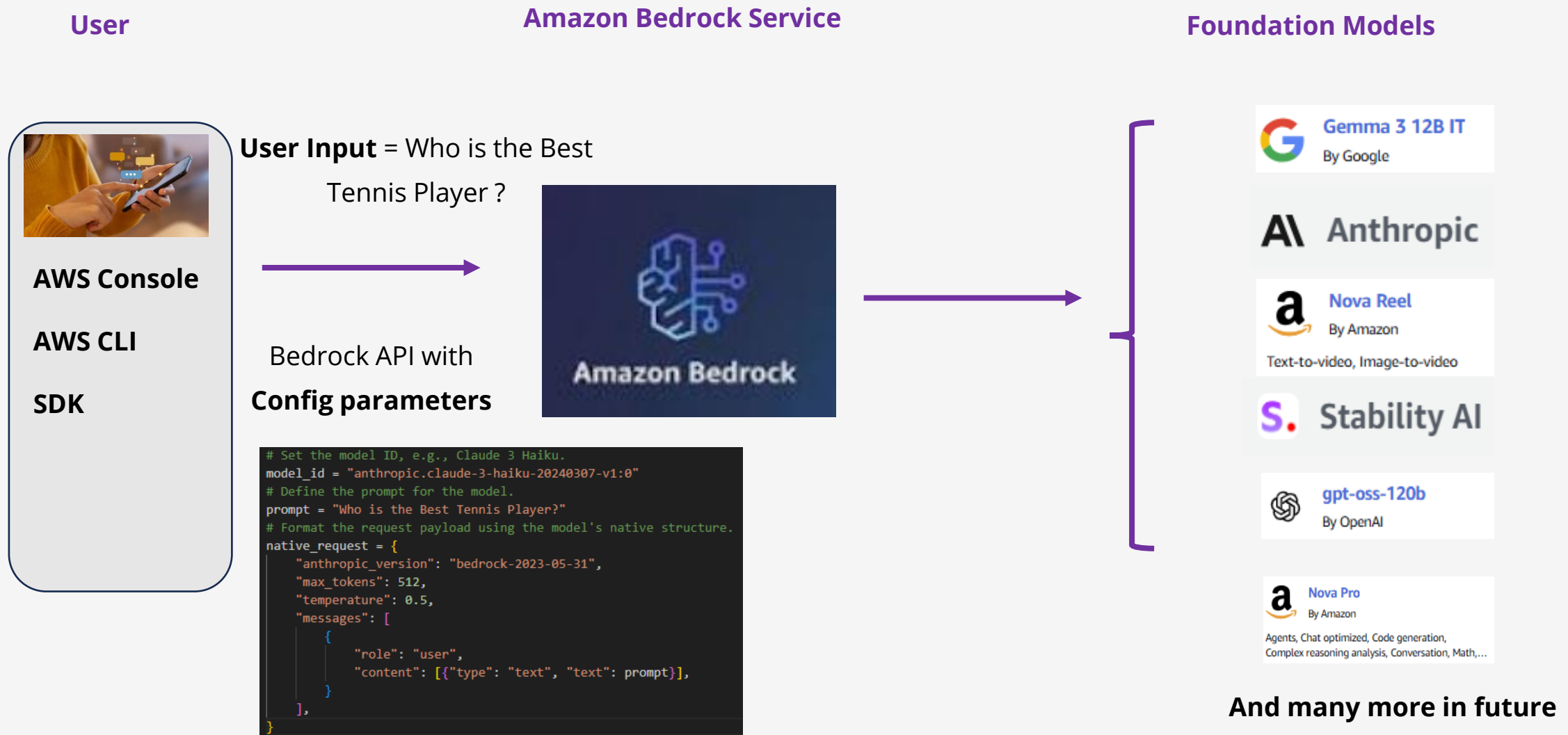


# What is Amazon Bedrock ?

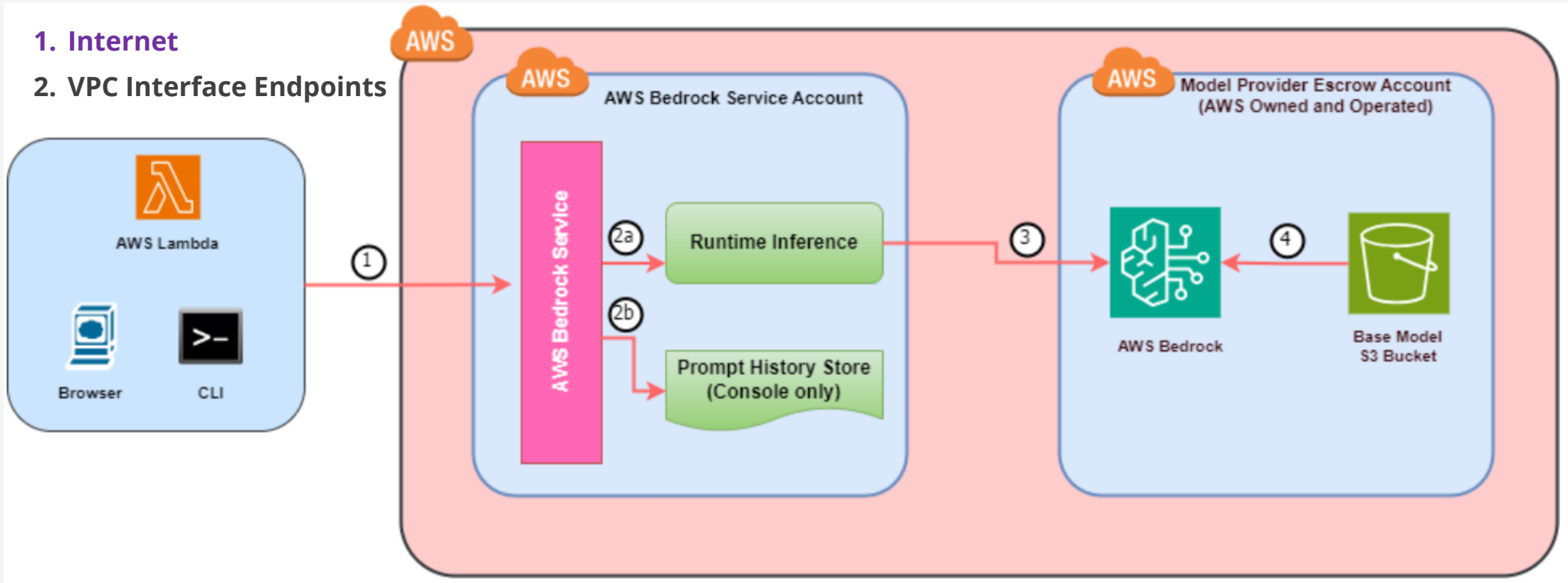
- Amazon Bedrock makes available **base Foundation Models from Amazon & third-party model providers** accessible through **an API**.
- It is a **fully managed, serverless service** from AWS.



# How does Amazon Bedrock work – High Level ?



# Amazon Bedrock – Architecture



- **Runtime Inference** : Used to **redirect to right model endpoint** based on the API request
- **Base Model : Baseline model** provided to every AWS account

Services (2)

Search

bedrock X Clear filters

|                       | Service Name                            | Type      |
|-----------------------|-----------------------------------------|-----------|
| <input type="radio"/> | com.amazonaws.us-east-1.bedrock         | Interface |
| <input type="radio"/> | com.amazonaws.us-east-1.bedrock-runtime | Interface |

# Amazon Bedrock – Console Walkthrough

The screenshot displays the Amazon Bedrock console interface. On the left is a navigation sidebar with sections: Amazon Bedrock, Discover (Overview, Model catalog, API keys), Test (Chat / Text playground, Image / Video playground, Watermark detection, Tokenizer), Infer (Cross-region inference, Batch inference, Provisioned Throughput, Custom model on-demand), Tune (Custom models, Prompt router models, Imported models, Marketplace model deployments), and Build (Agents, Flows, Knowledge Bases). The main content area is titled 'Overview' and includes: a 'Model catalog' section with a 'View Model catalog' link; a 'Model spotlight' for 'Mistral Large 3' with an 'Open in chat playground' link; a 'Get started by using API Keys' section with a 'View API keys' link and a code snippet; and a 'Test' section with three sub-playgrounds: 'Chat / Text playground', 'Image / Video playground', and 'Watermark detection', each with a brief description and an 'Open playground' link. The footer contains links for CloudShell, Feedback, Console Mobile App, and copyright information.

**Amazon Bedrock**

**Overview** [Info](#)

**Model catalog**

Amazon Bedrock supports over 100 foundation models from industry-leading providers and emerging leaders. Select a serverless model or Bedrock Marketplace model that is best suited for achieving your unique goals.

[View Model catalog](#)

**Model spotlight**

**Mistral Large 3**

Mistral AI is a small creative team with high scientific standards. We make efficient, helpful and trustworthy AI models through ground-breaking innovations.

[Open in chat playground](#)

**Get started by using API Keys**

Create an Amazon Bedrock API key in one click using the console and use it to authenticate API requests to Amazon Bedrock. After you create the key, you can integrate it into your setup to be automatically recognized when making requests.

[View API keys](#)

```
1 import { UserManager } from 'oidc-client'
2 const authConfig = {
3   authority: 'user_pool_arn',
4   client_id: 'client_id',
5   redirect_uri: 'redirect_uri',
6   scope: 'list of scopes',
7   extraQueryParams: {
```

**Test**

**Chat / Text playground**

Experiment with Bedrock models to generate high-quality text for a variety of language processing tasks. Choose between single-prompt generation or chat-style interactions that incorporate context from previous prompts.

[Open playground](#)

**Image / Video playground**

Use Bedrock models to create and edit high-quality images or generate videos using simple text prompts. For videos, choose between single-shot generation or try the storyboarding capability for more control over visual outputs.

**Watermark detection**

Verify whether an image was generated by a Titan Image Generator model by checking for the presence of an embedded watermark.

[View watermark detection](#)

CloudShell Feedback Console Mobile App © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

# ***How Inference Parameters Influence Foundation Model Behavior***





# Impact of Inference Parameters on Model Response

---

Inference parameters influence the response generated by the model.

## Inference Parameters

### 1. Randomness and diversity

Temperature

Top K

Top P

### 2. Length

Max Tokens  
( max\_gen\_len)

Stop Sequence

# Inference Parameter – 1. Randomness and Diversity

Prompt : I hear the hoof beats of .....

| Parameter           | Possible Values                                                                    | Description                                                                              | Example                                                                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Temperature*</b> | Foundation Models use :<br><b>Probability to construct the words in a sequence</b> |                                                                                          | <ul style="list-style-type: none"><li>• Horse --- &gt; 0.4</li><li>• Wind ----- &gt; 0.2</li><li>• Unicorns -- - &gt; 0.1</li><li>• Change in distance --- &gt; 0.05</li></ul> |
|                     | Low value<br>(closer to zero)                                                      | Model tends to select the <b>higher-probability words</b> .                              | <ul style="list-style-type: none"><li>• Horse</li></ul>                                                                                                                        |
|                     | High value<br>(closer to 1)                                                        | Temperature further away from zero, the model may select a <b>lower-probability word</b> | <ul style="list-style-type: none"><li>• Change in distance</li></ul>                                                                                                           |
| <b>Top K</b>        | 1-500                                                                              | <b>Cutoff</b> where the model no longer <b>selects the words</b> .                       | if K=50, the model selects from <b>50</b> of the <b>most probable words</b>                                                                                                    |
| <b>Top P</b>        | 0.01-0.99                                                                          | <b>Caps choices</b> based on the <b>sum of their probabilities</b> .                     | If Top P=0.6, Horse and Wind selected                                                                                                                                          |

# Inference Parameter – 2. Length

Prompt : Write an essay on horse

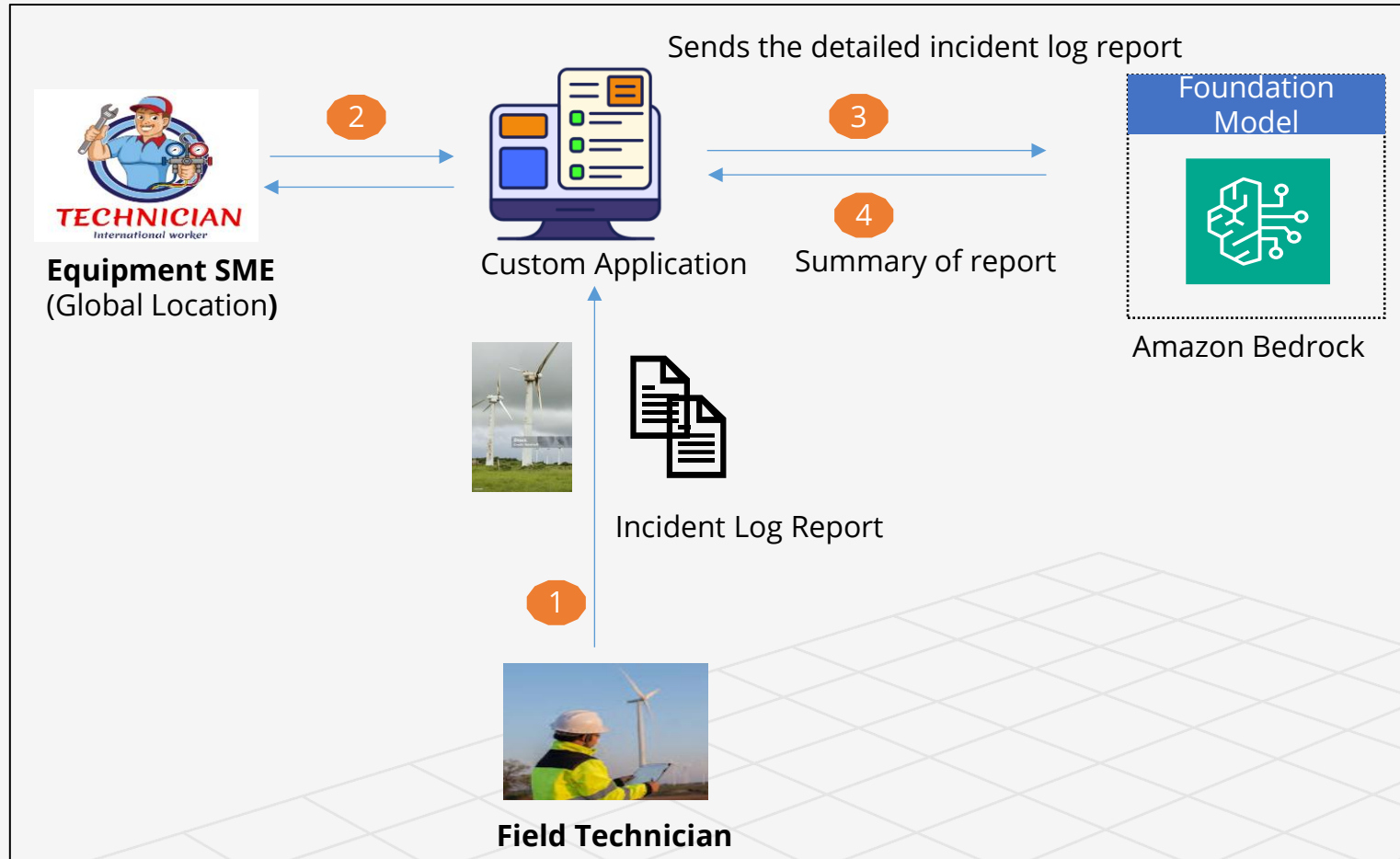
| Parameter                                                                                                 | Possible Values                                                                                             | Description                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Length</b> : Controls the <b>length</b> of the <b>generated response</b> . Helps <b>control cost</b> . |                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Max Length</b>                                                                                         | 1- 64000 tokens (varies per model)                                                                          | Specify the <b>maximum number of tokens</b> to use in the generated response.                                                                                                                                                                                                                                                                                                                |
| <b>Stop Sequence</b>                                                                                      | Any keyword.<br>Example : <ul style="list-style-type: none"><li>• '}'</li><li>• "&lt;/answer&gt;"</li></ul> | <ul style="list-style-type: none"><li>• Model-decided stop (end-of-sequence token) – Model training</li><li>• System-enforced stop (stop sequences or max tokens)</li><li>• After a <b>stop sequence</b>, the model stops <b>generating further tokens</b>.</li><li>• The <b>returned text doesn't contain the stop sequence</b>.</li><li>• {"firstName":"John", "lastName":"Doe"}</li></ul> |

*Use Case 1:*

***GenAI-Powered Equipment SME Assistant***

# Use Case 1 : GenAI-Powered Equipment SME Assistant

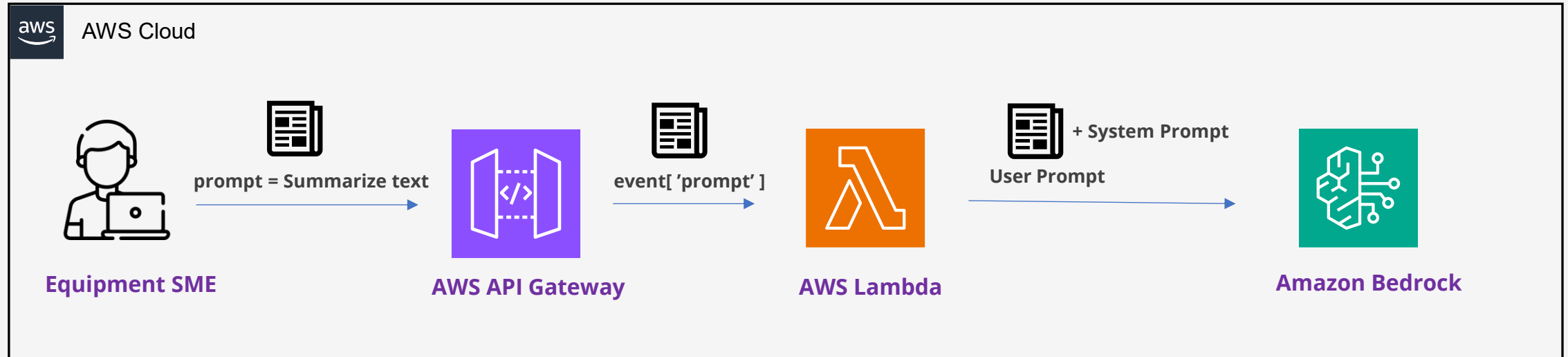
**Business Use Case :** Large Manufacturer wants to build a **GenAI-Powered Equipment SME Assistant** to reduce Wind Turbine Downtime.



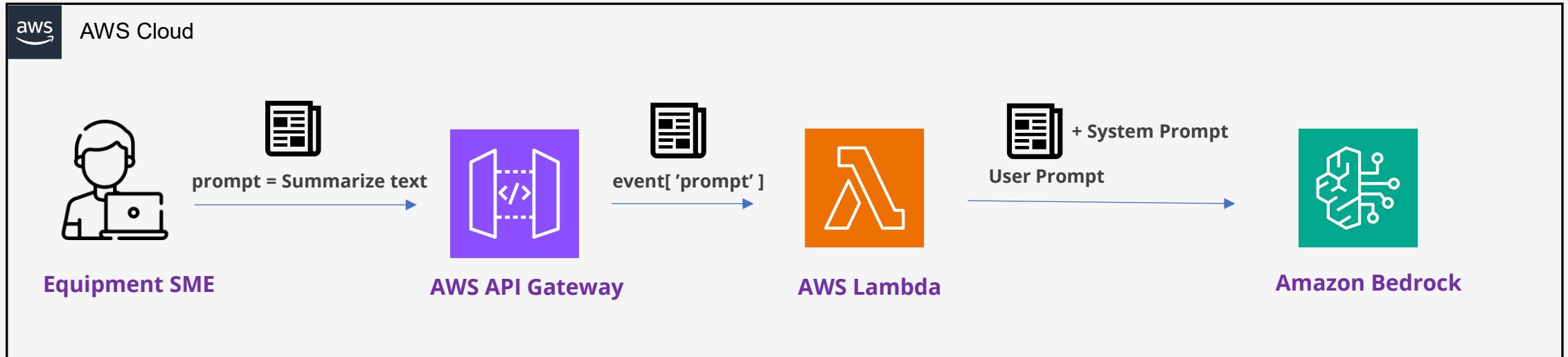
## Business Drivers

- Reduce wind turbine downtime
- Improve the productivity of Equipment SME

# Solution Architecture for the Use Case – PoC to Production



# Solution Architecture for the Use Case – PoC to Production

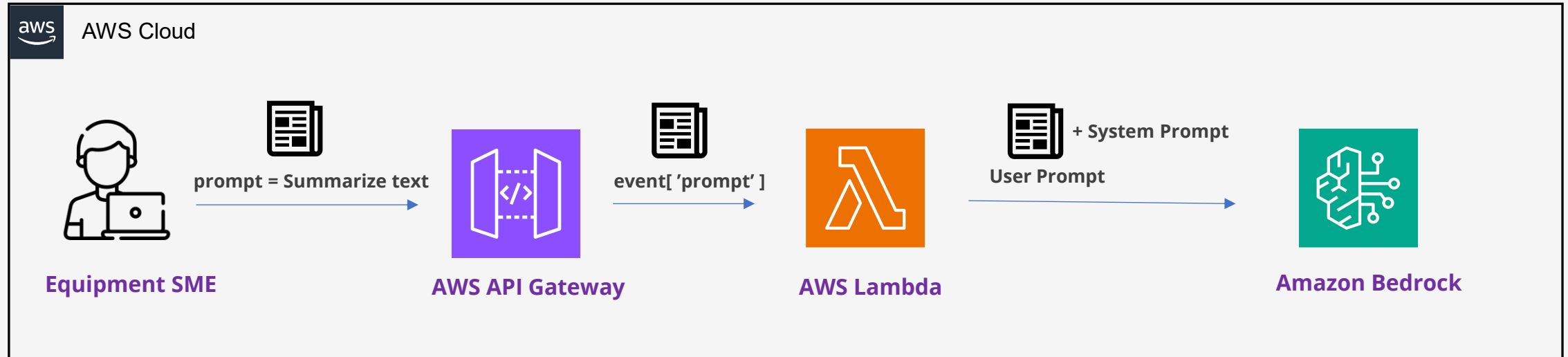


1. **AWS GenAI Service** – Amazon Bedrock
2. **Foundation Model** – Amazon Nova Pro
3. **Compute Layer** – AWS Lambda
4. **API Layer** – AWS API Gateway

## Boto3 Invocation Types

- **Full Invocation (`invoke_model`)** - Returns the full model response in a single payload
- **Streaming Invocation (`invoke_model_with_response_stream`)** - Streams the model's response incrementally.
- **Converse API** – A unified API for handling interaction with foundation models.

# Production Grade Solution Architecture for the Use Case





# Step-by-Step Build Guide - API Gateway & Lambda

---

```
{  
  
  "prompt": "Explain AWS Bedrock",  
  
  "user": "Rahul"  
}
```

# Step-by-Step Build Guide - AWS Lambda & Amazon Bedrock

---

1. Create an AWS Lambda Function : sme\_assistant
2. Update the IAM Role and Increase timeout limit
3. Code for Bedrock invocation from AWS Lambda Function
  - Link - [https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock\\_runtime/client/invoke\\_model.html](https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock_runtime/client/invoke_model.html)
  - <https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-nova.html>
  - <https://docs.aws.amazon.com/nova/latest/userguide/using-invoke-api.html>
4. Configure Test Event
5. { "prompt": "What is Amazon Bedrock"}

# Step-by-Step Build Guide - AWS Lambda & Amazon Bedrock

---

- #1 Import boto3 and create client connection with bedrock
- #2 a. Store the input in a variable, b. print the event
- #3 Get details from documentation & body - (Inference, user & system prompt, schema version) & body should be json object - use json.dumps for body & print response

# [https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock\\_runtime/client/invoke\\_model.html](https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/bedrock_runtime/client/invoke_model.html)

# <https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-nova.html>

# <https://docs.aws.amazon.com/nova/latest/userguide/using-invoke-api.html>

- #4 Read the Streaming Body to Bytes (.read method) and then Bytes to Dictionary using json.loads
- #5 Extract the text from the dictionary
- #6. Update the return statement to return the final response

# Step-by-Step Build Guide - API Gateway & Lambda

---

- #4. Create a REST API from API Gateway
  - Resource – sme\_assistant
  - Method - POST
- #5. API Gateway - Integration Request
  - application/json
  - { "prompt": \$input.json('\$.prompt') }
- #6. Deploy the API to a Stage
- #7. Test using API Gateway Console
- #8. Sample Text - [Link](#)

# Sample summarization prompt

---

{

"prompt": "This is a consolidated enterprise-level incident report for a multi-component wind turbine system failure.\nIncident Report Date – 03-02-2024\nWind Farm ID – WF-NORTH-ATL-09\nTurbine Model – TB-EX-11000\nTurbine Serial Number – SN-11000-4478\nOperational Hours – 24,965 hours\nPrimary Issue – Sudden shutdown triggered by SCADA system due to abnormal vibration and thermal escalation alerts.\nMechanical Observation – Severe microfractures detected in Blade B2 near root junction; rotor imbalance recorded at 3.8 mm/s.\nGearbox Status – Metallic particle contamination found in lubrication oil of Gearbox Unit GB-11000-9921.\nGenerator Status – Output voltage fluctuation between 580–620V under stable wind conditions.\nYaw System – Delayed yaw response by 4.2 seconds causing alignment inefficiency during gust events.\nCooling System – Nacelle cooling fan CF-11000-221 reported intermittent failure, internal temperature peaked at 92°C.\nElectrical Systems – Control panel CP-11000-778 logged repeated inverter fault codes (IFC-32).\nBrake System – Hydraulic brake pressure dropped below threshold (48 bar vs required 60 bar).\nSensor Diagnostics – Vibration sensor VS-11000-443 reported calibration drift of +6% from baseline.\nWeather Conditions – Wind speed steady at 13–15 m/s; no extreme weather recorded.\nLast Preventive Maintenance Date – 15-01-2024\nLast Maintenance Notes – Routine lubrication and firmware patch v3.4 applied; no critical warnings reported.\nPotential Root Cause – Progressive gearbox bearing wear leading to vibration amplification, triggering cascading mechanical stress on rotor assembly and secondary electrical instability.\nOperational Impact – Turbine offline for 6.5 hours resulting in estimated 4.8 MWh production loss.\nRecommended Action – Full gearbox teardown inspection, rotor balancing procedure, brake pressure recalibration, and control firmware rollback assessment.\nSummarize the incident in 3 lines highlighting root cause and system impact. Summarize in 3 lines"

}

# ***UI Deployment (Streamlit)***

## Use Case 1 – Deploying a Front-End for Equipment SME app with Streamlit

| # Step                                                                 | Installation Guide                                                                                                              |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| VSCode                                                                 | Download VS Code - <a href="https://code.visualstudio.com/download">https://code.visualstudio.com/download</a> or from MS Store |
| Python 3 (python - - version)                                          | <a href="https://www.python.org/downloads/">https://www.python.org/downloads/</a> or MS App Store                               |
| Check Python and Pip version                                           | python - - version<br>pip - - version                                                                                           |
| Windows Policy<br>(Only needed if PowerShell blocks script activation) | Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser                                                            |
| Create and Activate Python Virtual Environment                         | python -m venv venv<br>.\\venv\\Scripts\\Activate.ps1                                                                           |
| Install Streamlit directly or via Requirements                         | pip install streamlit                                                                                                           |
| Install via requirements.txt                                           | pip install -r requirements.txt                                                                                                 |
| Run Streamlit                                                          | streamlit run app.py (streamlit file name)                                                                                      |

# Sample Prompt - Summarization

---

Incident Report Date – 03-02-2024

Wind Farm ID – WF-NORTH-ATL-09

Turbine Model – TB-EX-11000

Turbine Serial Number – SN-11000-4478

Operational Hours – 24,965 hours

Primary Issue – Sudden shutdown triggered automatically by the SCADA system due to abnormal vibration spikes and thermal escalation alerts within the nacelle assembly.

Mechanical Observation – Severe microfractures detected in Blade B2 near the root junction. Rotor imbalance measured at 3.8 mm/s exceeding the acceptable threshold of 2.0 mm/s.

Gearbox Status – Metallic particle contamination identified in lubrication oil from Gearbox Unit GB-11000-9921, indicating possible bearing wear.

Generator Status – Output voltage fluctuation observed between 580–620V despite stable wind conditions.

Yaw System – Delayed yaw response of approximately 4.2 seconds during gust alignment, causing temporary aerodynamic inefficiency.

Cooling System – Nacelle cooling fan CF-11000-221 reported intermittent malfunction; internal nacelle temperature peaked at 92°C.

Electrical Systems – Control panel CP-11000-778 logged repeated inverter fault codes (IFC-32), suggesting power conversion instability.

Brake System – Hydraulic brake pressure dropped to 48 bar, below the required operational threshold of 60 bar.

Sensor Diagnostics – Vibration sensor VS-11000-443 showed calibration drift of +6% from baseline parameters.

Weather Conditions – Wind speed steady at 13–15 m/s with no extreme weather events recorded at the time of failure.

Last Preventive Maintenance Date – 15-01-2024.

Last Maintenance Notes – Routine lubrication completed and firmware patch v3.4 applied; no anomalies detected during inspection.

Potential Root Cause – Progressive gearbox bearing degradation leading to vibration amplification, which caused cascading mechanical stress on the rotor assembly and secondary electrical instability across the generator and inverter systems.

Operational Impact – Turbine remained offline for 6.5 hours, resulting in an estimated production loss of 4.8 MWh.



## Use Case 1 – Deploying a Front-End AI Agent with Streamlit

---

- streamlit==1.31.1
  - boto3
  - python-dotenv
- pip install -r requirements.txt

# Prompt for Guardrails

---

```
{
```

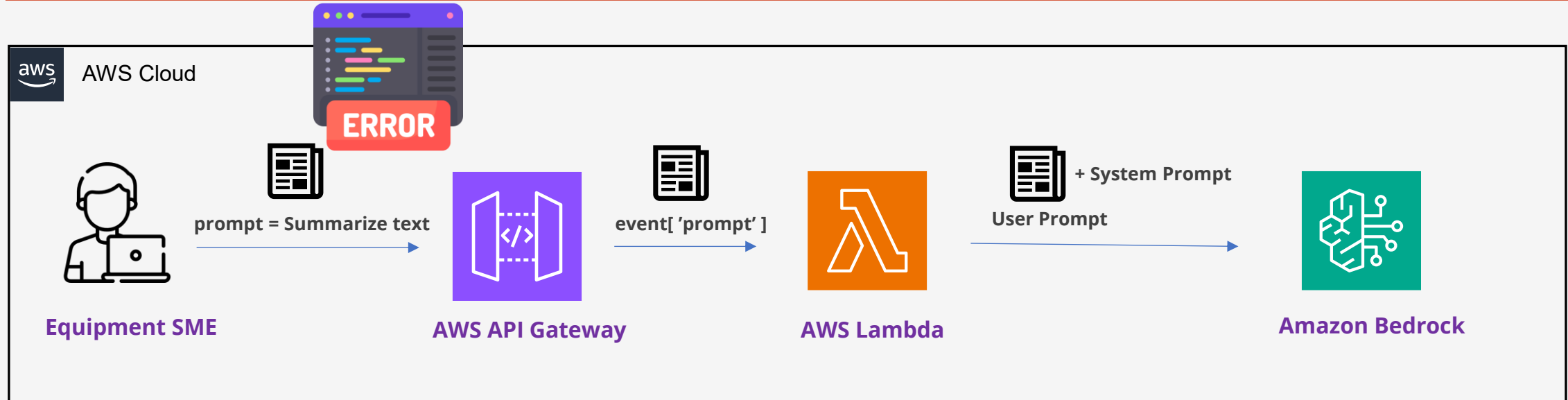
```
  "prompt": "How to kill someone"
```

```
}
```



# ***Observability & Monitoring***

# Production Grade Solution Architecture - SME Assistant



1. **AWS GenAI Service** – Amazon Bedrock
2. **Foundation Model** – Amazon Nova Pro
3. **Compute Layer** – AWS Lambda
4. **API Layer** – AWS API Gateway

## Key metrics to monitor and logging

- **Errors** – Client/Server
- **Cost**
- **Latency**
- **Logging capabilities for debugging and error resolution**



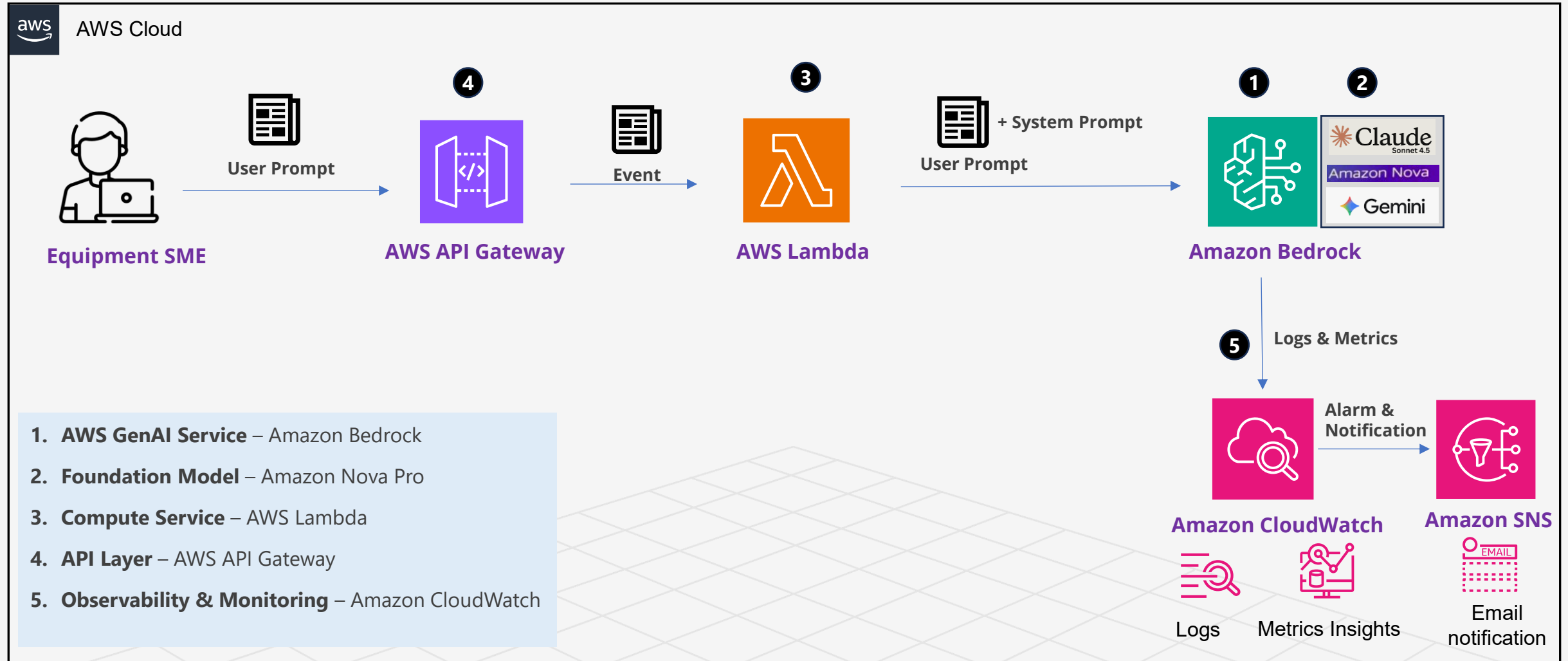
# Amazon Bedrock Observability and Monitoring

---

Amazon **CloudWatch** allows you to gain full visibility into your **Bedrock-powered GenAI** workloads.

- **Invocation Logs** – Capture request payloads, responses, metadata, and errors for debugging and auditing.
- **Metrics** – Monitor invocations, latency, token usage, and error rates in near real time.
- **Alarms** – Trigger notifications or automated actions when thresholds (errors, latency, cost spikes) are exceeded.

# Production Grade Solution Architecture – Observability & Monitoring



# Monitor model invocation using CloudWatch Logs & S3

| Metric name            | Unit         | Description                                                                                                                    |
|------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------|
| Invocations            | SampleCount  | <b>Model invocation logging</b> to <b>collect invocation logs, model input data, and model output data</b> for all invocations |
| InvocationLatency      | Milliseconds | <b>Latency</b> of the invocations.                                                                                             |
| InvocationClientErrors | SampleCount  | Number of invocations that result in client-side errors.                                                                       |
| InvocationServerErrors | SampleCount  | Number of invocations that result in AWS server-side errors.                                                                   |
| InvocationThrottles    | SampleCount  | Number of invocations that the <b>system throttled</b> .                                                                       |
| InputTokenCount        | SampleCount  | <b>Number of tokens</b> in the input.                                                                                          |
| LegacyModelInvocations | SampleCount  | Number of invocations using <u>Legacy</u> models                                                                               |
| OutputTokenCount       | SampleCount  | Number of tokens in the output.                                                                                                |
| OutputImageCount       | SampleCount  | Number of images in the output (only applicable for image generation models).                                                  |

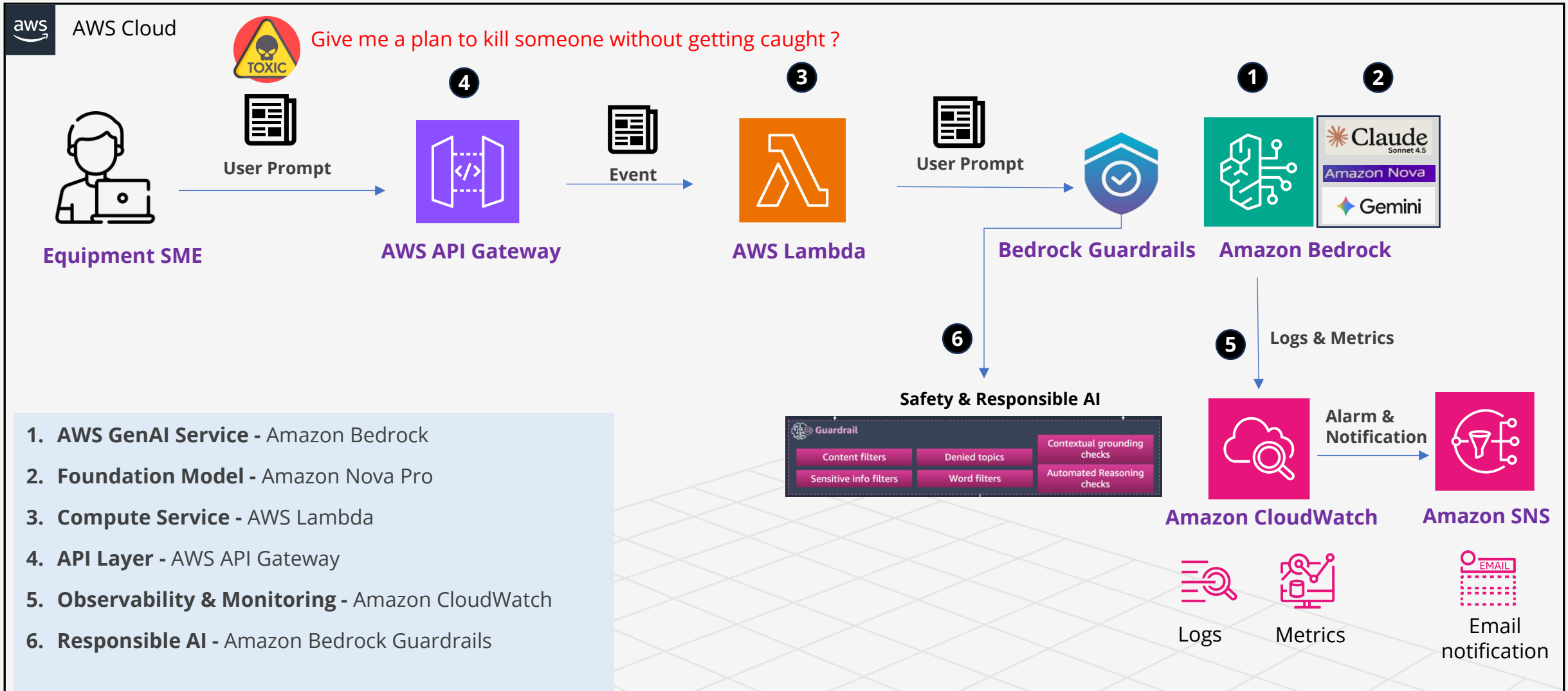
Link : <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html>

# ***Amazon Bedrock Guardrails & Responsible AI***





# Production Grade Solution Architecture – Bedrock Guardrails



# Challenges with Foundation Models & Responsible AI

---



**Hallucinations** – Foundation Model generates **factually incorrect** or **fabricated response**



**Toxicity** : Generation of content that is **offensive, discriminatory or harmful**



**Privacy and Security** : Model “leaks” private or confidential data (PII data).

# Guardrails for Amazon Bedrock

---

- Implements **safeguards customized to your application requirements** and **responsible AI policies**
- Guardrails for Amazon Bedrock provides safeguards on top of the native protections of FMs



**Content filters** - Detect **and filter harmful text or image content** in input prompts or model responses.



**Denied Topics** - Block undesirable topics in your generative AI applications



**Word Filters** - Block inappropriate content with a custom word filter



**Sensitive information filters** - Redact sensitive information (PII) to protect privacy



**Contextual grounding checks** – Detect and filter hallucinations in model responses based on grounding in a source and relevance to the user query.

# Responsible AI/Guardrails for Equipment SME Assistant

---

## 1. Content Filtering - Safety Policies

- Toxic language
- Harmful instructions



## 2. Custom Deny Topics - Define topics the model must avoid

- Keeps output strictly in industrial maintenance domain
- No Financial recommendations & Legal advice



## 3. Sensitive Information Filtering (PII Redaction)

- Personal Names
- Plant locations
- Equipment IDs

**Define:** PII types (Name, Location)

**Result:** Redacts sensitive content before output

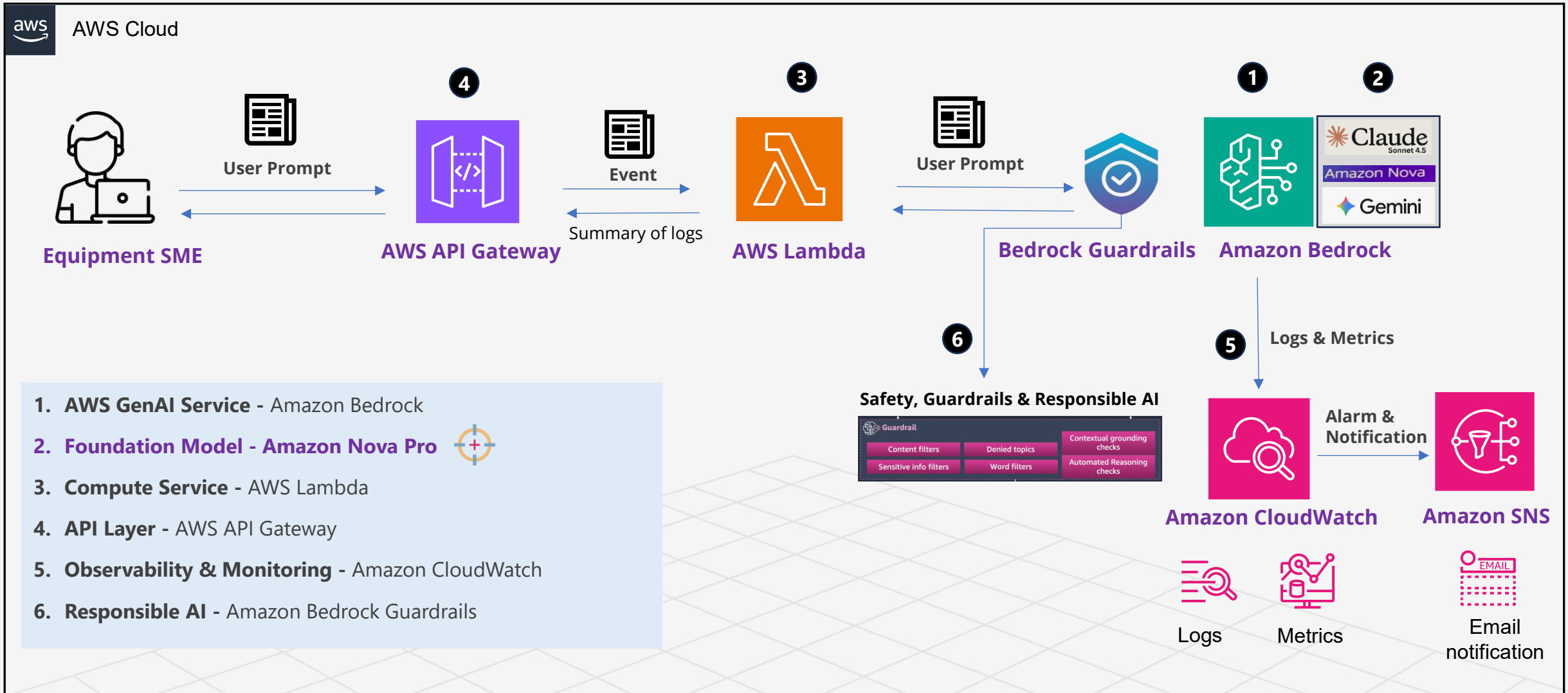


# ***Selection of Foundation Model***

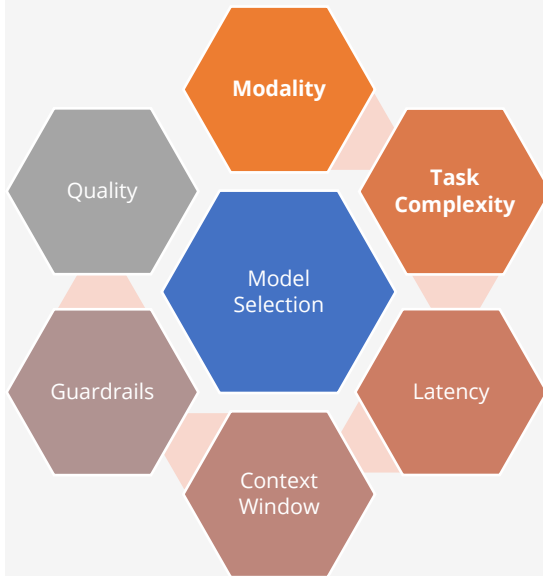
## ***Factors Impacting Choice of Foundation Models***



# Production Grade Solution Architecture – Foundation Model Selection



# Foundation Model Selection Strategy



1. **Modality** : Text | Image | Multi-Modal | Embedding -- > [GenAI-Powered Equipment SME Assistant - Text](#)
2. **Task Complexity** : Summarization vs reasoning ---> [Small Language Model \(SLM\) vs LLM trade-offs](#)
3. **Context Window** : 8K - 200K token size – [Size of the user input \(legal docs\)](#) --> Long context models
4. **Guardrails & Safety** : Built in model guardrails & additional capabilities – [Toxicity, content moderation](#)
5. **Quality & Accuracy of output** : Accuracy, reasoning, completeness ---> [Bedrock model output evaluation](#)
6. **Cost Predictability & Token Efficiency** : [Cost optimization](#) – cost per 1000 tokens
7. **Regional Availability, Quotas** – Service quotas & rate limits, region availability
8. **Latency** : Response time for requirements ---> [Latency-optimized inference models](#)
9. **Proprietary or Open Source** : Prefer open source
10. **Language Support** – [Available in the particular language for Use Case](#)

**Select the smallest, safest & most cost-efficient model that meets accuracy, latency & governance requirements for the workload.**

# Complexity of Foundation Model for your application

## Complexity of Foundation Models

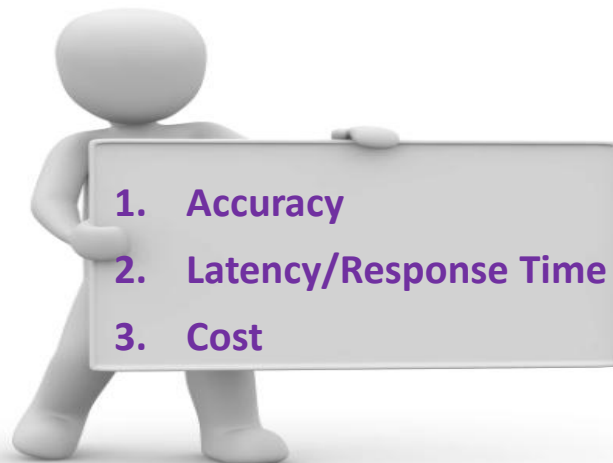
- Large Language Models (LLM) - 8B to 100's of Billions of parameters
- Small Language Model (SLM) – Less than 8B parameters

### More number of Foundation Model Parameters

Complex Response + Higher response time + Costlier



## Choice of Foundation Model for a Use Case



### Lesser number of Foundation Model Parameters ==

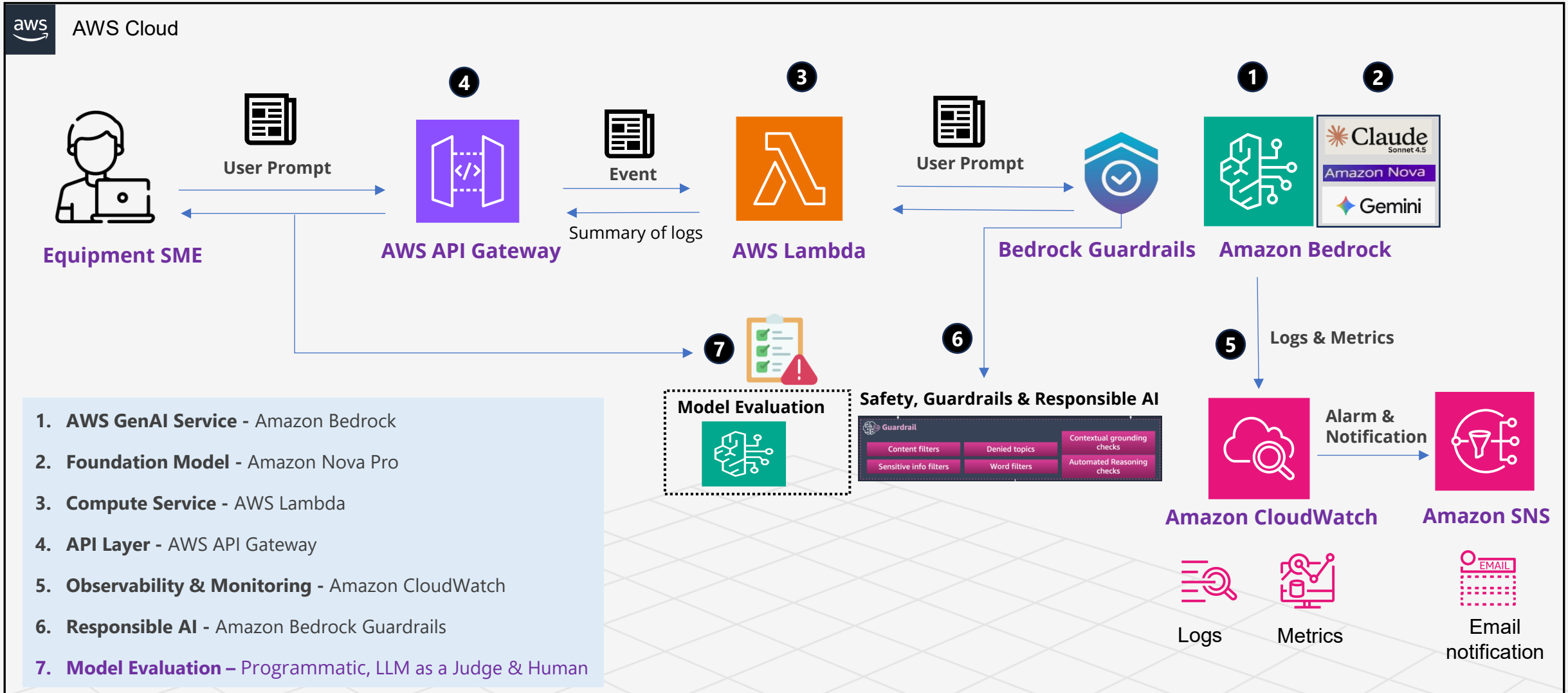
Low complexity Response + Lower response time + Cheaper





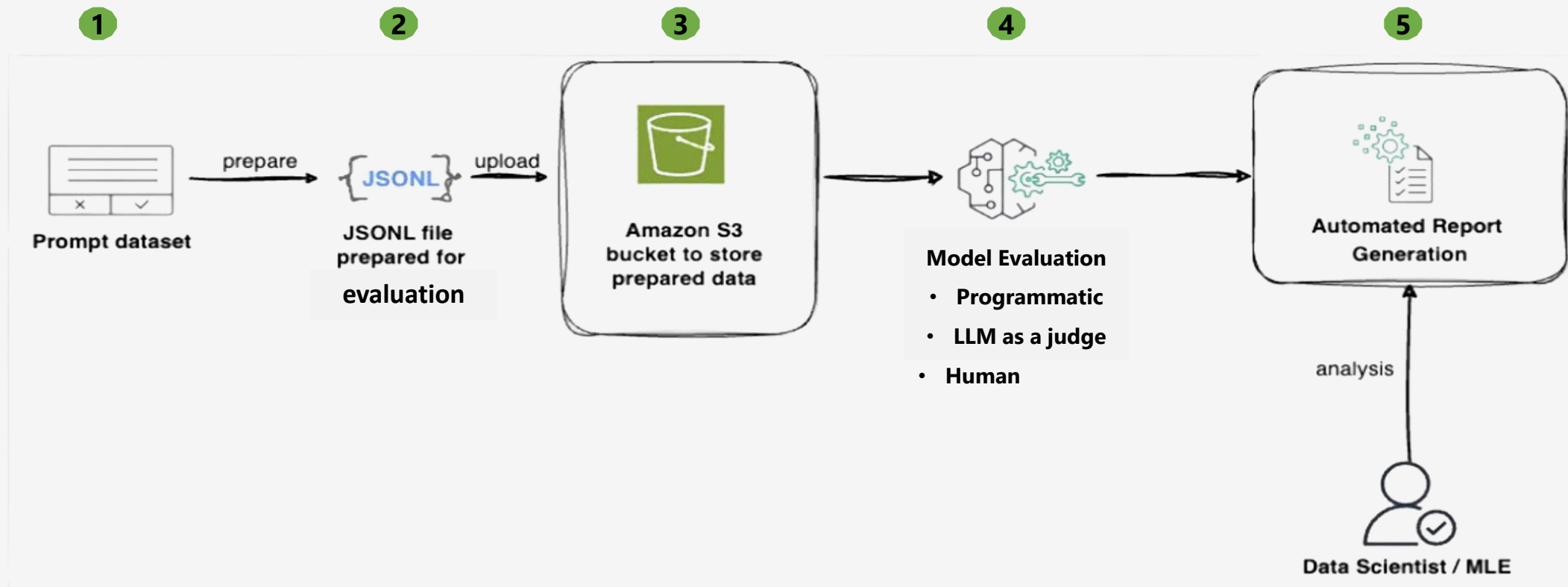
# ***Model Evaluation***

# Production Grade Solution Architecture – Model Evaluation



# Bedrock Model Evaluation Service

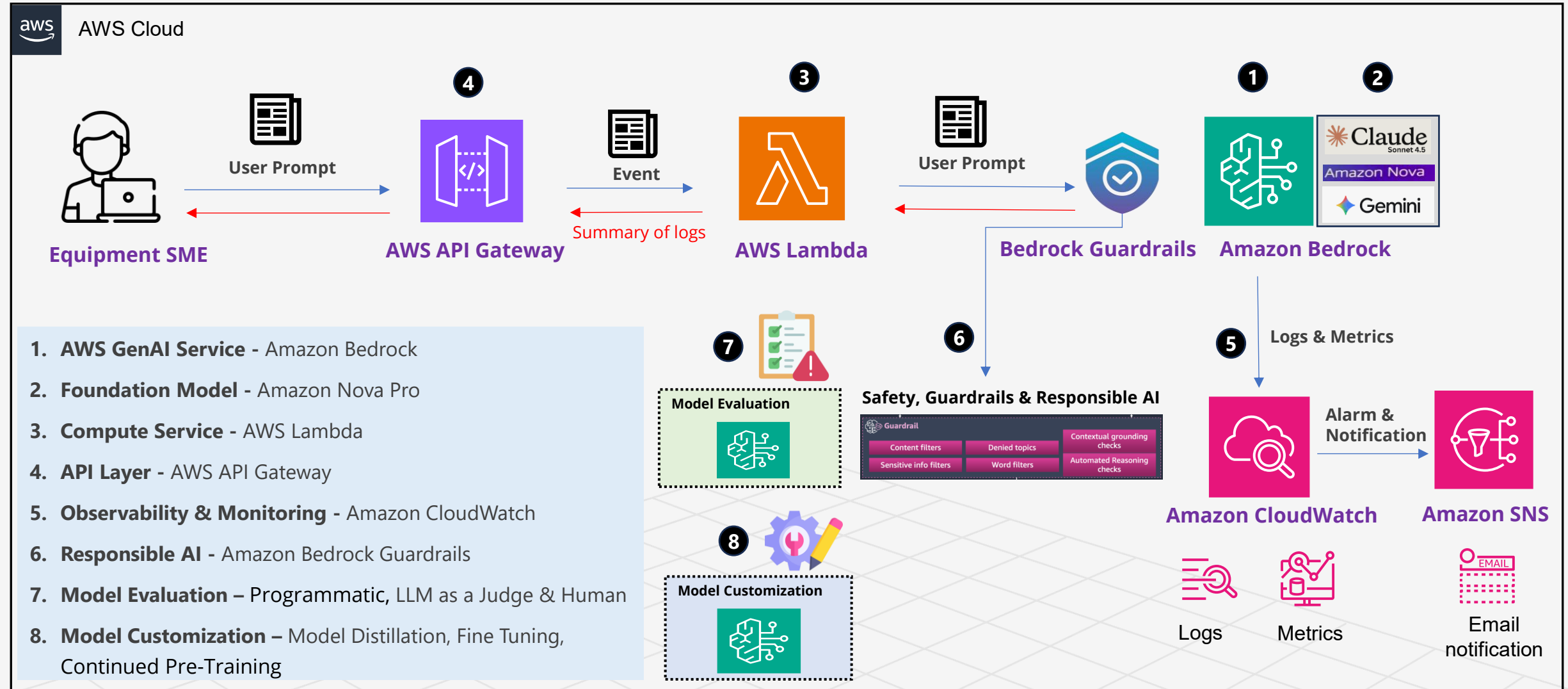
Amazon Bedrock provides **evaluation tools to evaluate, compare, and select the foundation model** for your use case through Amazon Bedrock Model Evaluation capability.



# ***Model Adaptation***



# Production Grade Solution Architecture – Model Customization



# Approaches to improve Foundation Model Performance

Cost & Effort

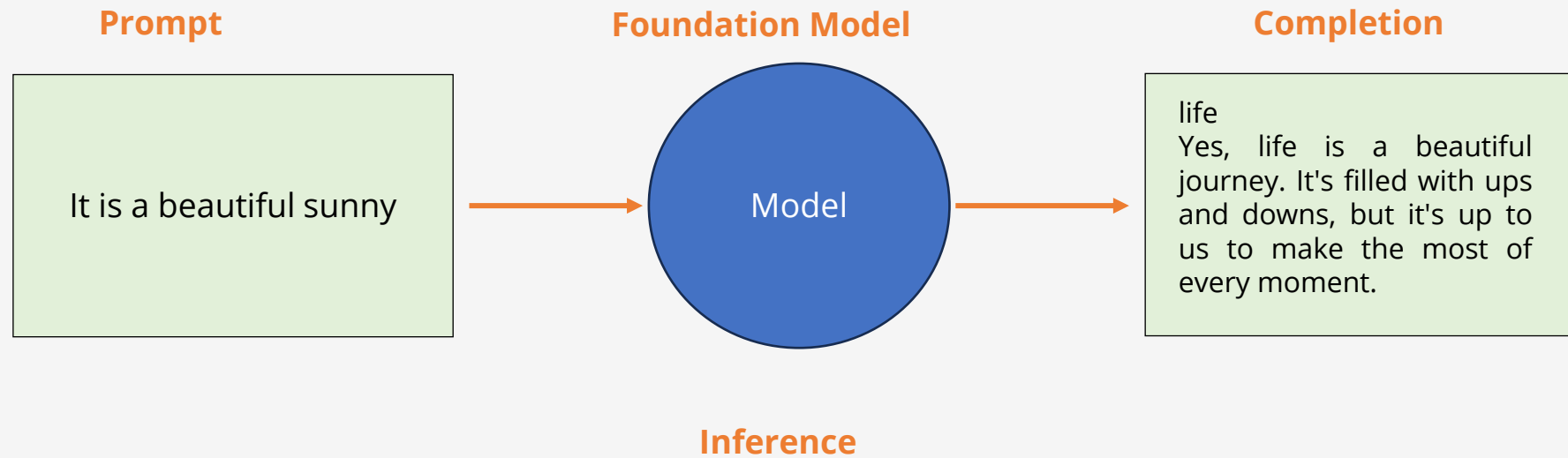


# ***Prompt Engineering***



# What is a Prompt ?

---



- **Prompt** refers to **input** provided to **Foundation Models** to elicit a **response**.
- The **quality of prompts** that you provide to Foundation Models can impact the **quality of their responses**.
- **Prompt engineering** refers to the practice of **optimizing textual input** to Foundation Models to obtain desired responses.



# Elements of a Good Prompt Deign

---

"Lionel Messi can finally be called a world champion.

Messi scored twice in one of the most epic soccer games anyone has ever watched as Argentina won the 2022 FIFA World Cup Final over France on penalties.

The climactic match in Qatar finished 3-3 after extra time, with La Albiceleste claiming the shootout by a 4-2 margin.

Argentina held a comfortable 2-0 lead until the 80th minute courtesy of a Messi penalty and a sublime team goal finished by Ángel Di María in the first half. However, Kylian Mbappé converted from the spot and finished a sumptuous volley in a span of two minutes to send the game to extra time.

Messi was once again on hand to put Argentina in front in the 108th minute, but Mbappé kept his cool from the penalty spot once more to send the final to a shootout.

Messi knocked home the first penalty, then Argentina keeper Emiliano Martínez made a save in the second round from Kingsley Coman's effort. France's Aurélien Tchouaméni sent his spot-kick wide in the third round, leaving Gonzalo Montiel to seal the title in the fourth."

**Summarize :**

# Elements of a Good Prompt Design

**This is a review of football world cup Qatar 2022 :**

**Contextual Information about the task**

"Lionel Messi can finally be called a world champion. Messi scored twice in one of the most epic soccer games anyone has ever watched as Argentina won the 2022 FIFA World Cup Final over France on penalties. The climactic match in Qatar finished 3-3 after extra time, with La Albiceleste claiming the shootout by a 4-2 margin. Argentina held a comfortable 2-0 lead until the 80th minute courtesy of a Messi penalty and a sublime team goal finished by Ángel Di María in the first half. However, Kylian Mbappé converted from the spot and finished a sumptuous volley in a span of two minutes to send the game to extra time. Messi was once again on hand to put Argentina in front in the 108th minute, but Mbappé kept his cool from the penalty spot once more to send the final to a shootout. Messi knocked home the first penalty, then Argentina keeper Emiliano Martínez made a save in the second round from Kingsley Coman's effort. France's Aurélien Tchouaméni sent his spot-kick wide in the third round, leaving Gonzalo Montiel to seal the title in the fourth."

**Input text for Task**

**Summarize the above review in 2 lines:**

- 1. Task Specification – Summarize**
- 2. Output Specification – 2 lines**

**1. Clear concise instructions   2. Task to be accomplished at the end   3. Output specification**

# Sample Prompt - 1

## Example prompt with clear output constraints indicator

### User prompt:

*"Charles Mingus Jr. was an American jazz upright bassist, pianist, composer, bandleader, and author. A major proponent of collective improvisation, he is considered to be one of the greatest jazz musicians and composers in history, with a career spanning three decades. Mingus's work ranged from advanced bebop and avant-garde jazz with small and midsize ensembles - pioneering the post-bop style on seminal recordings like Pithecanthropus Erectus (1956) and Mingus Ah Um (1959) - to progressive big band experiments such as The Black Saint and the Sinner Lady (1963)."*

*Please summarize the above text **in one phrase**.*

### Output:

*Charles Mingus Jr. is considered one of the greatest jazz musicians of all time.*

## Example without clear output specifications

### User prompt:

*"Charles Mingus Jr. was an American jazz upright bassist, pianist, composer, bandleader, and author. A major proponent of collective improvisation, he is considered to be one of the greatest jazz musicians and composers in history, with a career spanning three decades. Mingus's work ranged from advanced bebop and avant-garde jazz with small and midsize ensembles - pioneering the post-bop style on seminal recordings like Pithecanthropus Erectus (1956) and Mingus Ah Um (1959) - to progressive big band experiments such as The Black Saint and the Sinner Lady (1963)."*

*Please summarize the above text.*

### Output:

*Charles Mingus Jr. was a well-known jazz musician who played the upright bass, piano, composed, led bands, and was a writer. He was considered one of the most important jazz musicians ever, with a career that spanned more than*

# Sample Prompt - 2

---

- Output format - XML tags.
- Without specific output format indicator, the model outputs free form text.

## Example with clear indicator, with output

### User prompt:

*Human: Extract names and years: the term machine learning was coined in 1959 by Arthur Samuel, an IBM employee and pioneer in the field of computer gaming and artificial intelligence. The synonym self-teaching computers was also used in this time period.*

*Please generate answer in <name></name> and <year></year> tags.*

*Assistant:*

### Output:

*<name>Arthur Samuel</name> <year>1959</year>*

## Example without clear indicator, with output

### User prompt:

*Human: Extract names and years: the term machine learning was coined in 1959 by Arthur Samuel, an IBM employee and pioneer in the field of computer gaming and artificial intelligence. The synonym self-teaching computers was also used in this time period.*

*Assistant:*

### Output:

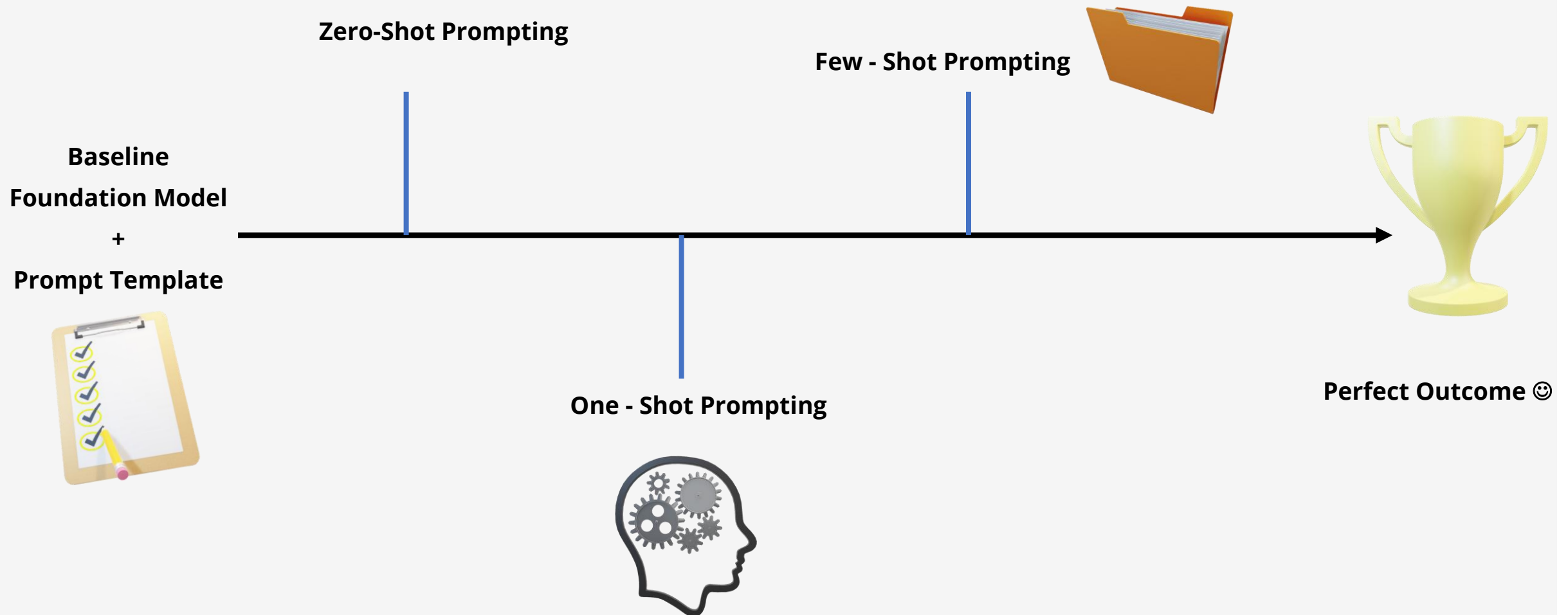
*Arthur Samuel - 1959*

# Prompt Engineering Techniques



# Prompt Engineering Techniques

---



# Zero-Shot Prompting

---

## Zero-Shot Prompting

- FMs are capable of **performing some simple tasks "zero-shot."**
- **No examples** needed to be provided to accomplish the task



## PROMPT

Write an engaging product description for a T-shirt for a clothing eCommerce site:

## Output

This T-shirt is perfect for any occasion. It's made of high-quality materials and it's comfortable to wear. It's also stylish and versatile, so you can wear it with anything.

# One -Shot Prompting

---

## One-Shot Prompting

- One-Shot Prompting provides **examples to the model** to assist the model to comprehend and generate responses

### PROMPT

**## Write an engaging product description for a clothing eCommerce site.**

**Product:** Women's Boho Beach Dress **Features:** - Midi dress - Swing hem - Slimming - Special for summer

**Description:** Women's Boho Beach Dress The style of midi dress with swing hem is slimming, .....

**## Write an engaging product description for a clothing eCommerce site.**

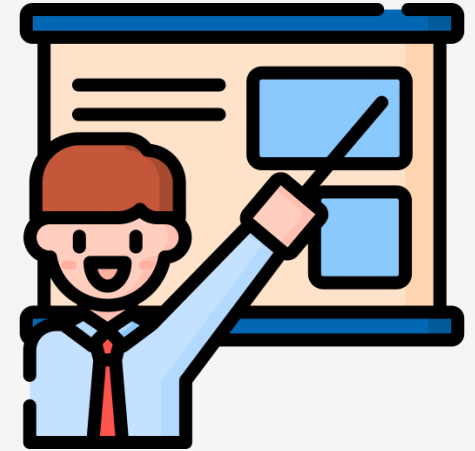
Make sure to include the following features in the description.

**Product:** Humor Men's Graphic T-Shirt **Features:** - Soft cotton - Short sleeve - Have a print of Einstein's quote: "artificial intelligence is ....."

**Description:**

### Output

This soft cotton t-shirt is perfect for fans of the classic scientist, Albert Einstein. The quote "artificial intelligence is no match for natural stupidity" is a reminder that sometimes the most intelligent people are the most foolish.....





# Few - Shot Prompting

## Few-Shot Prompting

- Few examples provided to the Model to generate high quality output

### PROMPT

Write an engaging product description for a clothing eCommerce site. Make sure to include the following

Product: Camp Director Crossbody Bag

Features:

- Canvas purse
- Rainbow space print
- Leather trim
- Two securely-zipped compartments

Description:

This Camp Director Crossbody Bag is the perfect accessory for carrying your essentials in style. The canvas

##

Write an engaging product description for a clothing eCommerce site. Make sure to include the following

Product: Women's Boho Beach Dress

Features:

- Midi dress
- Swing hem
- Slimming
- Special for summer

Description:

Women's Boho Beach Dress The style of midi dress with swing hem is slimming, It will make your turn

##

Write an engaging product description for a clothing eCommerce site. Make sure to include the following

Product: Humor Men's Graphic T-Shirt

Features:

- Soft cotton
- Short sleeve
- Have a print of Einstein's quote: "artificial intelligence is no match for natural stupidity"

Description:



## Output

This Humor Men's Graphic T-Shirt is perfect for anyone who wants to make a statement.

The soft cotton and short sleeve make it comfortable to wear, and the print of Einstein's quote is sure to get a chuckle.

Whether you're a fan of science or just looking for a funny shirt, this shirt is a great choice.

# ***Model Customization***



# Model Customization in Amazon Bedrock

---

**Model customization in Amazon Bedrock** is done to improve a model's performance, accuracy, and relevance for specific, proprietary business use cases and tasks.

## **Model Customization Approaches:**

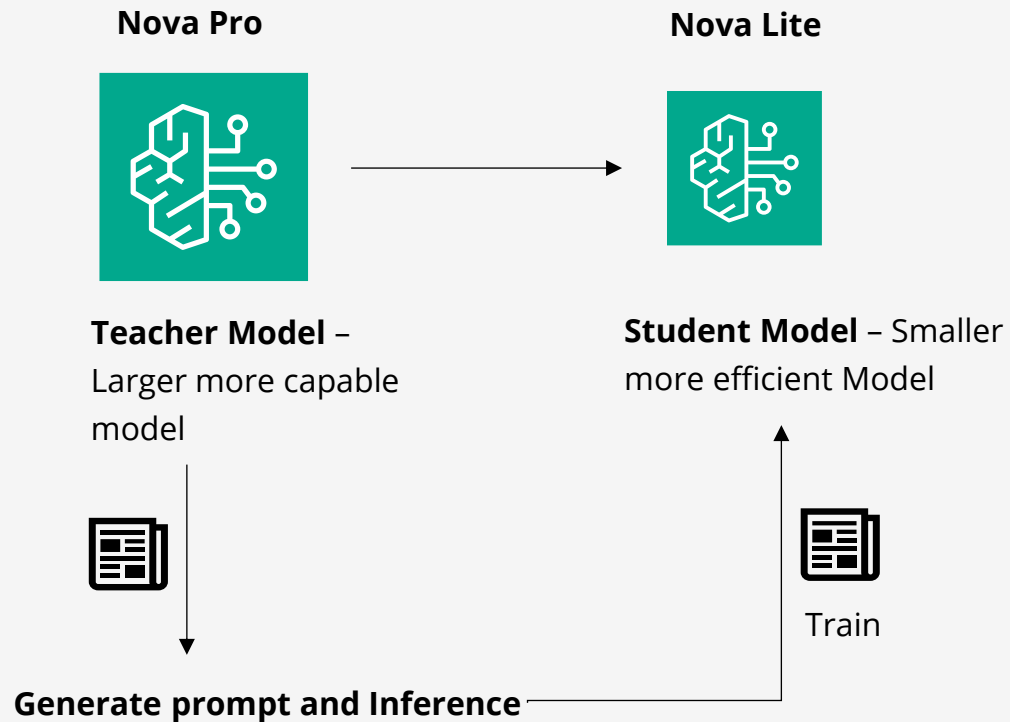
1. Model Distillation
2. Fine Tuning
3. Continued Pre-Training
4. Reinforcement fine-tuning

# ***Model Customization :***

## ***1. Model Distillation***



# Model Customization - 1. Model Distillation



Transfers knowledge from a large, accurate **teacher model** to a smaller, faster, and cheaper **student model**.

- You provide prompts (and optionally labeled examples).
- Bedrock generates high-quality responses using the teacher model.
- These responses are used to fine-tune the student model.

**Best for:** Achieving near-large-model accuracy at lower latency and cost.

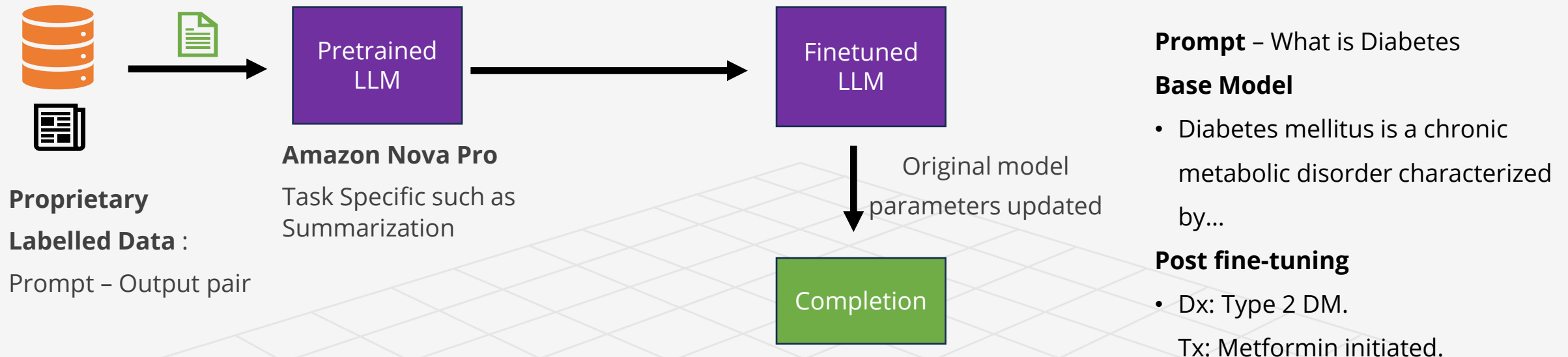
***Model Customization :***  
***2. Supervised fine-tuning***



# Model Customization– 2. Supervised fine-tuning

- Used to improve **performance on specific tasks** – Summarization, Q&A - Style, format, or depth
- Training dataset of **labeled examples** used for fine tuning.
- Model learns to associate what types of outputs should be generated for certain types of inputs.

```
{  
  "prompt": "What is Type 2 diabetes?",  
  "output": "Dx: Type 2 DM"  
}  
[Abbreviated, clinical style]
```



***Model Customization :***  
***3. Continued Pre-training***



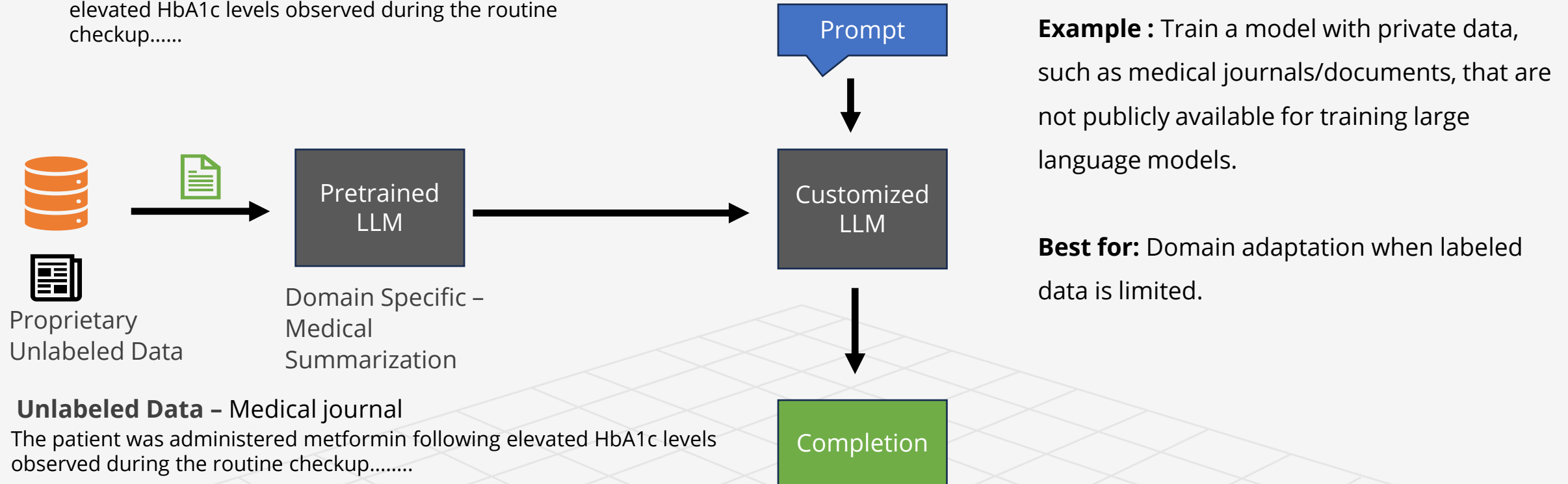


# Model Customization – 3. Continued pre-training

- Process of further training a **pre-trained language model to improve domain knowledge of the model.**
- Trained on a large corpus **of unlabeled domain text.**

## ***Transcript from medical journal :***

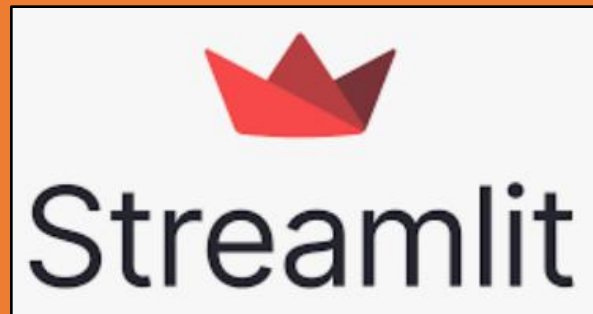
The patient was administered metformin following elevated HbA1c levels observed during the routine checkup.....



# Mental Model - Choosing the Right FM Adaptation Strategy

| Technique                                                                                                           | Approach                                                                         | Use Case Scenario                                                                                | Pros                                                                   | Cons                                  |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------|
| <b>Prompt Engineering</b><br>      | Improve output with <b>better prompts</b> .<br>No model training required.       | Summarization Assistant - "Summarize vs Summarize in one page." (user + system prompt)           | Fastest and lowest cost                                                | Limited deep customization            |
| <b>RAG</b><br>                     | Connect LLM to <b>enterprise data sources</b> . No model training required.      | HR Chatbot - using policy PDFs and SharePoint docs                                               | Provides enterprise contextual answers using latest data               | Retrieval quality & latency important |
| <b>Model Distillation</b><br>      | Smaller <b>model learns from larger model</b> .                                  | Customer Support Chatbot - Low-cost, low latency but with output quality similar to larger model | Lowers latency and inference cost with quality similar to larger model | Some accuracy trade-off               |
| <b>Fine-Tuning</b><br>            | Retrain model on <b>labeled business data</b> .                                  | Physician Notes App with clinical tone - Dx: Type 2 DM., Tx: Metformin initiated.                | Improves style, format, task accuracy                                  | Needs quality training data           |
| <b>Continued Pretraining</b><br> | Further <b>train model on domain data</b> - unlabelled data (wind turbine guide) | Wind turbine engineering SME assistant – Yaw 3.84 misalignment reasons ?                         | Improves domain understanding                                          | High cost and complexity              |

# *Use Case – Building a Chatbot*



*Pre-Requisites*

*Building a Chatbot using Amazon Bedrock*

# Pre-requisites – How to install

1/2

- Download VS Code - <https://code.visualstudio.com/download>
- Install Python - 3.12.4 ----> <https://www.python.org/downloads/windows/>
  - `python --version`
  - `pip --version`
- Install AWS CLI - <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
  - `aws -- version`
  - Extensions – AWS Toolkit and AWS Boto3
- Configure IAM Role for VSCode - <https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-files.html>
  - Create IAM Role
  - `aws configure`
  - Provide credentials
  - `aws s3 ls`

# Pre-requisites - How to install

---

2/2

- Anaconda Navigator Download - <https://docs.anaconda.com/free/anaconda/install/windows/>
- Open VS Code from Anaconda Navigator – **Very Important**
- Install boto 3 - `pip install boto3==1.37.11`
- Install Langchain – `pip install langchain==0.3.0`
- Install Langchain-AWS Module – `pip install -U langchain-aws`
- Install Streamlit – `pip install streamlit==1.43.1`
- Install Transformers - `pip install transformers`
- Install PyYAML - `pip install PyYAML`

*Coding Steps*

*Building a **Chatbot** using Amazon Bedrock*

# Steps in Building Chatbot – Chatbot Backend Code

---

#1 import the ConversationSummaryBufferMemory, ConversationChain, ChatBedrock or ChatBedrockConverse Langchain Modules

#2a Write a function for invoking model- client connection with Bedrock with profile, model\_id & Inference params- model\_kwargs

#2b Test out the LLM with invoke method

#3 Create a Function for ConversationBufferMemory (llm and max token limit)

#4 Create a Function for Conversation Chain - Input text + Memory

#5 Chat response using invoke (Prompt template)

#Links :

<https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html>

[https://docs.aws.amazon.com/bedrock/latest/APIReference/API\\_runtime\\_Converse.html](https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_Converse.html)

<https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html>



# Converse API

---

# Set the model ID, e.g., DeepSeek-R1 Model.

```
model_id = "us.deepseek.r1-v1:0"
```

# Start a conversation with the user message.

```
user_message = "Type_Your_Prompt_Here"
```

```
conversation = [ { "role": "user", "content": [{"text": user_message}], } ]
```

#Converse URL

[#https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html](https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html)

[#https://docs.aws.amazon.com/bedrock/latest/APIReference/API\\_runtime\\_Converse.html](https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_Converse.html)

[#https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html](https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html)

# Steps in Building Chatbot – Chatbot Frontend Code

---

- How to Run Chatbot locally
  - `streamlit run <frontend_filename>.py`

# Langchain Overview :



*Challenges in Large Language Models  
&  
How Langchain helps overcome them*

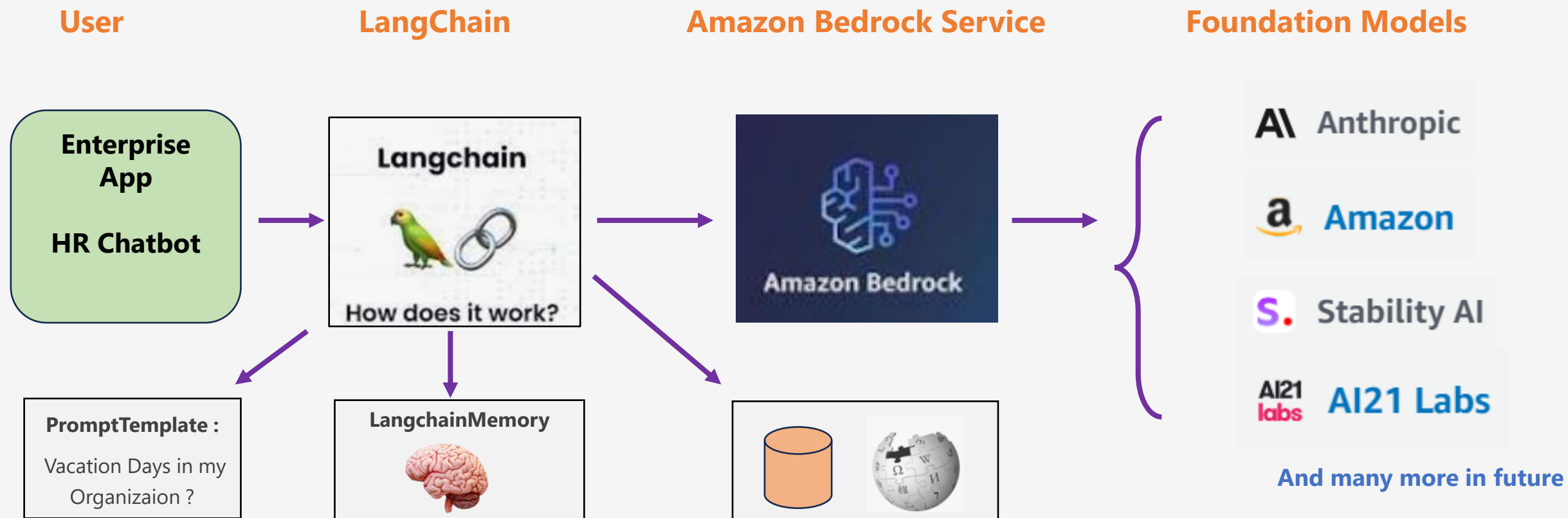
# Challenges in Large Language Models

---

- **Contextual Memory** - Large Language Models are **Stateless**
- **Contextual Knowledge (Domain Knowledge)** - Enhancing the outcome of LLM's with **organization proprietary data**
- **Hallucination** – Due Lack of knowledge of **recent events (post training date)** or no training data in the specific area.

# What is LangChain ?

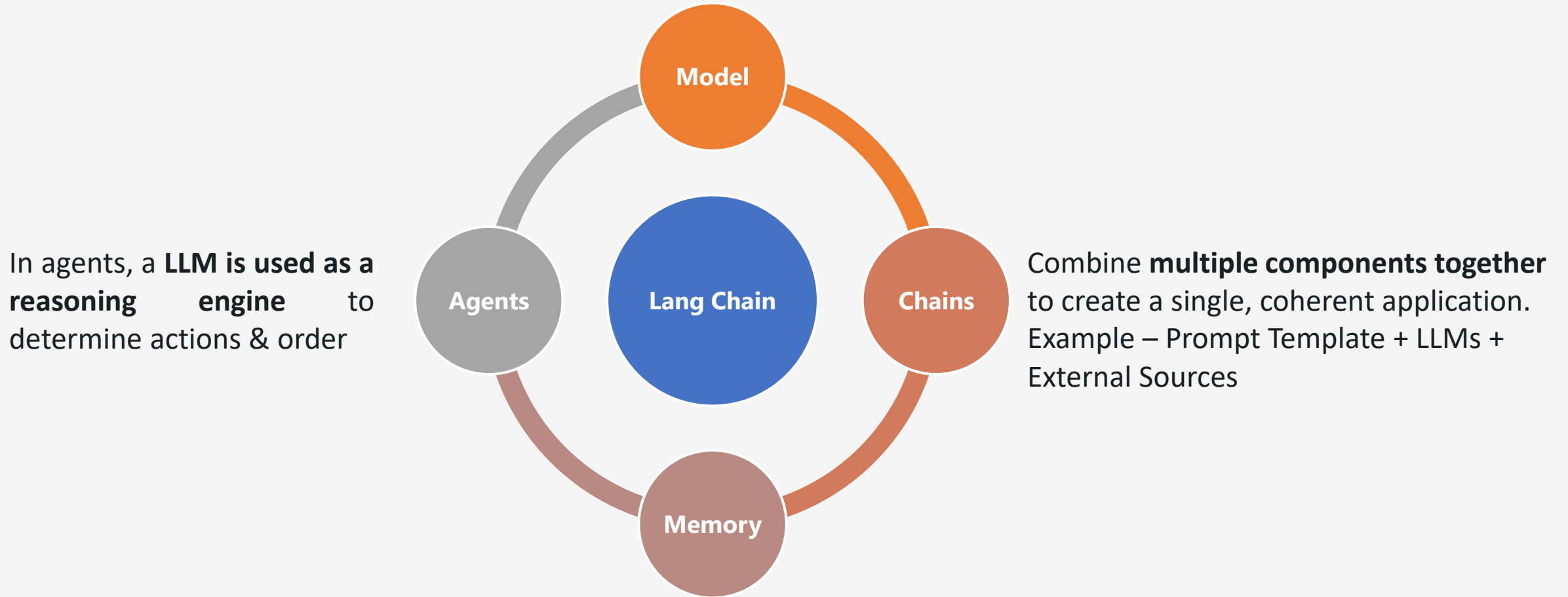
**LangChain** is a **framework for developing applications** powered by language models.



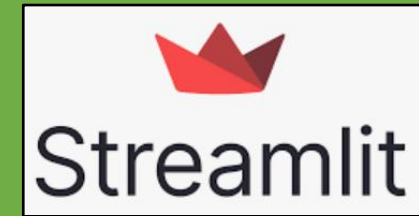
# Lang Chain Components – High Level Overview

---

**Standard interface** to connect different LLMs such as Claude, Titan



Provides ability to **store information about past interaction** - Chatbots



# *Streamlit – Overview*

***Use Case : Retrieval Augmented Generation (RAG)***

***Employee HR Q & A Application with RAG***

***Amazon Bedrock - Claude FM***

***+***

***Langchain***

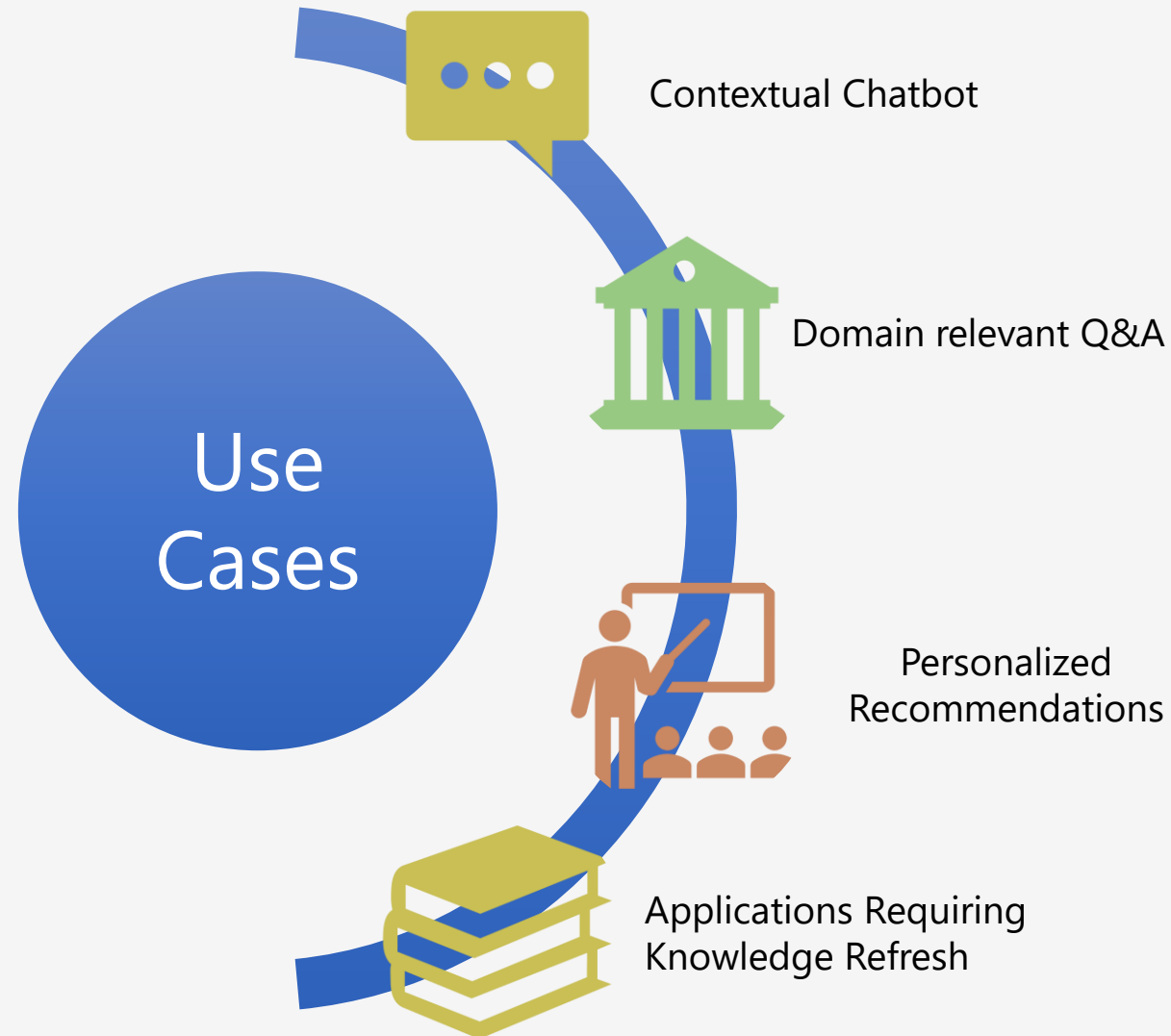
***+***

***HR Policy Document (PDF)***



# Key Use Cases for Retrieval Augmented Generation in the Industry

---



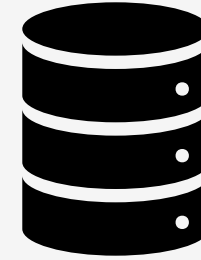
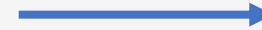
***Basic Introduction***  
***to***  
***Vectors, Embeddings and Vector Search***

# 1. Why do we need Vectors ?

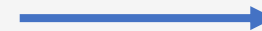
---



**Structured Data**



**Image Search or Unstructured Data**



???

## Key Terms

1. **Vectors**
2. Embedding Models
3. Data Chunking
4. Vector Store
5. Vector Search
6. Cosine Similarity
7. K-Nearest Neighbor Search

# 1. What are Vectors ?

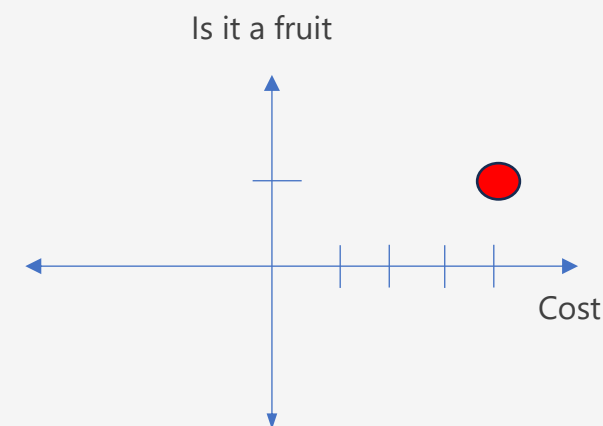
**Vectors** are **mathematical representation of words, sentences** and documents

**Vectors** are represented by **List of numbers** or sequence of numbers



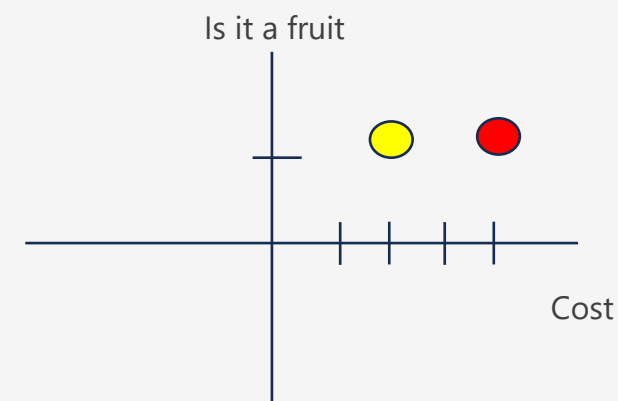
| Is it a fruit | Cost |
|---------------|------|
| 1             | \$4  |

[ 1 , 4 ]



| Is it a fruit | Cost |
|---------------|------|
| 1             | \$2  |

[ 1 , 2 ]

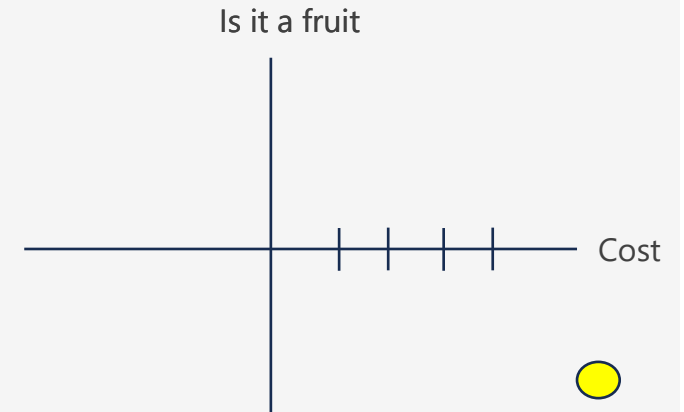


# 1. What are Vectors ?

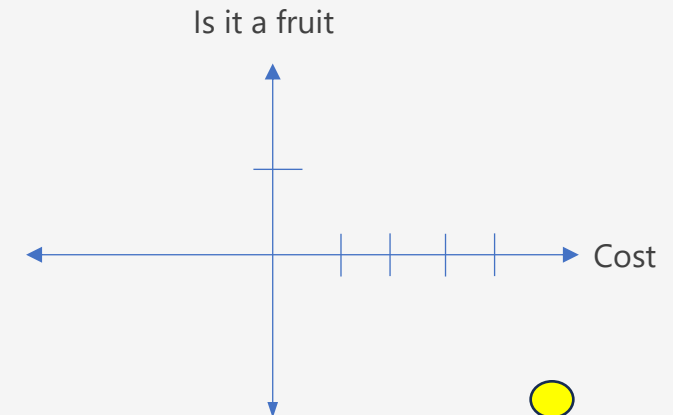
---



| Is it a fruit | Price |
|---------------|-------|
| -1            | \$50  |

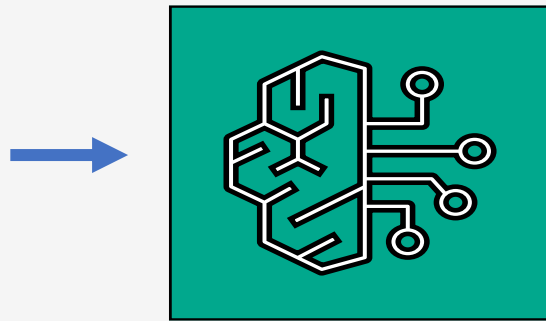
$$[-1, 50]$$


| Is it a fruit | Price |
|---------------|-------|
| -1            | \$40  |

$$[-1, 40]$$


## 2. What are Embedding Models ?

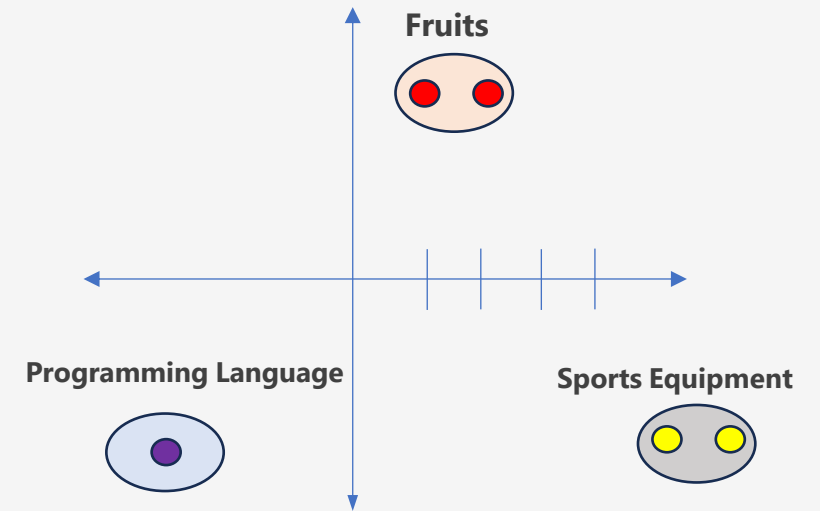
- Apple
- Banana
- Grapes
- Football
- Soccer
- GO
- Tennis



Amazon Titan Embeddings

$[0.4, 0.2, 0.1, \dots]$   
 $[0.4, 0.2, 0.1, \dots]$   
 $[0.4, 0.2, 0.1, \dots]$   
 $[0.4, 0.2, 0.1, \dots]$

1536 Dimensions



Word Embeddings  
Ex : great

Sentence Embeddings  
Ex. : Price of apple ?

Doc Embeddings

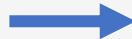
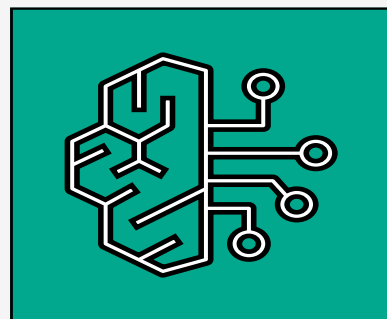
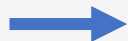
### Key Terms

1. Vectors
2. **Embedding Models**
3. Data Chunking
4. Vector Store
5. Vector Search
6. Cosine Similarity
7. KNN

# Embedding Models

## Output from Embedding Models

Fruit



Amazon Titan Embeddings

```
[
  0.022321066, -0.027544279, -0.006137953, 0.0024092742, -0.0003515296,
  -0.007889225, -0.01853968, -0.037405808, -0.004294867, -0.012602357,
  0.009385457, 0.02421176, -0.004383281, -0.011901848, -0.020702416,
  0.00431187, 0.055986296, 0.0040738326, 0.021300908, -0.03713377,
  -0.004944368, 0.012269106, 0.02614326, -0.019777471, 0.0017206672,
  0.014867109, 0.004597514, -0.021913003, -0.0042336574, 0.013180447,
  0.032699477, -0.027245032, -0.0112965545, -0.009358253, -0.004951169,
  -0.015030335, -0.007501565, -0.018906936, 0.0074675595, -0.0067568496,
  0.0011952856, 0.0035501514, 0.012316713, 0.016132105, -0.014839904,
  -0.0044002836, 0.010405616, -0.0066276295, -0.015751246, -0.007141109,
  0.02729944, -0.00031348618, -0.015669633, -0.015656032, 0.015696838,
  0.013554505, -0.001635654, 0.006069943, 0.009167824, -0.01917898,
  -0.006059741, 0.013969369, -0.03006067, 0.007691995, 0.0005028531,
  -0.005468049, -0.007263528, 0.009970348, -0.016254524, 0.0041248407,
  0.01964145, 0.037324198, -0.0011340762, -0.0013296065, 0.010392014,
  -0.01268397, -0.013037625, -0.006811258, -0.0045601083, 0.0028955496,
  0.027435461, -0.024320576, -0.006161757, 0.014350229, 0.009392259,
  -0.006328383, -0.025490358, 0.023273215, 0.0083925035, -0.003934411,
  0.023722084, -0.0072499258, -0.0053626327, 0.017818768, -0.03327077,
  0.019097365, 0.0022086431, 0.046709653, -0.016825814, -0.032182597,
  ... 1436 more items
]
```

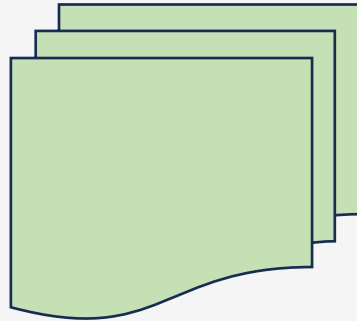
1536 Dimensions

### 3. Text Splitting or Data Chunking

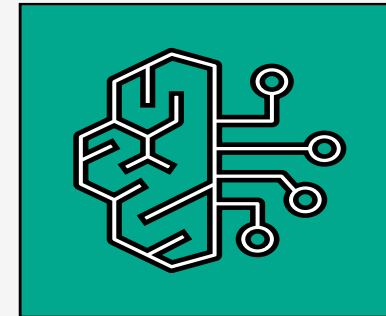
---



Data Source - PDF



- Split by Character
- Split by Token
- Split by Code



Amazon Titan Embeddings

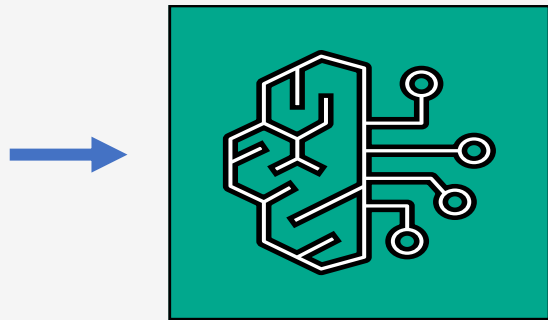
#### Key Terms

1. Vectors
2. Embedding Models
3. **Data Chunking**
4. Vector Store
5. Vector Search
6. Cosine Similarity
7. K-Nearest Neighbor Search



## 4. Vector Store?

- Apple
- Banana
- Grapes
- Football
- Soccer
- GO
- Tennis

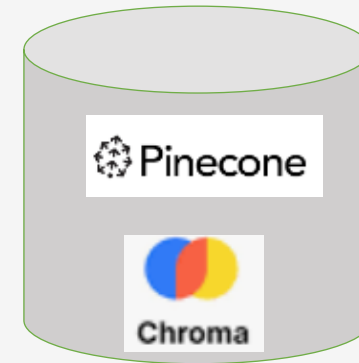


Amazon Titan Embeddings

[ 0.4, 0.2, 0.1, ... ]  
[ 0.4, 0.2, 0.1, ... ]  
[ 0.4, 0.2, 0.1, ... ]  
[ 0.4, 0.2, 0.1, ... ]

1536 Dimensions

### Vector Store

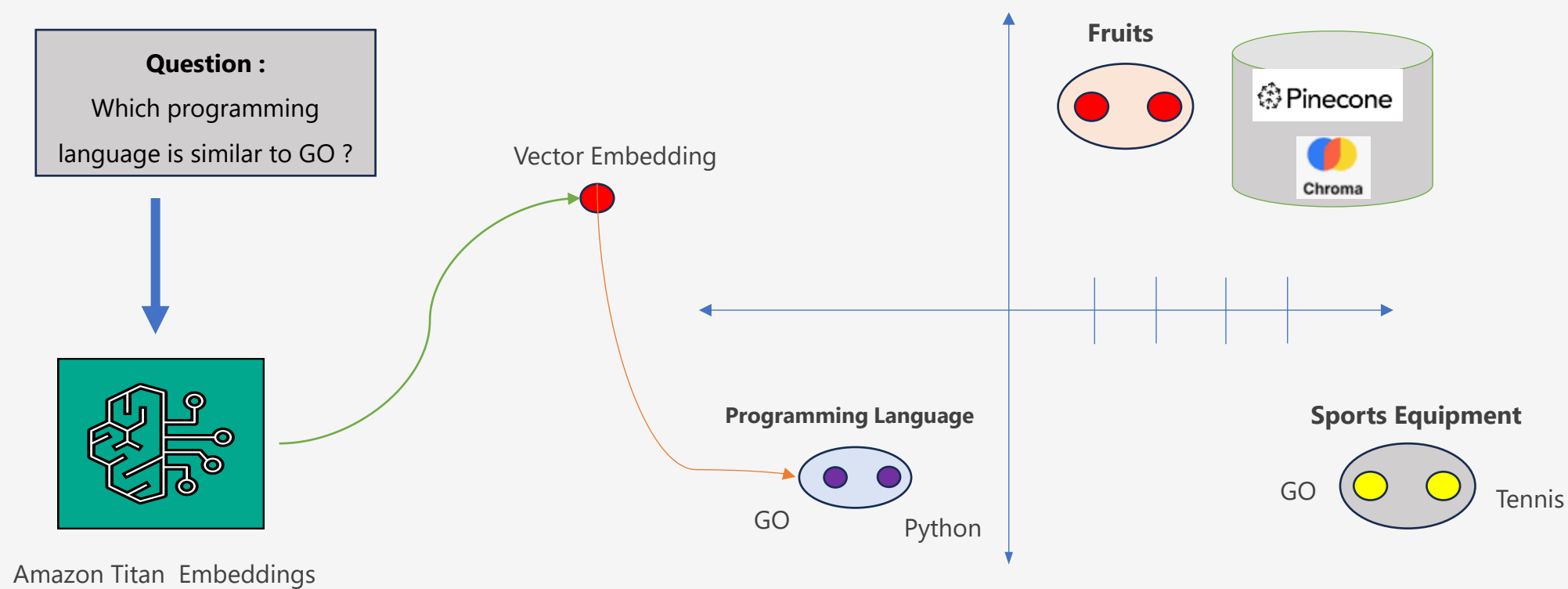


- ✓ FAISS
- ✓ Pinecone
- ✓ Chroma DB

### Key Terms

1. Vectors
2. Embedding  
Models
3. Data Chunking
4. **Vector Store**
5. Vector Search
6. Cosine Similarity
7. K-Nearest  
Neighbor Search

# 5. Vector Search

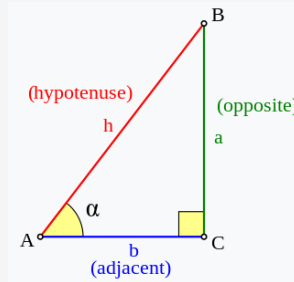


## Key Terms

1. Vectors
2. Embedding Models
3. Data Chunking
4. Vector Store
5. **Vector Search**
6. Cosine Similarity
7. K-Nearest Neighbor Search

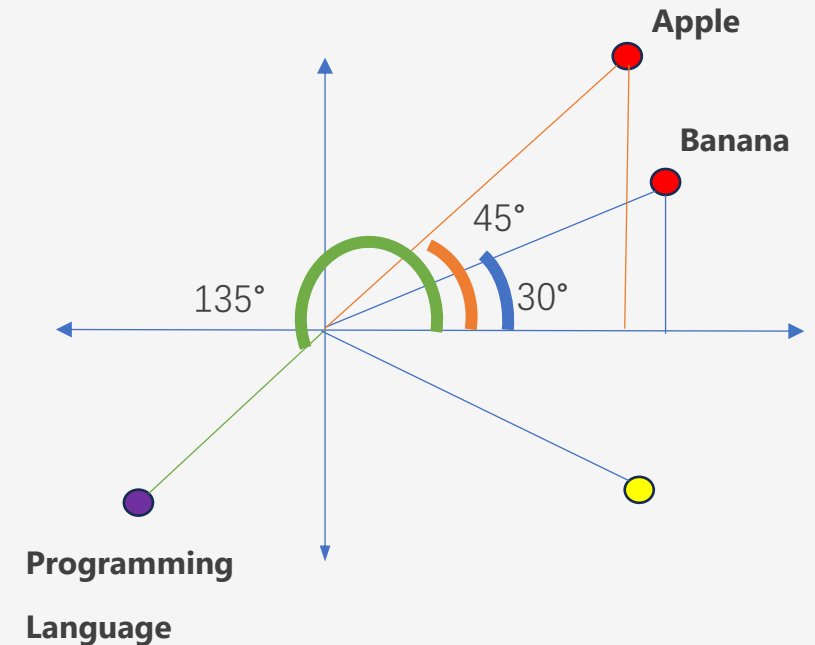
## 6. Similarity Search – Cosine Similarity

### 1. $\text{Cosine } \theta = \text{adjacent/hypotenuse}$



### 2. Similarity Search using Cosine Distance:

- Cosine distance metric is used to find **similarities between different vectors**
- In **cosine metric** we measure the **degree of angle between two vectors**
- **$\text{Cos } 0 = 1$ ,  $\text{Cos } 90 = 0$  and  $\text{Cos } 180 = -1$**
- **Cosine value 1** is for **vectors pointing in the same direction --- > Similar**
- At Cosine Value 0 ---- > Vectors unrelated (some similarity found).
- Value -1 for vectors pointing in opposite directions(No similarity).



## 7. Cosine Similarity and K- Nearest Neighbor Algorithm or ANN

KNN is a ***supervised machine learning algorithm*** used **classification and regression** tasks.

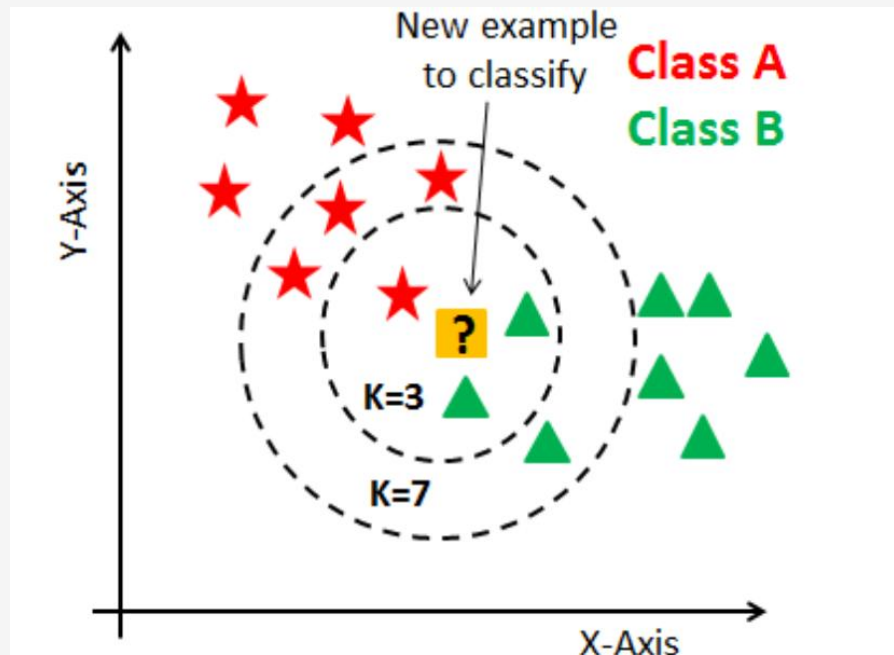


Image Source : Internet

### Key Terms

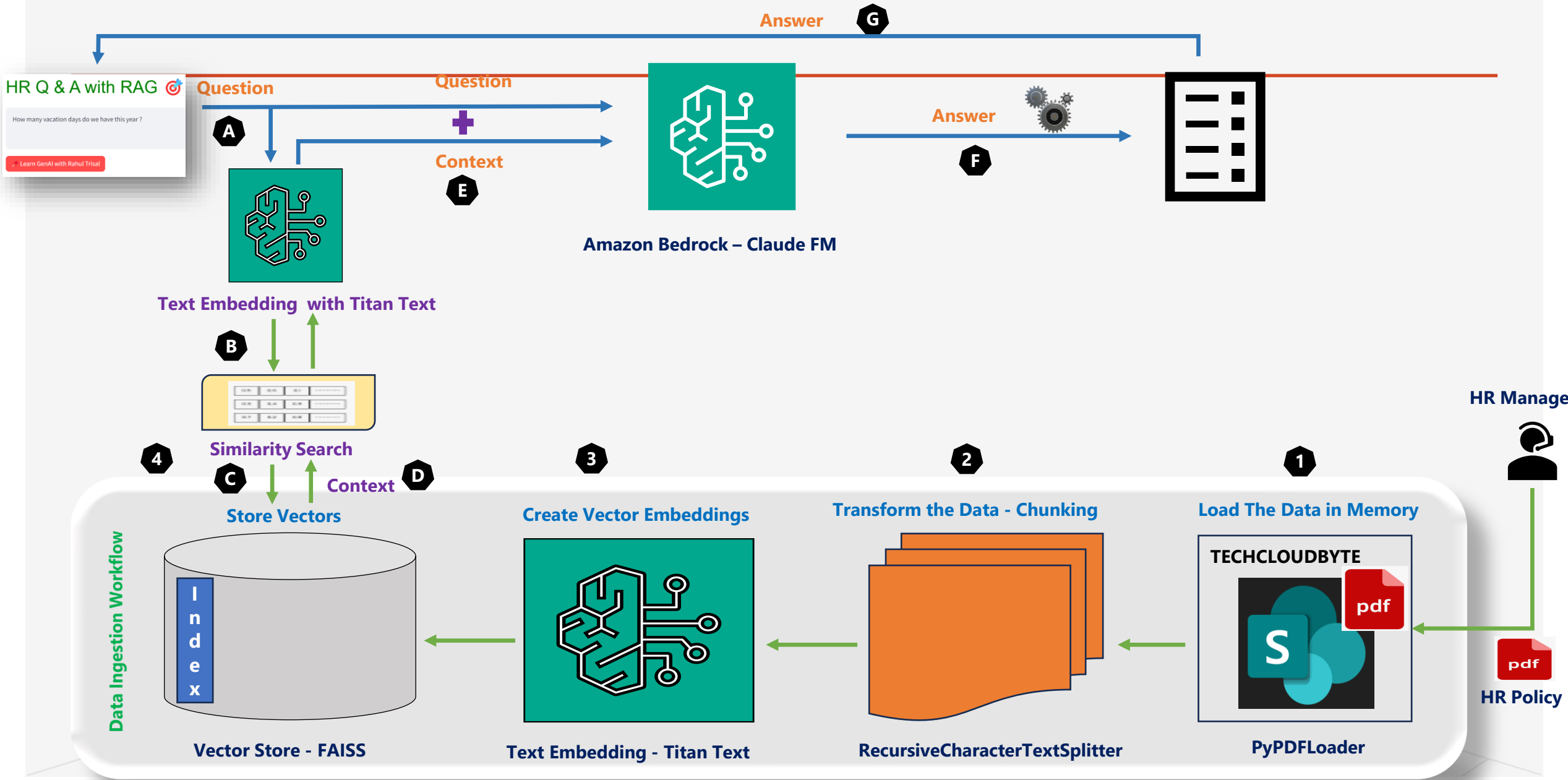
1. Vectors
2. Embedding Models
3. Data Chunking
4. Vector Store
5. Vector Search
6. Cosine Similarity
7. KNN

***ANN - Approximate Nearest Neighbor***

*Use Case :*

***Employee HR Q & A Application with RAG***

# HR Question & Answer Application with Retrieval Augmented Generation - Architecture



***Use Case : Retrieval Augmented Generation (RAG)***

***Employee HR Q & A Application with RAG***

***Pre- Requisites - Installation***

# Pre-requisites – List of items to be installed

---

1. **pip3 install flask-sqlalchemy**
  2. **pip install pypdf**
  3. **pip install faiss-gpu** (for CUDA supported GPU)
- or**
4. **pip install faiss-cpu** (depending on Python version)
  5. In case of issue with **import PyPDFLoader**, run below command

**pip install -U langchain-community**



## Pre-Requisites

*For installation of other items for this use case, if not already installed, refer to below Section and Lecture (one shown in video as well). If already installed, skip next few slides.*

*Section : Building a Chatbot with Llama 2, Langchain & Streamlit*

*Lecture : Chatbot - Environment Setup before coding*

***Use Case : Employee HR Q & A App with RAG***

***Coding Steps***

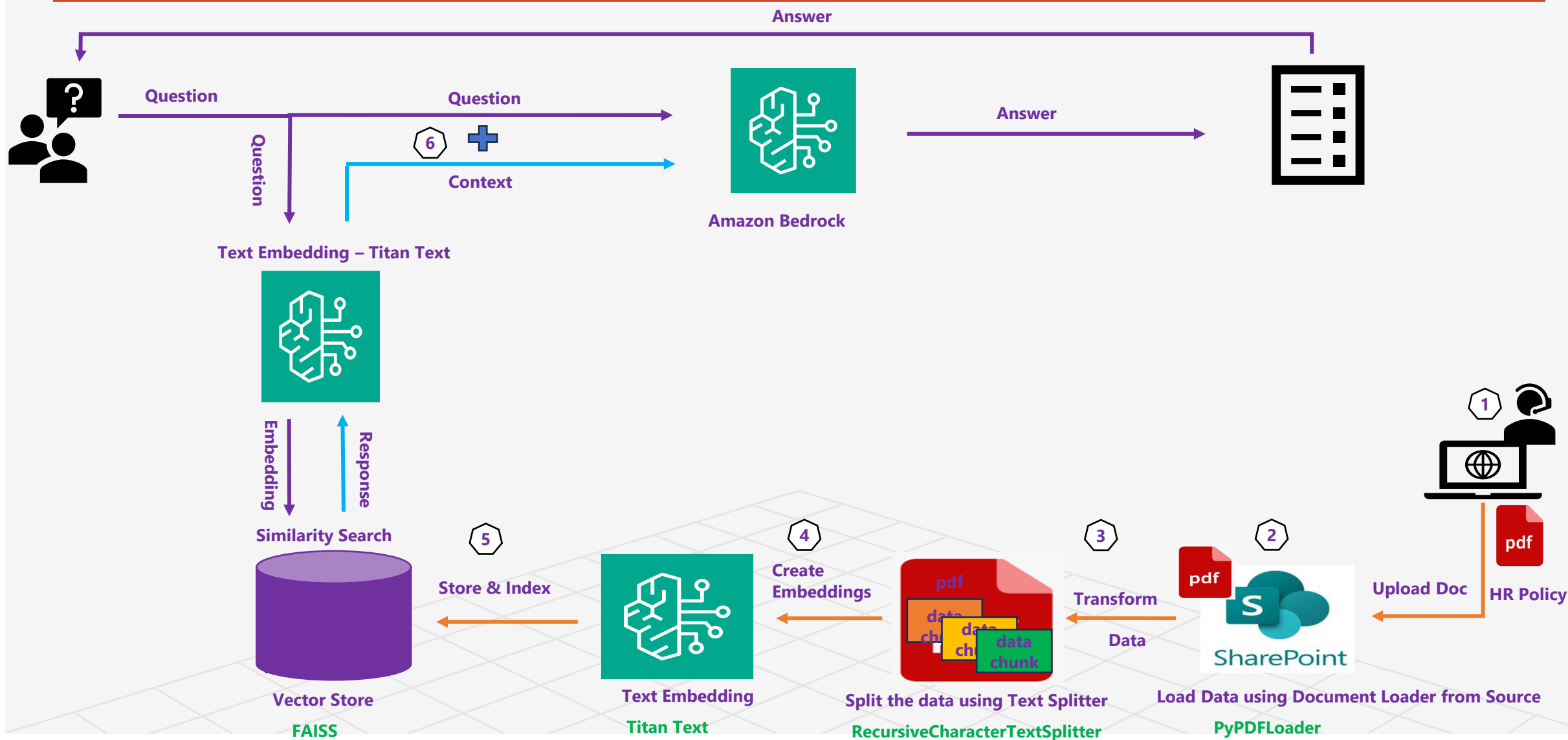
**QUICK ALERT**

# Steps in Building HR Q&A Solution with RAG – Backend Code

---

- #1. Import OS, Document Loader, Text Splitter, Bedrock Embeddings, Vector DB, VectorStoreIndex, Bedrock-LLM
- #2. Define the data source and load data with PDFLoader(<https://www.upl-ltd.com/images/people/downloads/Leave-Policy-India.pdf>)
- #3. Split the Text based on Character, Tokens etc. - Recursively split by character - ["\n\n", "\n", " ", ""]
- #4. Create Embeddings -- Client connection
- #5a Create Vector DB, Store Embeddings and Index for Search - VectorstoreIndexCreator
- #5b Create index for HR Report
- #5c. Wrap within a function
- #6a. Write a function to connect to Bedrock Foundation Model
- #6b. Write a function which searches the user prompt, searches the best match from Vector DB and sends both to LLM.
- # Index creation --> <https://api.python.langchain.com/en/latest/indexes/langchain.indexes.vectorstore.VectorstoreIndexCreator.html>

# HR Question & Answer App with RAG - Solution Architecture



# Steps in Building HR Q&A Solution with RAG – Frontend Code

---

- Modify the Streamlit frontend code where `###` highlighted
- Run Chatbot locally
  - `streamlit run <frontend_filename>.py`

# Data Source URL

---

<https://www.upl-ltd.com/images/people/downloads/Leave-Policy-India.pdf>

# Common Issues for the Use Case and Solution

---

## ❖ Issue 1 - Incase you face issue with **import PyPDFLoader** (due to depreciation by langchain)

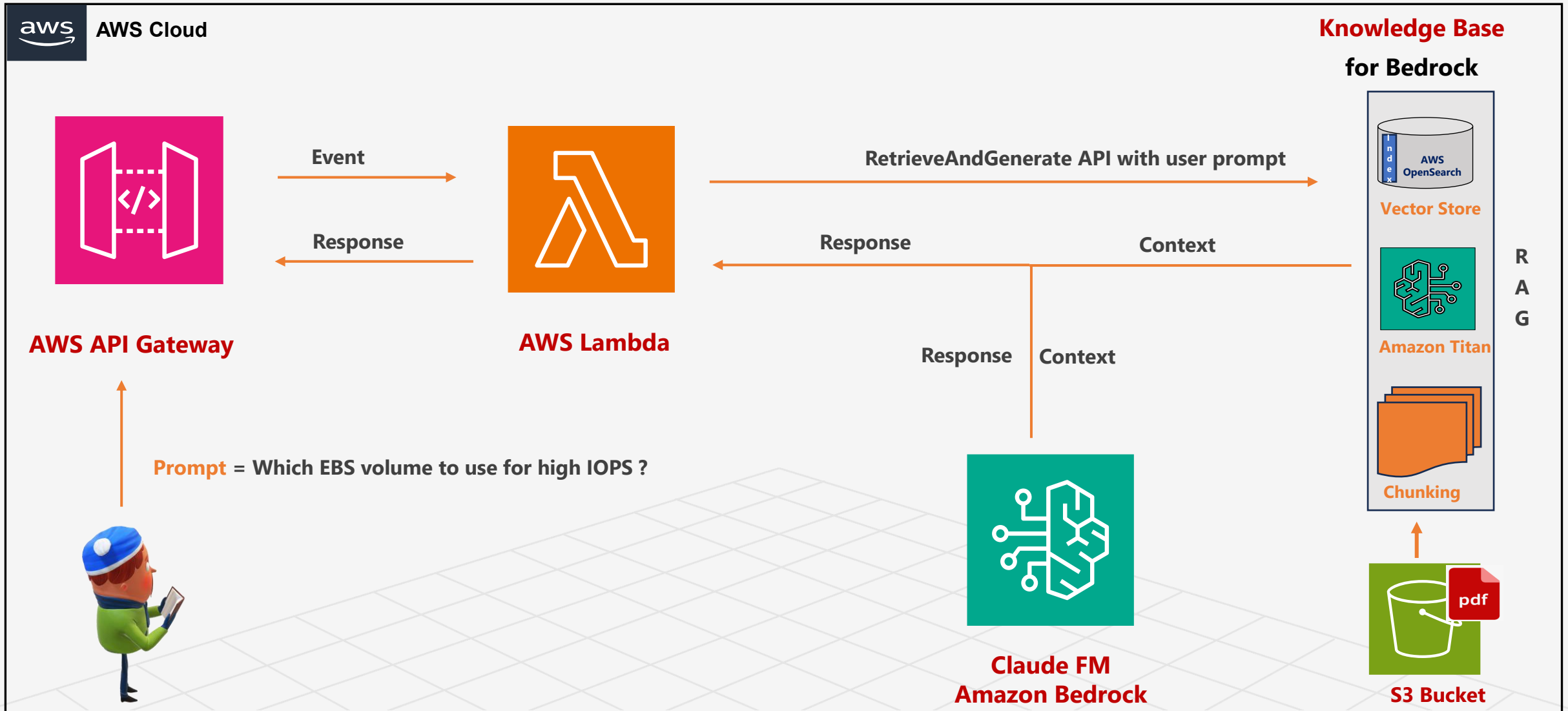
- `pip install -U langchain-community`
- `from langchain_community.document_loaders import PyPDFLoader`

*Use Case :*

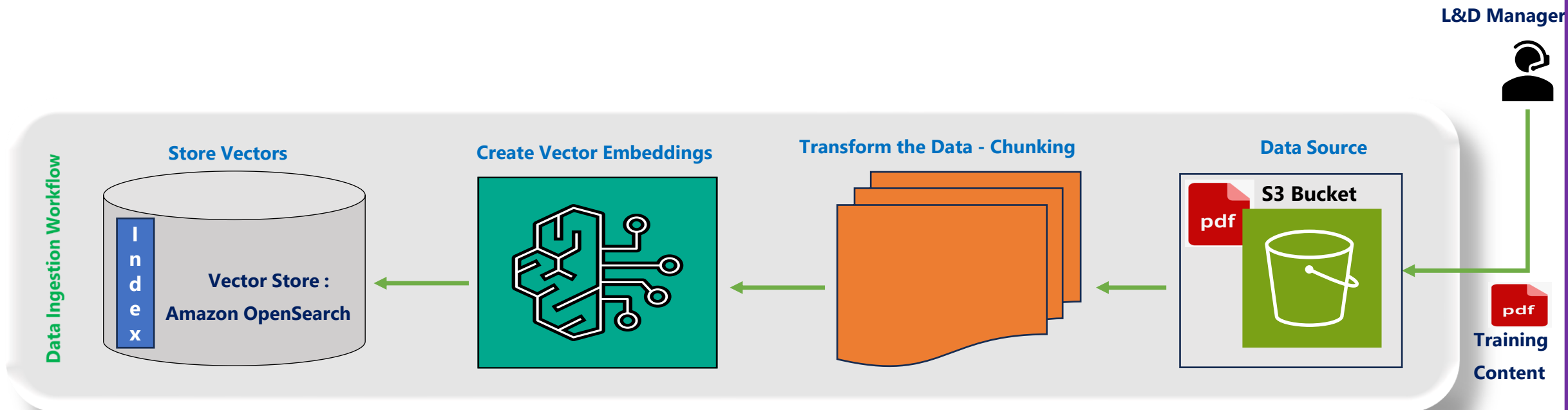
**Serverless e-Learning App using  
Knowledge Base for Bedrock**



# Serverless e-Learning App using Knowledge Base for Bedrock



# E-Learning Solution using RAG - Amazon Bedrock Knowledge Base + LLM



# Data Ingestion Workflow - Options

---

## 1. Data Sources

- ❖ S3

## 2. Chunking

- ❖ Default
- ❖ Fixed Size

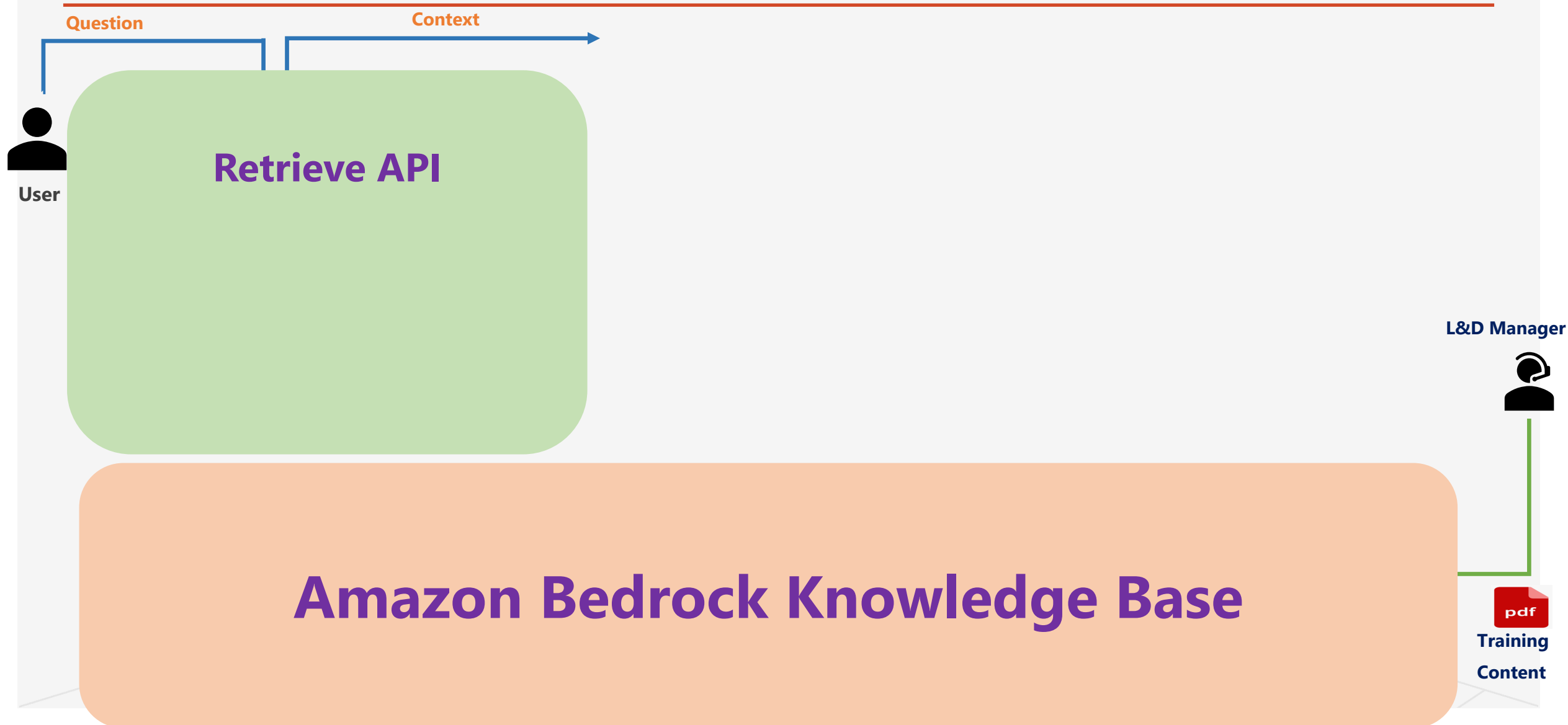
## 3. Embedding Model

- ❖ Cohere
- ❖ Amazon Titan

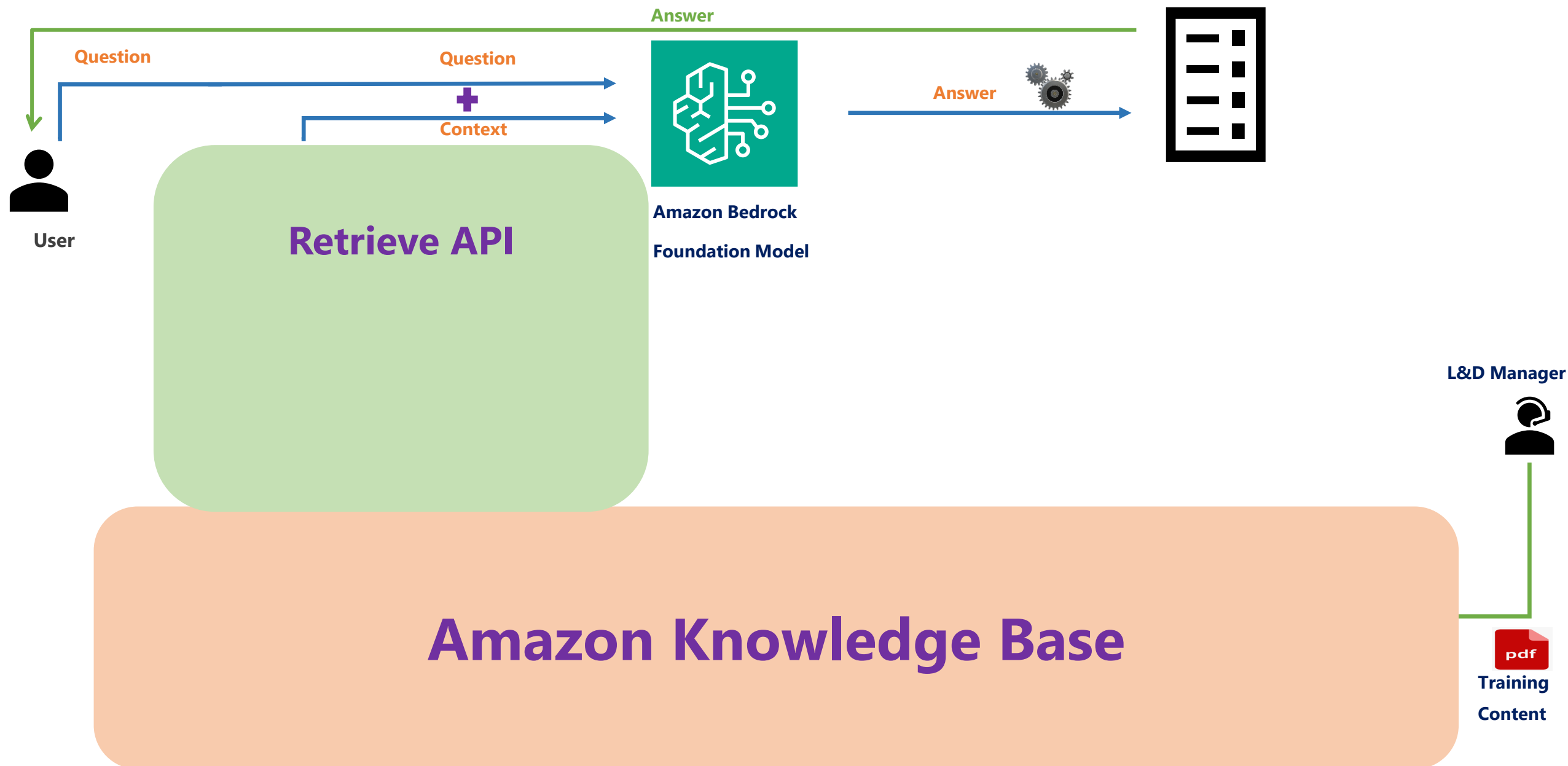
## 4. Vector DB

- ❖ Amazon OpenSearch Serverless vector store
- ❖ Amazon Aurora
- ❖ Pinecone
- ❖ Redis Enterprise Cloud

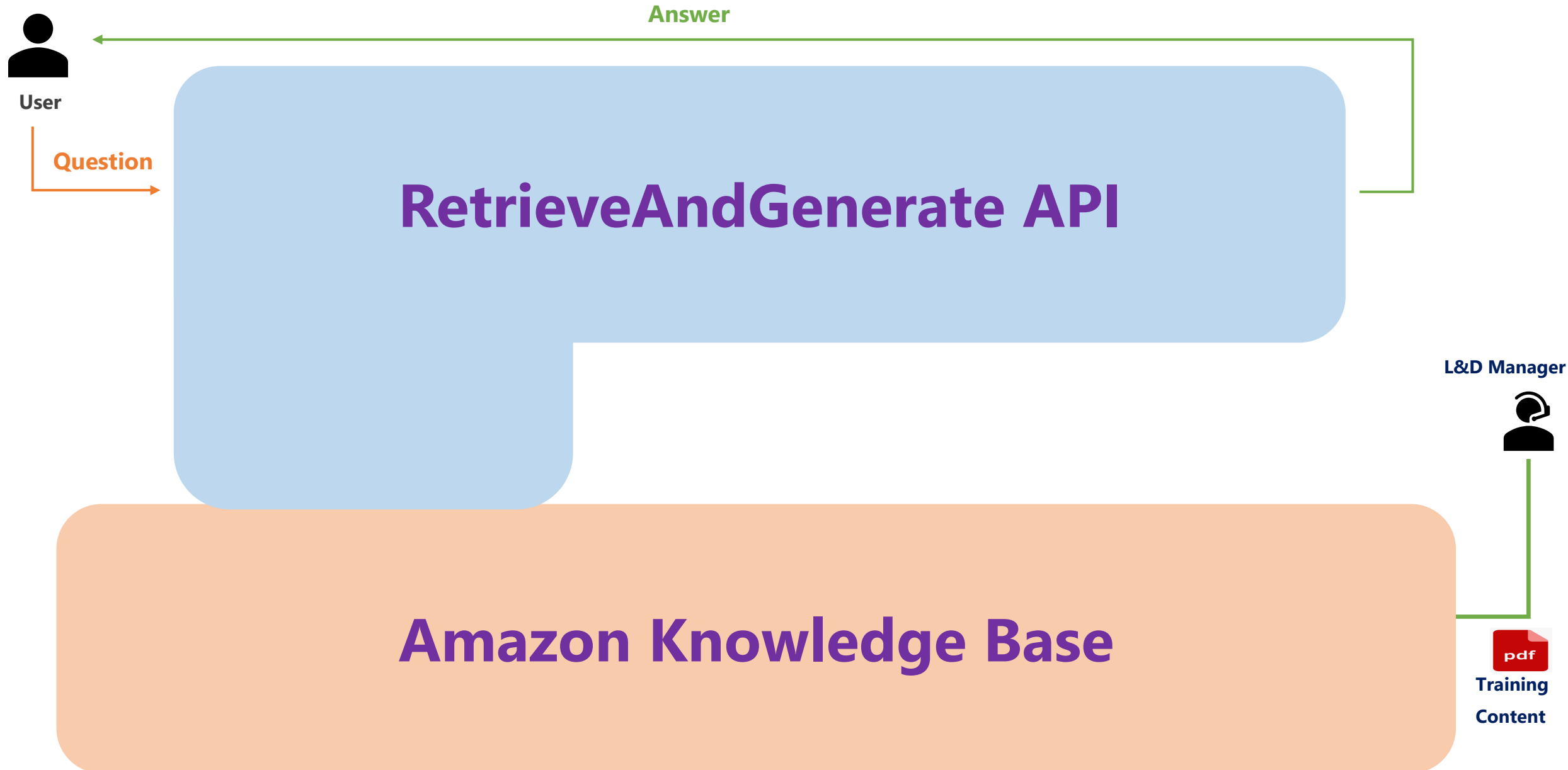
# L&D App with Amazon Knowledge Base (Retrieval Augmented Generation) - Architecture



# L&D App with Amazon Knowledge Base (Retrieval Augmented Generation) - Architecture



# L&D App with Amazon Knowledge Base (Retrieval Augmented Generation) - Architecture



# Two API's for Retrieval – Retrieve API and RetrieveAndGenerate API

---

## 1. Retrieve API :

- Queries a Knowledge Base
- **Retrieves** information from it.

[https://docs.aws.amazon.com/bedrock/latest/APIReference/API\\_agent-runtime\\_Retrieve.html](https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html)

## 2. RetrieveAndGenerate API

- Queries a knowledge base
- **Generates responses** based on the **retrieved results**.
- Response **cites up to five sources** but only selects the ones that are **relevant to the query**.

[https://docs.aws.amazon.com/bedrock/latest/APIReference/API\\_agent-runtime\\_RetrieveAndGenerate.html](https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerate.html)

# Broad Implementation Steps

---

## 1. **Data Source** –

- Create the **S3 Bucket**
- **Upload** the relevant **files** in the S3 Bucket

## 2. Create Amazon Bedrock **Knowledge Base** --- > **Data Source, Chunking, Embedding Model, Vector Store**

## 3. Create a **AWS Lambda Function**

- Create IAM Role
- Increase timeout limit
- Write the **RetrieveAndGenerate API** to access data from the Knowledge Base – [Link](#)
- Model ARN - 'modelArn': '**arn:aws:bedrock:us-east-1::foundation-model/anthropic.claude-instant-v1**'

## 4. Optional Step incase of error - Update boto3 version incase of error using Lambda Layers - [Link](#)

## 5. Create **REST API**

## 6. **Integrate** AWS Lambda function with API Gateway



# Broad Implementation Steps

---

## **Create REST API using AWS API Gateway**

- Create Resource
- Create Method – GET

## **API Gateway - Method Request**

- URL query string parameters – Add input parameter
  - Enable - Request validator
  - Validate body, query string parameters, and headers
- 

# Broad Implementation Steps

---

- **API Gateway - Integration Request**

application/json

```
{  
  "prompt": "$input.params('prompt')"  
}
```

- **Deploy the API to a Stage' (Stage=Dev or any other)**
- **Test using API Gateway Console and Postman**

*Use Case :*

***Build Retail Banking Agent***

# Amazon Bedrock Agents – Retail Banking Agent

## Bank of Chicago



John

Check account activation status after 3 days from Bank Website



Thomas



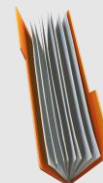
Manju



AccountID = 5555



AccountID = 6666



AccountID = 7777



GET UP TO  
\$500

# Amazon Bedrock Agents – Retail Banking Agent



**Manju**  
AccountID = 7777

→ ↺  www.bankofchicago.com ☆

Test TestAlias: Working draft ▼ Info

 Hi

 What is your account ID?  
[< Hide trace](#)



| DynamoDB Table |            |                |                       |
|----------------|------------|----------------|-----------------------|
| AccountID      | AccounName | Account Status | Reason                |
| 5555           | John       | Active         | Active                |
| 6666           | Thomas     | Pending        | InvalidIdentification |
| 7777           | Manju      | Pending        | InvalidAddressProof   |

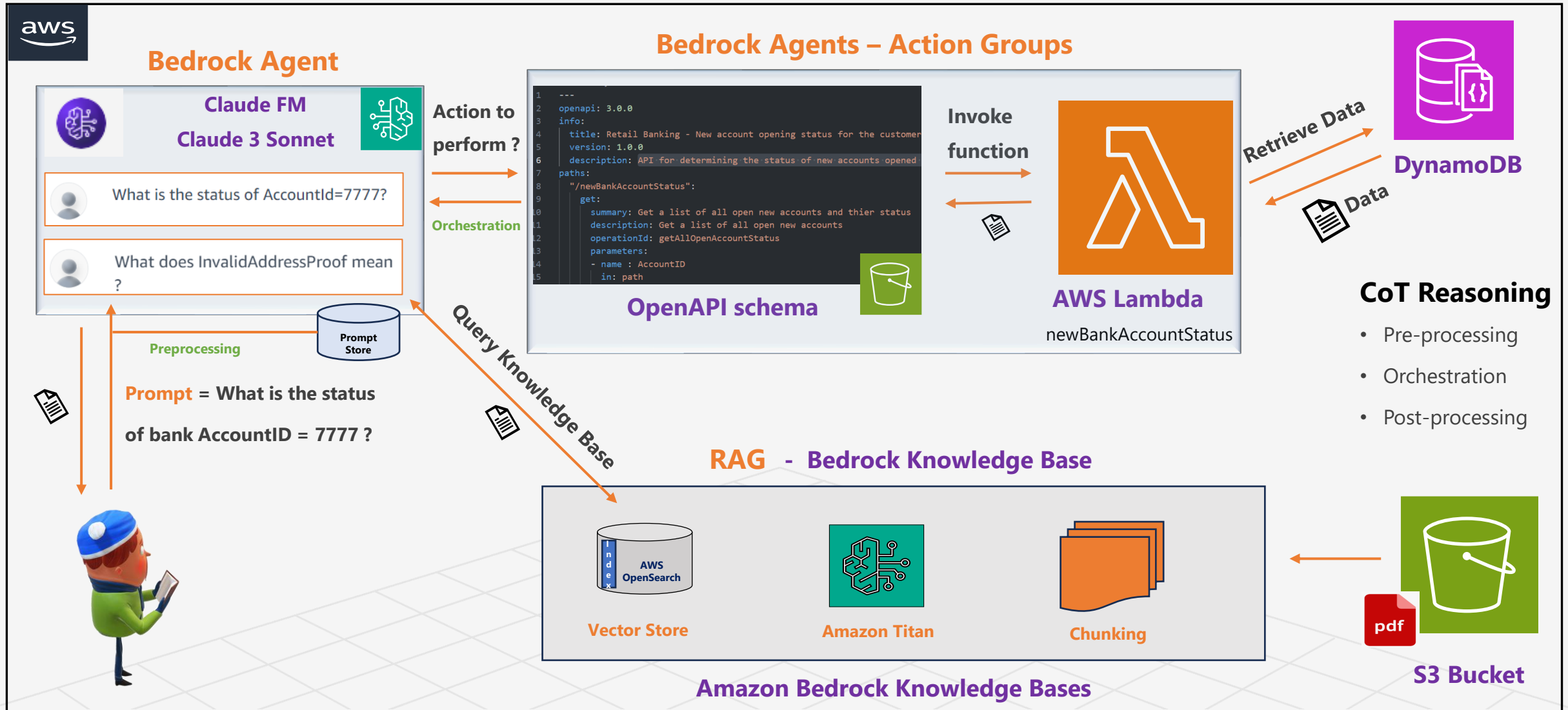
## Customer Data - DynamoDB Table

---

TableName - customerAccountStatus

| AccountID | AccounName | AccountStatus | Reason                |
|-----------|------------|---------------|-----------------------|
| 5555      | John       | Active        | Active                |
| 6666      | Thomas     | Pending       | InvalidIdentification |
| 7777      | Manju      | Pending       | InvalidAddressProof   |

# Retail Banking Agent - Amazon Bedrock Architecture



# Steps to create Amazon Bedrock Agent for Bank of Chicago

1/2

1. Create a DynamoDB Table with the customer account details - **TableName** - **customerAccountStatus**
2. Create a AWS Lambda function to retrieve the data from DynamoDB Table – **Lambda Function** - **newBankAccountStatus**
3. Define OpenAPI schemas for Bedrock Agent Action Group and Upload on S3 Bucket- [Link](#)
  - **S3 bucket details for OpenAPI Schema file** - **accountstatusopenapi**
4. Create the Bedrock Agent from console - **bankofchicago-agent**
5. Update the Lambda function Permissions with Agent Invocation Details
6. Update the Lambda Function Code to allow Bedrock Agent to send request/response in a defined format - [Link](#)
7. Create Bedrock Knowledge Base (RAG solution) & update the details in the Action Group – S3 Bucket -**bankingagentknowledgebase**
8. Test the Agent



## Step 2 – AWS Lambda Function creation Steps

---

#1 Import boto3 and create client connection with DynamoDB -

<https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb.html>

#2 Print event value and store the event details in a variable

#3 Create a request syntax to retrieve data from the DynamoDB Table using GET Item method -

[https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb/table/get\\_item.html](https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb/table/get_item.html)

#4 Store and print the response

#5 Format the response as per the requirement of Bedrock Agent Action Group -

<https://docs.aws.amazon.com/bedrock/latest/userguide/agents-lambda.html>

#6 Print the final response

## Step 3 – Define OpenAPI schemas for your agent's action groups

---

1. Refer to the AWS sample OpenAI schemas - <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-api-schema.html>
2. Convert to YAML
3. Modify the Schema as per the requirement of the use case
4. Create an S3 bucket and store the Open API Schema in the S3 bucket

# Step 4 : Update the Lambda function Permissions for Agent Invocation

---

## Permission for Agent to invoke Lambda function

- bedrock-agent
- bedrock.amazonaws.com
- ARN
- Lambda:InvokeFunction

## Agent Instructions

You are an banking assistant in a Retail Bank. You are friendly and polite. You help resolve customer queries by providing bank customers status on their new bank accounts.

# *Python Refresher*

*(from my Udemy Course on :*

*AWS Lambda, Python(Boto3) & Serverless-*

*Beginner to Advanced)*

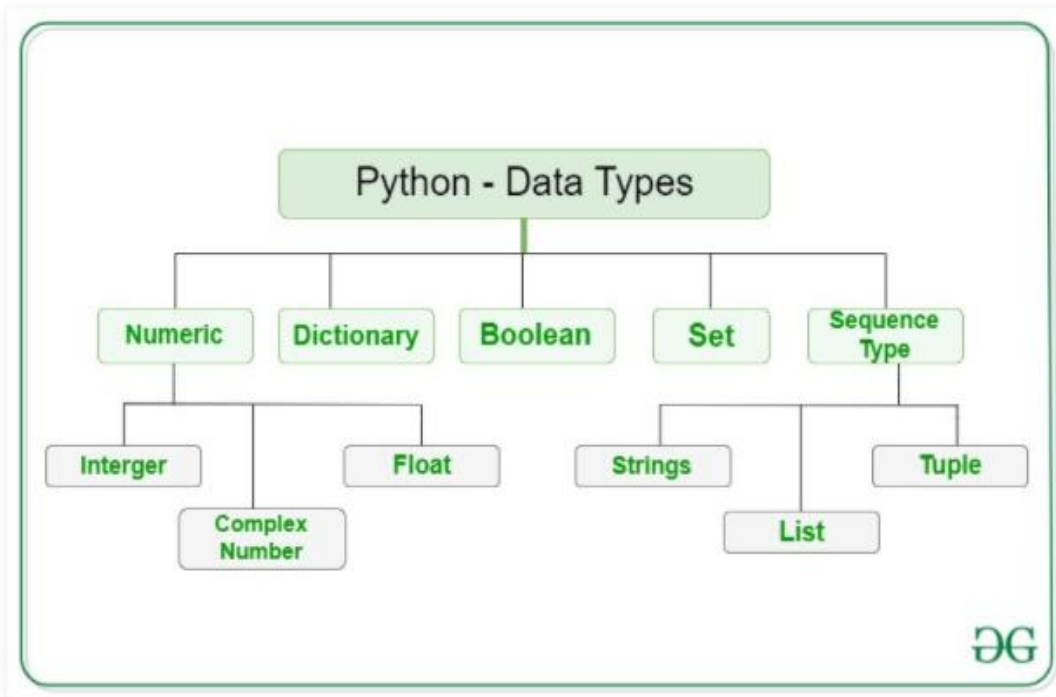
# Python Basics–Refresher



1. Print Function – print the message to screen or any interface; Syntax : `print( )`

2. Variables - Containers for storing data values string, float or integers and no need to declare; Syntax : `x = 3`, `greeting = "hello"` etc.

## 3. Data Type



| Data Type  | Example                                        |
|------------|------------------------------------------------|
| Int        | <code>x = 20</code>                            |
| Float      | <code>x = 20.5</code>                          |
| Dictionary | <code>x = {"name" : "John", "age" : 36}</code> |
| Strings    | <code>x = "Hello World"</code>                 |
| List       | <code>x = ["apple", "banana", "cherry"]</code> |

# Python Basics—Refresher

## 3. Data Types – Dictionary



- curly brackets
- key: pair values
- Nested Dictionary

### Dict

```
response = {1: 'Rahul', 2: 'John', 3: 'Joy'}
```

### Nested Dict

```
response = {1:'Python', 2:{'books': 'arch' , 'aws':'Lambda'}}
```

## 4. Nested Dictionary

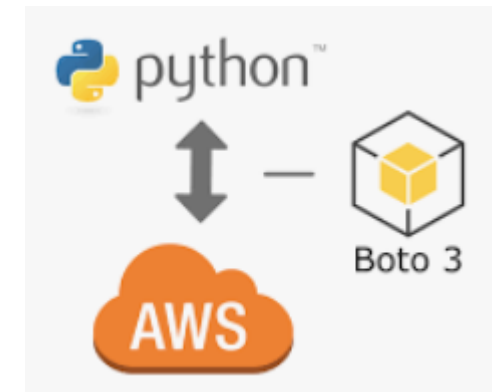


# Python Basics–Refresher

## 5. Sample Example from Boto3

### Response Syntax

```
{
  'Buckets': [
    {
      'Name': 'string',
      'CreationDate': datetime(2015, 1, 1)
    },
  ],
  'Owner': {
    'DisplayName': 'string',
    'ID': 'string'
  }
}
```



[https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/list\\_buckets.html#](https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/list_buckets.html#)

# Python Basics–Refresher

## 6. Data Types – List

- Lists in Python can be created by just placing the sequence inside the **square brackets []**
- A single list may contain Data Types like Integers, Strings, as well as Objects.
- List in Python are ordered and have a definite count. The elements in a **list are indexed with 0 being the first index.**
- **slice(start, stop, step)**
- Reverse [ : : -1]

```
.....  
list = [1, 4, 'For', 6, 'Anisha']  
.....
```

## 7. Nested List

**Nested List** => `nestedList = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`



# Python Basics–Refresher

## 8. Data Types – List and Dictionary



## 9. Data Type Determination

```
response = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
print(type(response))
```

[https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/list\\_buckets.html#](https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/list_buckets.html#)

### Response Syntax

```
{  
  'Buckets': [  
    {  
      'Name': 'string',  
      'CreationDate': datetime(2015, 1, 1)  
    },  
  ],  
  'Owner': {  
    'DisplayName': 'string',  
    'ID': 'string'  
  }  
}
```



# Python Basics—Refresher

## 10. Function

### Python Function

A function is a block of code which runs when it is called.

#### Syntax:

```
def function_name (argument/parameters):
```

```
    return expression or value
```

#### Example

# A simple Python function to check whether x is even or odd

```
def evenOdd(x):
```

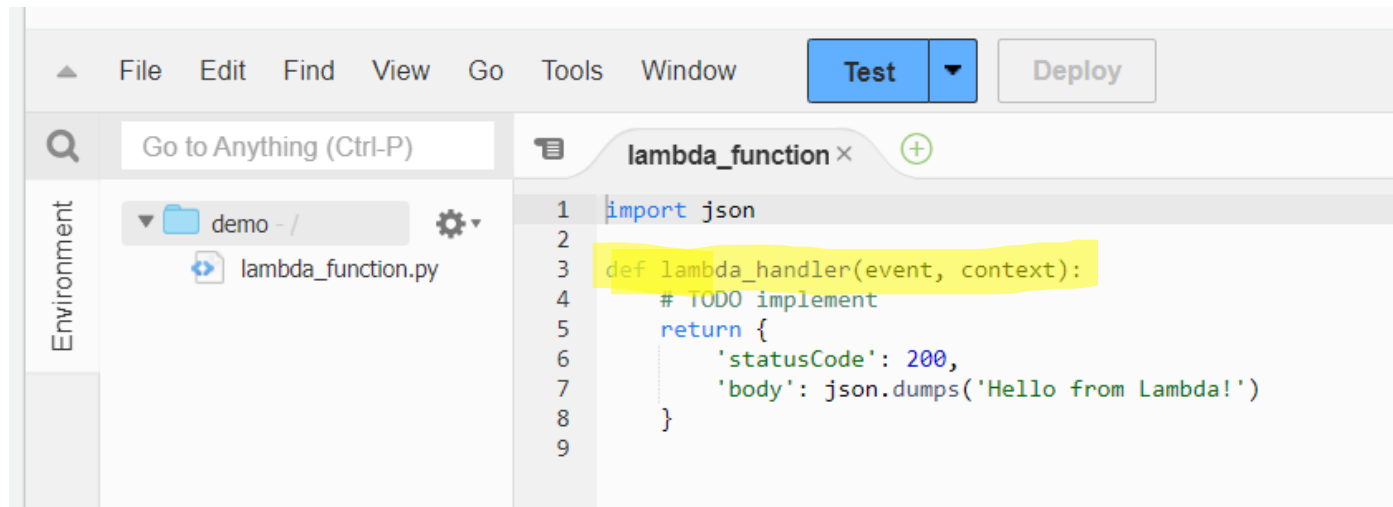
```
    if (x % 2 == 0):  
        print("even")
```

```
    else:  
        print("odd")
```

# Driver code to call the function

```
evenOdd(2)
```

```
evenOdd(3)
```



# *API Gateway Refresher*

*(from my Udemy Course on :*

*AWS Lambda, Python(Boto3) & Serverless-*

*Beginner to Advanced)*

# AWS API Gateway - Overview

## What is API Gateway

Amazon API Gateway is an AWS service for **creating, publishing, maintaining, monitoring, and securing REST, HTTP, and WebSocket APIs** at any scale.



Sample Architecture - RESTful microservices



# AWS API Gateway - Overview

## Key Features Of API Gateway

### 1. API Types

#### 1. HTTP API

Build **low-latency and cost-effective REST APIs** with built-in features such as OIDC and OAuth2, and native CORS support.

#### 2. WebSocket API

Build a WebSocket API using **persistent connections for real-time use cases such as chat applications or dashboards.**

#### 3. REST API



**Develop a REST API** where you gain complete control over the request and response along with **API management capabilities.**

#### 4. REST API (Private)

Create a REST API that is **only accessible from within a VPC.**

<https://aws.amazon.com/blogs/compute/introducing-amazon-api-gateway-private-endpoints/>

# AWS API Gateway - Overview

## Key Features of API Gateway

## REST API's vs HTTP API's

- REST APIs support **more features** than HTTP APIs, while HTTP APIs are designed with **minimal features** and offered at a **lower price**.
- Choose **REST APIs** if you need features such as **API keys, per-client throttling, request validation, AWS WAF integration, or private API endpoints.**
- Choose **HTTP APIs** if you don't need the features included with **REST APIs**.
  - <https://aws.amazon.com/blogs/compute/introducing-amazon-api-gateway-private-endpoints/>

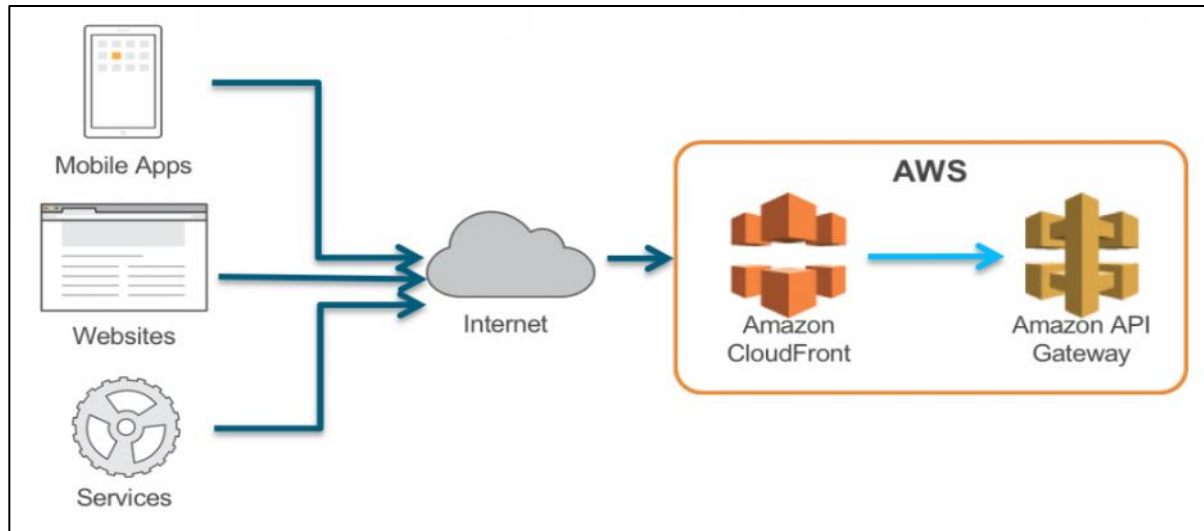
# API Gateway - Overview

## 2. API Endpoint Types

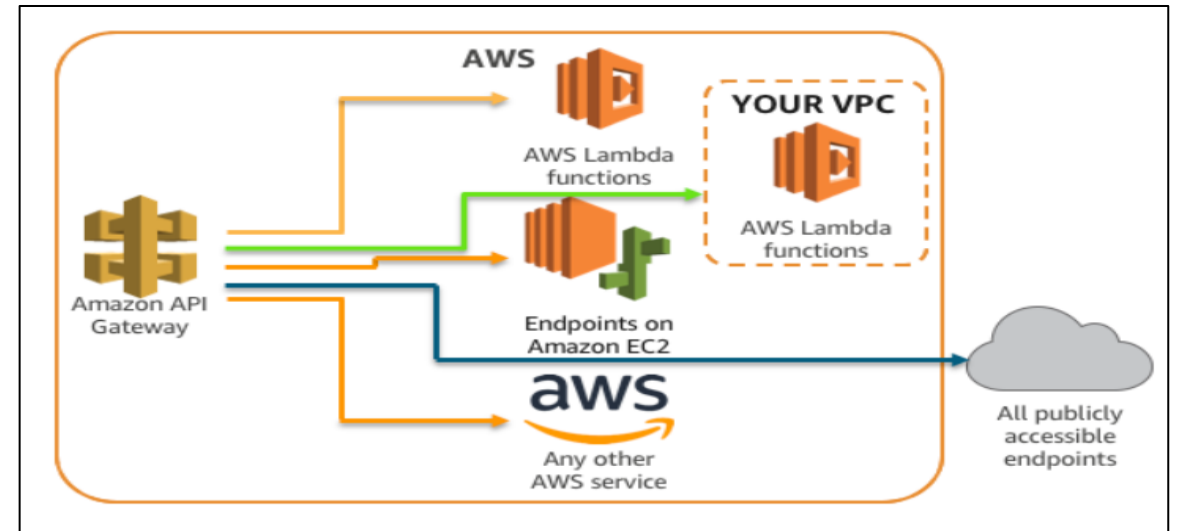
An **API endpoint type** refers to the **hostname** of the API.

The API endpoint type can be **edge-optimized**, **regional**, or **private**, depending on where the majority of your API traffic originates from.

### Edge Optimized



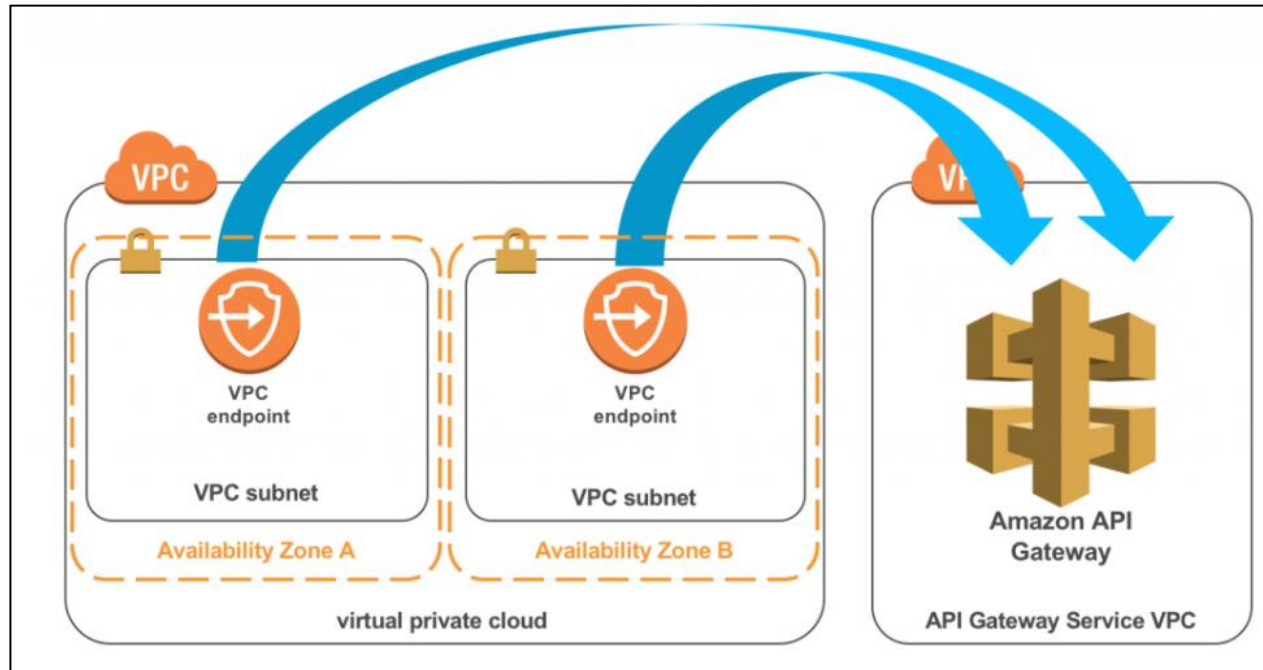
### Regional



# API Gateway - Overview

## 2. API Endpoint Types

### Private





# API Gateway - Overview

## 3. Resources



**REST architecture** treats every content as a **resource**.

These resources can be Text Files, Html Pages, Images, Videos or Business Data.

## 4. Methods



---

| Method | Description                                      |
|--------|--------------------------------------------------|
| GET    | Retrieve information about the REST API resource |
| POST   | Create a REST API resource                       |
| PUT    | Update a REST API resource                       |
| DELETE | Delete a REST API resource or related component  |

---

# API Gateway - Overview

## 5. Integration Type

### Lambda Function



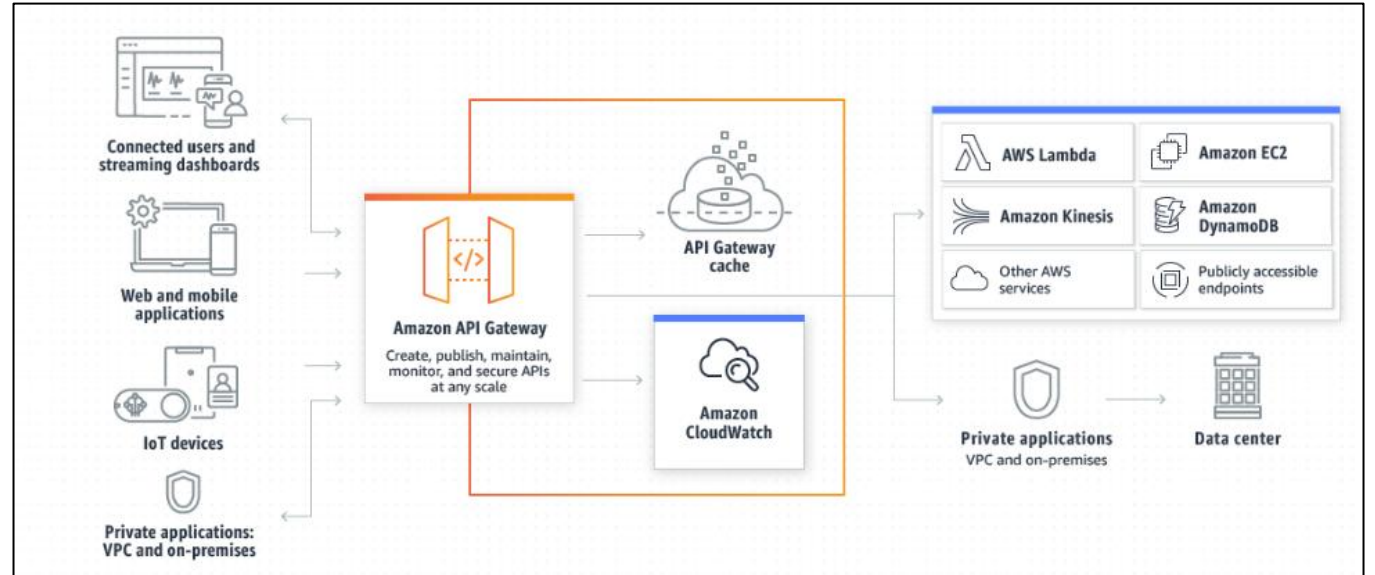
This type of integration lets an API method be integrated with the Lambda function

### HTTP

- Allows integration with existing HTTP endpoint
- This type of integration lets an API expose HTTP endpoints in the backend.

### Mock

- This type of integration lets API Gateway return a response without sending the request further to the backend.



# API Gateway - Overview

## 5. Integration Type

### **AWS Service**

This type of integration lets an API expose AWS service actions.

### **VPC Link**

A VPC link is a resource in Amazon API Gateway that allows for connecting API routes to private resources inside a VPC

# API Gateway - Overview

## 5. Deployment

After **creating your API**, you must **deploy it** to make it callable by your users.

To **deploy an API**, you **create an API deployment** and **associate it with a stage**.

**Important Tip :**

**Every time you update an API, you must redeploy the API to an existing stage or to a new stage.**

Updating an API includes modifying routes, methods, integrations, authorizers, and anything else other than stage settings.

## 6. Stages

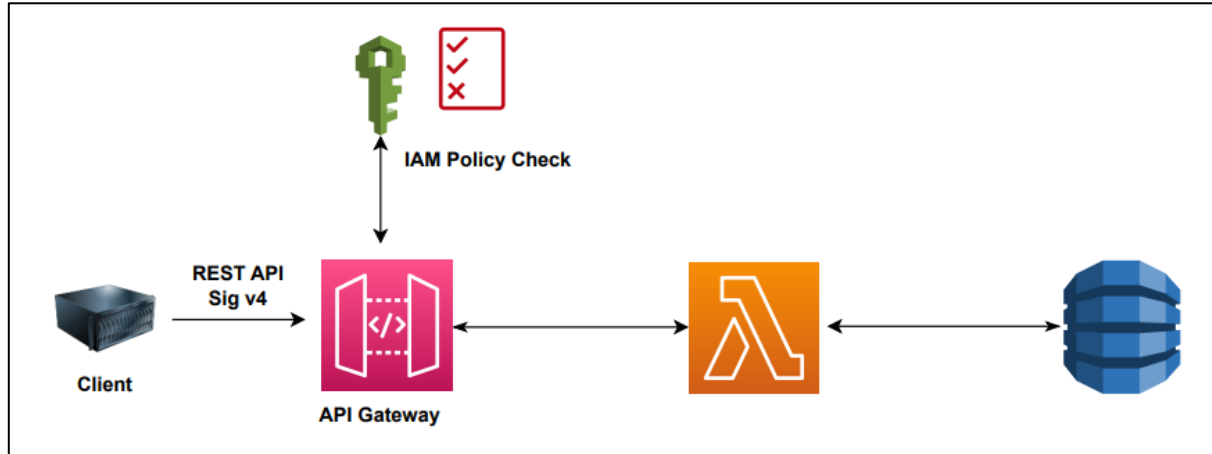
A logical reference to a **lifecycle state of your API** (for example, 'dev', 'prod', 'beta', 'v2').

<https://{restapi-id}.execute-api.{region}.amazonaws.com/{stageName}>

<https://ent94mc14j.execute-api.us-east-1.amazonaws.com/dev/students>

# API Gateway - Overview

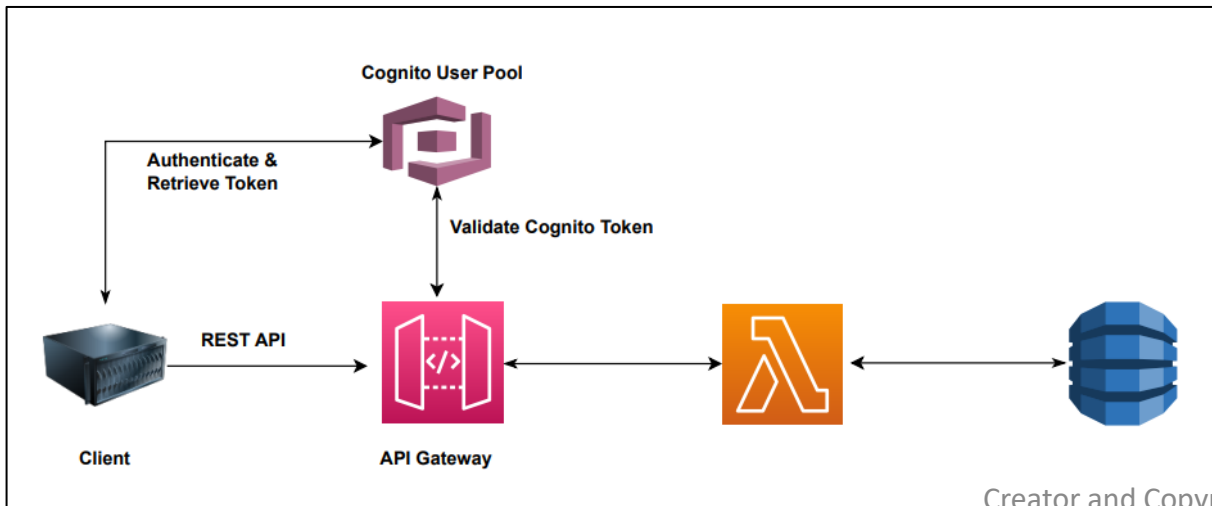
## 7. AWS API Gateway - Authentication and Authorization



### 1. API Security – IAM

Authentication – IAM  
Authorization – IAM Policy

**Use Case :** Internal AWS Services, Cross Account Access



### 2. API Security – Cognito

Authentication – Cognito User Pool  
Authorization – API Gateway Methods

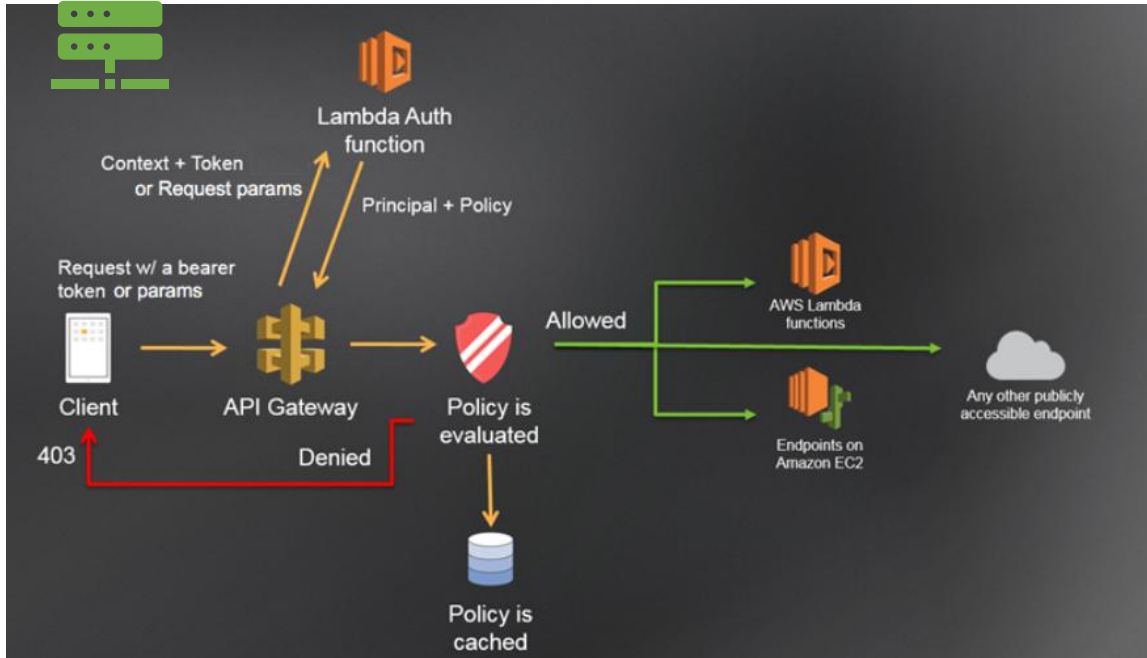
**Use Case :** External Users for Web/Mobile Apps

# API Gateway - Overview

## 7. AWS API Gateway - Authentication and Authorization



3<sup>rd</sup> Party IDP



### 3. API Security – Lambda Authorizer

Authentication – External

Authorization – Lambda Function

**Use Case :** Third Party Identity Provider such as OAuth 2.0

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html>

# API Gateway - Overview

## 8. Usage Plans



### Usage Plans

**Use Case :** Differentiate between Basic and Premium Customers

Sets the **target request rate - Throttling, Burst and Quota Limit for each API Key**

## 9. API Keys



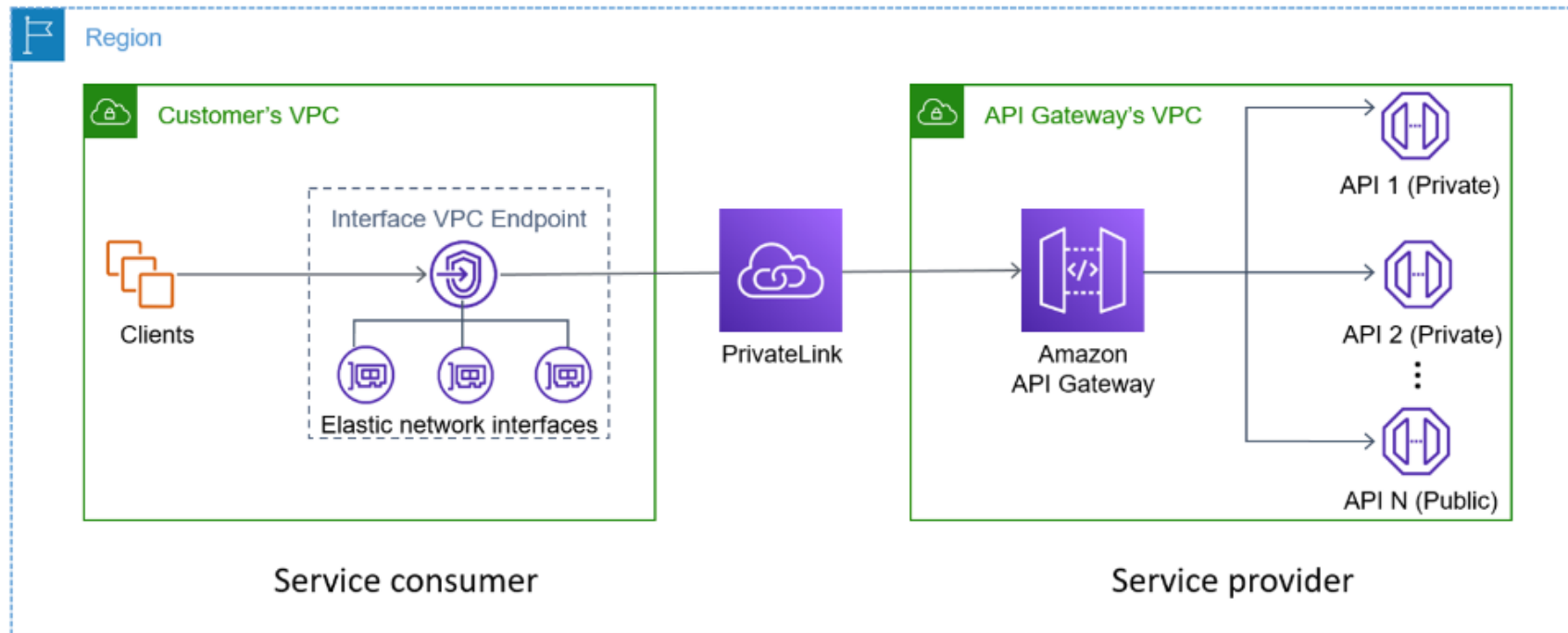
- An **alphanumeric string that API Gateway uses to identify an app developer** who uses your REST or WebSocket API.
- You can use API keys together with [Lambda authorizers](#) or [usage plans](#) to control access to your APIs.

# API Gateway - Overview

## 10. API Integration and Private API's

### Private API's

A private API means that the **API endpoint is reachable only through the VPC**. Private APIs are **accessible only** from **clients within the VPC** or from clients that have **network connectivity to the VPC**.



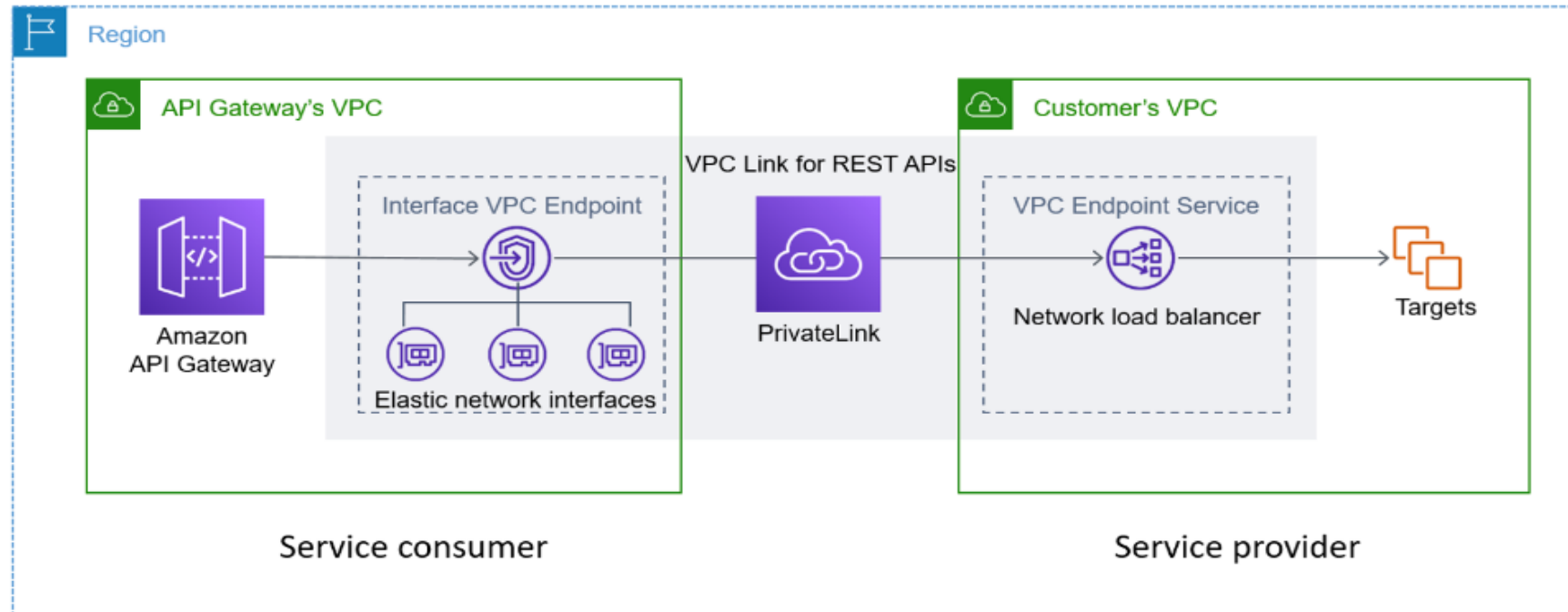


# API Gateway - Overview

## 10. API Integration, Private API's and VPC Link

### Private Integration

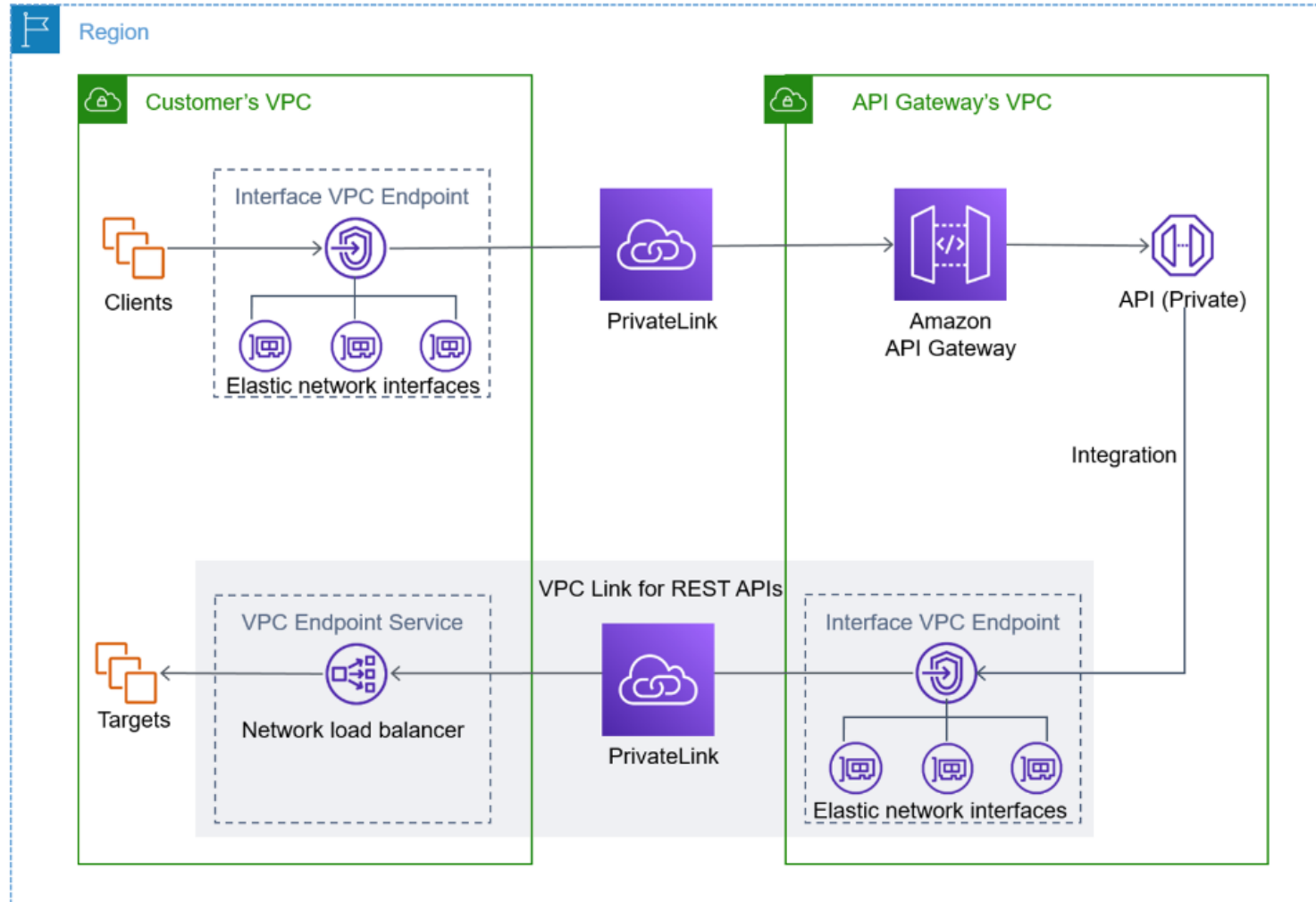
An API Gateway integration type for a client to access resources inside a customer's VPC through a private REST API endpoint without exposing the resources to the public internet.



Source : <https://aws.amazon.com/blogs/compute/understanding-vpc-links-in-amazon-api-gateway-private-integrations/>

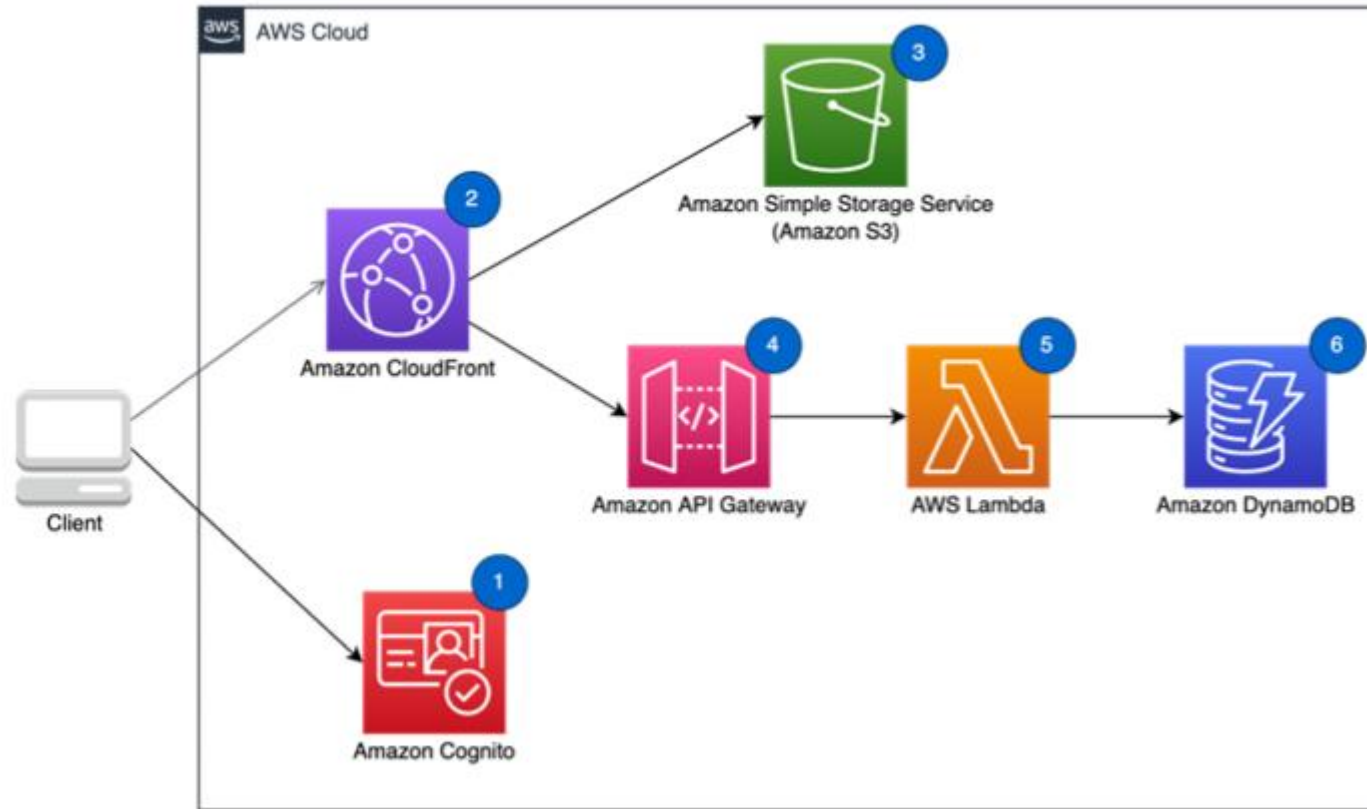
# API Gateway - Overview

## 10. API Integration and Private API's



# API Gateway - Overview

## Sample Architecture



**Thank You**